

Architect — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/Architect`. Canonical sources stay in place.

Phase — 01-architect-thinking

Directory: `01-architect-thinking/`

Source: `01-architect-thinking/00-orientation.md`

Architect Path

Šta ovo gradi

Principal/staff architect nivo. Ne memorisanje — razumevanje dovoljno da znaš šta da pitaš i šta hoćeš.

Sa AI-jem ti ne trebaš detalje. Trebaš: - Mentalni model koji prepoznaje pravi problem - Vocabulary da komuniciraš precizno - Decision framework za tradeoffe - Komunikacijske veštine da odbraniš odluku

Svaki modul: decision table + anti-patterns + šta da pitaš AI + lab.

Moduli

#	Oblast	Problem koji rešava
01	<code>01-architect-thinking/</code>	Kako donositi odluke pod pritiskom, sa nepotpunim informacijama
02	<code>02-system-boundaries/</code>	Kada deliti sistem, kada ne — i kako to obrazložiti
03	<code>03-data-and-storage/</code>	Koji storage za koji problem; migracije bez downtime-a
04	<code>04-async-events-queues/</code>	Kada async, ko garantuje šta, kako ne izgubiti podatke
05	<code>05-reliability-failure/</code>	Kako sistem degraduje, SLO/SLI praktično, cascading failures
06	<code>06-scaling-performance/</code>	Bottleneck-first skaliranje, capacity math, caching kao odluka
07	<code>07-communication-decisions/</code>	ADR, stakeholder narrative, odbrana odluke pod pritiskom
08	<code>08-capstone-drills/</code>	6 realnih scenarija + referenca šta da pitaš AI
09	<code>09-api-design/</code>	REST vs gRPC vs GraphQL, versioning, breaking changes, idempotency
10	<code>10-security-boundaries/</code>	Trust boundaries, auth/authz placement, secrets, audit log
11	<code>11-observability/</code>	Metrics/logs/traces kao arhitekturna odluka, SLO alerting
12	<code>12-deployment-architecture/</code>	Deployment strategije, DB migracije u deploymentu, rollback design
13	<code>13-network-layer/</code>	Load balancers, TLS termination, DNS, CDN — gde komponente stoje
14	<code>14-technology-evaluation/</code>	Framework za evaluaciju tehnologija, PoC dizajn, TCO

Redosled: 01 → 02 → 03 → 04 → 05 → 06 → 07 → 09 → 10 → 11 → 12 → 13 → 14 → 08. Modul 08 (capstone drills) na kraju — ali `08/02-what-to-ask-ai.md` možeš koristiti odmah kao referencu. Moduli 01 i 07 su najvažniji.

Referentni sistem

Spine kroz sve module: `Symfony API` → `Redis queue` → `Go worker` → `Postgres`

Primeri koriste ovaj stack. Ako radiš sa drugim stackom, principi su isti — zameni komponente.

Kako koristiti

Svaki modul ima `01-*.md` (framework + decision tables) i `02-lab-*.md` (vežbe).

Radi labove. Arhitektura se uči kroz odluke, ne kroz čitanje. Kada naiđeš na pravi problem u radu, vrati se na relevantan modul kao referencu.

Modul 08 (`08-capstone-drills/02-what-to-ask-ai.md`) je referenca koju možeš koristiti odmah — pre nego što završiš ostale module.

Source: `01-architect-thinking/01-thinking-framework.md`

Architect thinking framework

The shift from senior engineer to architect is not about knowing more technology. It is about changing what question you ask first.

Senior engineer: “How do I build this?” Architect: “Should we build this, and if so, which version of it, given what we cannot change?”

Constraints-first

Before proposing any solution, list what constrains the solution space. Engineers jump to solutions. Architects start with constraints because a solution that violates a constraint is not a solution.

Constraint categories to always audit:

Category	Examples
Team	2 backend devs, no dedicated ops, no Kubernetes experience
Ops maturity	Manual deploys, no observability beyond logs, no on-call rotation
Existing stack	Symfony API, Go worker, Postgres 14, Redis, RabbitMQ
Budget	No new paid services, or \$X/month ceiling
Compliance	GDPR data residency, PCI scope, audit trail requirements
Timeline	Must ship in 6 weeks, cannot touch payment flow before Q4
Reversibility	API contract already published to 3 clients, cannot break it

Rule: Any option that violates a hard constraint is eliminated before evaluation. Write the constraints down before opening your editor or talking to AI. If you cannot list at least 4 constraints, you have not understood the problem.

Failure-first thinking

For any design, the first question is: “What breaks first and under what conditions?” — not “does this work?”

Happy-path thinking produces systems that work in demos and fail in production. Failure-first thinking produces systems that degrade gracefully.

For every component, ask three questions: 1. What is the failure mode? (not “if it fails” — it will fail, ask how) 2. What does the user observe when this happens? 3. What does the system do without human intervention?

Applied to the reference stack:

Component	Failure mode	User observes	System behavior without intervention
RabbitMQ queue	Queue full (backpressure)	API returns 503 or hangs	Messages dropped or publisher blocks
Go worker	Crash mid-processing	Order stuck in “processing”	No retry unless supervisor restarts
Postgres	Slow queries (not down)	API latency spike, timeouts	Connection pool exhausted, cascading timeout
Redis	OOM eviction	Cache misses, DB spike	Depends on eviction policy — silent failure

Design decisions flow from this table, not from the happy path. If the worker crashes mid-processing, you need idempotent message handling and dead-letter queues before you need anything else.

Options before decisions

Never propose one solution. Always produce 2–3 options with explicit tradeoffs. The decision belongs to you or your team. The options are the architect’s core contribution.

Format for each option:

```
Option N: [Name]
What it is: one sentence
Cost: implementation time, infrastructure cost, ops overhead
What it forecloses: what you can no longer do easily if you pick this
Risk: what can go wrong with this choice
```

Why “do nothing” is always an option: it is often the correct one. Naming it forces an honest comparison. If “do nothing” is obviously worse, you can show that. If it is not obviously worse, you have found a scope problem.

Reversibility as a dimension

Every architectural decision sits on a spectrum from fully reversible to effectively irreversible.

Reversible (default to these when uncertain): - Adding a new API endpoint (does not break existing clients) - Adding a new queue consumer (additive) - Adding a Redis cache layer in front of Postgres - Changing internal service configuration

Irreversible (require explicit justification): - Postgres schema changes that remove or rename columns used by multiple consumers - Published API contracts (once clients depend on them, you own them) - Data model decisions that split or merge tables at scale - Choosing a message format for an event stream other services consume - Infrastructure choices that create vendor lock-in

Practice: Before finalizing a decision, ask: “If we discover this is wrong in 6 months, what does it cost to undo?” If the answer is “a coordinated multi-team migration and potential downtime,” that is an irreversible decision. Treat it accordingly — slow down, write the ADR, get more input.

The 3-year test

“Will the team that inherits this system in 3 years understand why this decision was made?”

This is not about documentation for its own sake. It is a quality check on the decision itself. If you cannot explain a decision clearly enough for a future engineer to understand the constraints you faced, one of two things is true:

1. The decision is not well-reasoned — you made it for unstated reasons (familiarity, momentum, “everyone does it this way”)
2. The decision is fine but needs an ADR

Use this as a filter. If a decision fails the 3-year test and you cannot write a clear ADR for it, reconsider the decision before writing anything down.

Decision-making process (step by step)

1. **Gather constraints** — list what cannot change before anything else. Budget, team, stack, compliance, timeline, reversibility constraints. Minimum 4.
 2. **State the problem precisely** — one sentence, observable outcome. Not “improve performance.” Instead: “The Go worker fails to process all orders within 30 seconds during peak load of 500 orders/minute, causing user-visible stuck orders.”
 3. **Generate 2–3 options including “do nothing”** — each option should be distinct, not variations of the same approach. If your three options are all “use a different queue,” you have not generated real options.
 4. **For each option:** cost (time and money), what it forecloses, what it risks. Fill in the format above. No option is presented without all three.
 5. **Make a decision and write an ADR** — the ADR records: what was decided, why, what was rejected and why, and what would change the decision. Date it.
-

Anti-patterns

“The modern way to do this is X” — Modernness is not a constraint. It is not a reason. RabbitMQ is “older” than Kafka. It is also the right choice for most teams under 20 engineers. Defend choices with constraints, not with currency.

Designing for scale you do not have — Designing a Symfony API for 10,000 requests/second when current peak is 50 is engineering theater. It costs real complexity for imaginary load. Scale when you have evidence of need.

Solving the problem you imagine instead of the problem you have — “Users complain about slowness” does not mean the database is slow. Measure before designing. State the observable problem in step 2 before generating options.

One option presented as “the only reasonable choice” — This is either groupthink, salesmanship, or incomplete thinking. If you truly cannot generate a second option, you do not understand the problem space well enough yet.

Premature abstraction — Building a plugin system, an event bus, or a generic framework before you have two concrete use cases. Wait for the second use case before abstracting. This applies to microservice splits: one service split is speculative, three service splits with a shared pain point is architecture.

Consensus by exhaustion — Holding design discussions until the most vocal person wins is not a decision process. Options + criteria + explicit decision is.

What to ask AI

These prompts get architectural output, not tutorial output.

For generating options:

“Given these constraints: [list them explicitly], what are 3 options for [state the problem precisely]? For each option: implementation complexity (days), operational overhead (ongoing), what it forecloses, and what would make it the wrong choice.”

For failure-first analysis:

“Given this design: [Symfony API → RabbitMQ → Go worker → Postgres], what are the top 3 failure modes? For each: how does it manifest to the user, does the system recover without intervention, and what is the minimal mitigation?”

For reversibility check:

“I am deciding between [option A] and [option B]. Which decision is harder to reverse in 18 months and why? What would make it irreversible?”

For the 3-year test:

“Here is my proposed ADR: [paste it]. What constraints or reasoning am I missing that a future engineer would need to understand this decision?”

For constraint discovery:

“I need to add [feature] to a Symfony API + Go worker + Postgres + Redis stack with a team of 3. What constraints am I likely not thinking about that typically eliminate options in this scenario?”

ADR format (minimal)

```
## ADR-NNN: [Title]

**Date:** YYYY-MM-DD
**Status:** Accepted

**Context:**
[One paragraph: what problem, what constraints, what was the forcing function]

**Decision:**
[What we decided, one sentence]

**Options considered:**
- Option A: [why rejected]
- Option B: [why rejected]
- Option C (chosen): [why chosen]

**Consequences:**
[What this enables, what this forecloses, what would change this decision]
```

The ADR is not bureaucracy. It is the artifact that makes the 3-year test passable.

Source: 01-architect-thinking/02-lab-constraints-and-failure.md

Lab: Constraints audit and failure-first design

Two exercises. Do Exercise 1 before reading the example answers. Do Exercise 2 before comparing against the failure table.

Exercise 1: Constraints audit

Scenario: “We need to add real-time notifications to our Symfony + Go worker app. Users should see when their order status changes.”

This is a real request from a stakeholder. Before proposing anything, your job is to audit the constraints.

Step 1 — Fill the constraints template

Copy this template. Fill it in before reading further.

CONSTRAINTS AUDIT

=====

Problem statement (one sentence, observable outcome):

Team constraints:

- Team size:
- Ops experience:
- On-call / alerting setup:

Stack constraints:

- Current stack:
- What we cannot change:
- Existing client contracts (API consumers):

Infrastructure constraints:

- Hosting model (bare metal / managed / k8s):
- Current monitoring:
- Budget ceiling for new services:

Compliance / data constraints:

- Data residency:
- PII in notifications:
- Audit requirements:

Timeline constraints:

- Hard deadline:
- What must not be disrupted:

Reversibility constraints:

- Which decisions here are hard to undo:

Minimum 6 constraints before proceeding. If you have fewer than 6, you have not understood the problem.

Step 2 — Generate 3 options

With constraints filled in, generate 3 options for delivering real-time order status notifications.

For each option: - Name it - One sentence: what it is - Implementation complexity (days, rough) - Ops overhead (ongoing) - What it forecloses - Which constraints eliminate it (if any)

Step 3 — Eliminate options by constraints

For each option, mark: **VIABLE**, **ELIMINATED BY [constraint]**, or **RISKY — requires [mitigation]**.

Example answers — Exercise 1

Read only after completing your own audit.

Plausible constraints for this scenario (assuming a 3-person team, no dedicated ops):

1. Team has no experience operating WebSocket servers at scale — SSE or polling is lower ops risk
2. Stack is Symfony (PHP) + Go worker; no Node.js or dedicated realtime service in production
3. No current on-call rotation — silent failures in push channels will not be noticed quickly
4. GDPR applies — notification payload cannot leak order details from other users (multi-tenant check)
5. RabbitMQ already in stack — event publishing is already possible without new infrastructure
6. API already has Symfony consumers; changing the API contract for websocket upgrade would break existing clients
7. Timeline: 6 weeks — no time to introduce and stabilize a new managed service (e.g., Pusher, Ably)
8. Budget: no new paid services approved this quarter

Three options:

Option A: Server-Sent Events (SSE) from Symfony API - What it is: Long-lived HTTP connections from the browser to a Symfony endpoint that pushes events - Complexity: 3–5 days - Ops overhead: Connection count monitoring, PHP worker limits

(fpm config), sticky sessions if load balanced - Forecloses: Easy horizontal scaling (SSE connections are stateful per server) - Constraint check: RISKY — fpm worker exhaustion under load; manageable with low user count

Option B: Polling from the frontend every 5–10 seconds - What it is: Frontend calls a Symfony endpoint to fetch latest order status - Complexity: 1 day - Ops overhead: Minimal; just another API endpoint with Redis cache - Forecloses: True real-time (5–10 second lag); not viable if latency requirement is sub-second - Constraint check: VIABLE if latency requirement is “near real-time” not “instant”

Option C: Managed WebSocket service (Pusher/Ably) - What it is: Events published from Go worker → managed push service → browser via WebSocket - Complexity: 3–4 days - Ops overhead: External dependency, vendor account management, cost per connection - Forecloses: Full control of delivery guarantees, cost scaling - Constraint check: ELIMINATED — budget constraint (no new paid services this quarter) + timeline risk of new vendor onboarding

Result: Option C is eliminated by constraints. Option B is viable immediately. Option A is viable with a risk mitigation note on fpm capacity. This is an architect’s output — not “we should use WebSockets.”

Exercise 2: Failure-first design

Given stack: Symfony API → RabbitMQ queue → Go worker → Postgres

For each component below, answer all three questions before reading the failure table.

Questions for each component: 1. What is the failure mode? 2. What does the user observe? 3. What does the system do without human intervention?

Components to analyze: - Queue full (RabbitMQ backpressure / max-length reached) - Go worker crashes mid-processing - Postgres slow (not down — queries taking 5–30x normal time) - Symfony API timeout on queue publish (RabbitMQ unreachable or slow ACK)

Fill in this table:

FAILURE TABLE			
=====			
Component	Failure mode	User observes	System behavior (no human)
-----	-----	-----	-----
Queue full			
Go worker crash mid-proc			
Postgres slow (not down)			
API timeout on publish			

Failure table — example answers

Read only after completing your own table.

Component	Failure mode	User observes	System behavior without intervention
Queue full	RabbitMQ hits max-length; publisher gets channel error or message rejected	API returns 500 or hangs (depending on publish timeout config); order appears to not submit	Without dead-letter queue: messages dropped silently. With DLQ: messages routed to DLQ, still not processed. Queue does not self-drain.
Go worker crash mid-processing	Worker process exits during message processing (panic, OOM, signal)	Order stuck in “processing” state indefinitely	Message is un-acked, returns to queue after visibility timeout (if configured). Without requeue config: message lost. With supervisor restart: worker restarts but same message may reprocess — requires idempotency.
Postgres slow (not down)	Long-running queries block connection pool; pool exhausts	API latency spikes; requests start timing out; users see errors on read-heavy pages first	Connection pool fills; new requests queue then fail. Read replica (if exists) may absorb reads. Without intervention: cascading timeout — API, worker, everything that touches DB starts failing. Self-recovery: only if slow queries finish and pool drains.
API timeout on publish	Symfony cannot reach RabbitMQ (network partition, RabbitMQ restart, slow ACK)	Order submission fails; user sees error or spinner timeout	Without circuit breaker: every API request that publishes hangs for full timeout (e.g., 30s), exhausting PHP-FPM workers rapidly. With circuit breaker: fast fail after threshold, returns 503. Without either: full API degradation within seconds at moderate load.

What the failure table tells you

Each row is a design requirement, not just an observation.

Queue full requires: - Dead-letter queue configuration - Max-length policy that rejects (not silently drops) - Alerting on queue depth before it hits max

Worker crash mid-processing requires: - Idempotent message processing (safe to reprocess) - Message requeue on worker crash (RabbitMQ nack + requeue=true) - Dead-letter queue for messages that fail repeatedly - Supervisor or container restart policy

Postgres slow requires: - Connection pool sizing with a ceiling (pgBouncer or application-level) - Query timeout at the application layer (not just DB) - Alerting on pool saturation, not just query count - Separate read replica for read-heavy operations if write-path isolation matters

API timeout on publish requires: - Publish timeout configured short (500ms–1s, not 30s) - Circuit breaker on the RabbitMQ client - Fallback: synchronous DB write to an outbox table if publish fails - Or: accept the 503 and let the client retry — simpler, fewer moving parts

None of these are optional polish. They are the system. The happy path is already obvious. This table is the architecture.

Debrief checklist

After both exercises, check:

- ☐ Did you fill in constraints before generating options?
- ☐ Did any of your options get eliminated by your own constraint list?
- ☐ Is your problem statement one sentence with an observable outcome?
- ☐ Does your failure table have a row for each component, not just the most obvious one?
- ☐ Did the failure table change any of your option choices from Exercise 1?

If the failure table changed your options: good. That is the point. Failure-first analysis is a design input, not a retrospective.

What to ask AI for these exercises

Constraint discovery:

"I need to add real-time order status notifications to a Symfony + Go worker app. Team size: 3. No dedicated ops. Stack: Symfony, Go, Postgres, Redis, RabbitMQ. What constraints am I likely overlooking that would eliminate options before I evaluate them?"

Option generation:

"Given these constraints [paste list], generate 3 options for real-time order notifications. For each: implementation complexity in days, ops overhead, what it forecloses, and which of my constraints eliminates it."

Failure analysis:

"For this stack: Symfony API → RabbitMQ → Go worker → Postgres, give me a failure table. For each component: the realistic failure mode, what the user observes, and what happens without human intervention. Include Postgres slow (not down) as a distinct case from Postgres down."

Turning the failure table into requirements:

"Here is my failure table: [paste it]. For each failure mode, what is the minimal mitigation that a 3-person team with no dedicated ops can actually maintain? Rank by user impact."

Phase — 02-system-boundaries

Directory: 02-system-boundaries/

Source: 02-system-boundaries/01-when-to-split.md

When to Split

The most expensive architectural mistake is splitting too early. The second most expensive is not splitting when you should have. Most teams make the first mistake.

The Split Decision Table

Split when you have a concrete, present-day reason. Not a hypothetical future one.

Condition	Justifies Split	Notes
Independent deployment required	Yes	Different teams need to ship without coordinating release windows
Different scaling requirements (proven, not projected)	Yes	One component is CPU-bound at 10x the rate of another
Different team ownership with explicit boundaries	Yes	Two product teams, two codebases, two on-call rotations
Different failure blast radius needed	Yes	Payments going down should not take the catalog down
Different compliance/security scope	Yes	PCI DSS scope creep is real and expensive — isolate it
“It’s getting big”	No	Size is not a reason. Complexity inside a module is a reason.
“Microservices are what everyone does”	No	Social proof is not an architectural argument
“We might need to scale this later”	No	Scale when the load exists. Premature scaling is waste.
“This feels like a separate concern”	No	Separate concern → separate module, not separate service
“We want to use a different language/framework for this”	Weak	Tech diversity adds ops cost. Needs strong justification.

Monolith → Modular Monolith → Microservices

This is not a progression. It is a choice. Each point on the spectrum has a cost profile.

Monolith - One deployment unit, one database, one process - Ops cost: minimal — one thing to deploy, monitor, debug - Team coordination: high coupling if modules not enforced at the code level - Data consistency: trivially strong — everything in one transaction - Debugging: stack traces are complete, no distributed tracing needed - Right for: teams under 10 engineers, domains not yet well understood, early products

Modular Monolith - One deployment unit, one (or few) databases, enforced internal module boundaries - Ops cost: minimal — still one deployment - Team coordination: modules give teams ownership without service overhead - Data consistency: strong — still in-process, but modules don’t share each other’s tables directly - Debugging: still simple - Right for: most systems. This is the under-used option. It gives 80% of the organizational benefit of microservices at 20% of the ops cost.

Microservices - N deployment units, N databases, N on-call rotations - Ops cost: high — service mesh, distributed tracing, circuit breakers, retry budgets, schema registries - Team coordination: explicit API contracts required, versioning required - Data consistency: eventual by default — distributed transactions (saga, 2PC) are painful - Debugging: requires correlation IDs, centralized logging, distributed tracing from day one - Right for: large orgs with well-understood domain boundaries, independent team deployment velocity is the primary constraint

Conway’s Law, Practically

Organizations design systems that mirror their communication structures.

This is not a suggestion. It is an observation about what will happen whether you plan for it or not.

If you want a service boundary between Orders and Payments, you need a team boundary between Orders and Payments. If both are owned by one team, the boundary will erode. Shared standups, shared backlogs, and convenience couplings will do it.

What this means in practice: - Don't design service boundaries that your org chart can't support - If you're a 4-person backend team, you do not have the team topology for 8 microservices - If you want the technical boundary, first establish the organizational boundary (separate squad, separate roadmap, separate PagerDuty rotation) - The inverse is also true: if two teams are sharing one service, expect that service to become two services eventually, whether you plan it or not

Seam Identification

How to find natural split points in an existing system:

Data ownership: Which tables does each domain write to? If Domain A writes to Domain B's tables, they are not separate domains — they are one domain with confusing naming. A seam exists where a domain owns its writes and other domains can only read through an API or event.

Transaction boundaries: Where do your database transactions end? If paying for an order and reserving inventory are in the same transaction, splitting them requires a saga. That is real complexity. Find where transactions don't cross.

Deployment frequency: Which parts of the system change together? If Catalog and Orders are always deployed together because they share code, they aren't a natural split. If Notifications has never caused a revert of an Orders deploy, it has a natural boundary.

Team ownership: Who is on-call for what? Where is the blame game when something breaks? Natural ownership lines are natural service lines.

Failure characteristics: Which components crash independently today? Which bring each other down? A component that crashes alone and recovers alone is a candidate for extraction. A component that always takes 3 other things with it when it fails is deeply coupled.

The Modular Monolith as a Final Answer

A modular monolith is not a stepping stone. For many systems, for years, it is the correct architecture.

Symfony's module system, bounded contexts enforced through namespacing and explicit interfaces between modules, a single Postgres database where modules own their schema but don't query each other's tables directly — this gives you:

- Independent development and testing of modules
- Clear ownership and blast radius control in the codebase
- The ability to extract a module to a service later, when you have the operational maturity and team structure to support it
- None of the distributed systems tax in the meantime

The question is not “will we eventually need microservices?” The question is “do we need microservices now, given our team size, load, and failure isolation requirements?”

Decision Table: Team/Load → Architecture

Team Size	Monthly Deploys	Scaling Gap Between Components	Failure Isolation Needed	Recommendation
< 6 engineers	Any	None proven	No	Monolith
< 6 engineers	Any	None proven	Yes (e.g. payments)	Modular monolith
6–20 engineers	< 20/month	None proven	Partial	Modular monolith
6–20 engineers	> 20/month	Proven on 1–2 domains	Yes	Modular monolith + 1–2 extracted services
> 20 engineers	High	Multiple proven gaps	Yes, per domain	Microservices where justified

Anti-Patterns

Splitting by technical layer, not domain. `AuthService`, `NotificationService`, `EmailService` are not domain services — they are infrastructure concerns. They cross-cut every feature. Splitting them creates circular dependencies or creates a god-service that calls everything.

Microservices with a shared database. If two “services” write to the same database schema, you have a distributed monolith: you got all the ops cost of microservices and none of the isolation benefits. Schema changes are still coordinated. Transactions still cross service boundaries. Deployments are still coupled.

Splitting before understanding domain boundaries. You find domain boundaries by running the system and watching where the seams appear. If you’ve been operating the system for six months and you still don’t know which team owns the Order entity, splitting will not clarify that. It will just make the confusion more expensive.

Šta da pitaš AI

These prompts produce useful analysis. Paste your actual system context where indicated.

- “We have [describe system: main entities, who writes to what, team structure, current load]. Which parts have independent scaling requirements? Which have different failure blast radii? List the ones that justify splitting and the ones that don’t.”
 - “Draw the data ownership boundaries in this system: [paste schema or entity list]. Which entities are exclusively written by one domain? Which are written by multiple domains? Where are the shared write conflicts?”
 - “If we had to split this system in 6 months, where would the natural seams be given this team structure: [describe teams, ownership, on-call]? What organizational changes would need to happen before the technical split?”
 - “We are considering splitting [component] into its own service. Apply the split decision criteria: independent deployment need, scaling gap, team ownership boundary, failure isolation, compliance scope. What is the case for and against?”
-

Source: `02-system-boundaries/02-lab-boundaries.md`

Lab: Boundaries — Symfony E-Commerce Monolith

Spine project variant: Symfony monolith for an e-commerce platform. Apply the split decision framework from `01-when-to-split.md`.

System Context

Stack: Symfony, single Postgres database, no queue yet, no cache layer.

Team: 3 backend developers. No dedicated DevOps. Deployments are manual and happen 2–3 times per week, coordinated across the team.

Load: 50,000 orders/month (~1,700/day, ~70/hour at peak). Product catalog: 80,000 SKUs. No significant read/write imbalance currently.

Domains in the codebase (all in one Symfony app, mixed namespacing): - Product catalog: browse, search, pricing, categories - Inventory: stock levels, reservations, restocking triggers - Orders: cart, checkout, order state machine, fulfillment tracking - Payments: charge, refund, payment method storage - Notifications: email and SMS dispatch on order events - User accounts: registration, login, profile, address book

Schema (simplified):

<code>users</code>	– <code>id</code> , <code>email</code> , <code>password_hash</code> , <code>created_at</code>
<code>addresses</code>	– <code>id</code> , <code>user_id</code> , <code>street</code> , <code>city</code> , <code>country</code>
<code>products</code>	– <code>id</code> , <code>sku</code> , <code>name</code> , <code>description</code> , <code>price</code> , <code>category_id</code>
<code>categories</code>	– <code>id</code> , <code>name</code> , <code>parent_id</code>
<code>inventory</code>	– <code>id</code> , <code>product_id</code> , <code>warehouse_id</code> , <code>quantity</code> , <code>reserved_qty</code>
<code>orders</code>	– <code>id</code> , <code>user_id</code> , <code>status</code> , <code>total</code> , <code>created_at</code> , <code>fulfilled_at</code>
<code>order_items</code>	– <code>id</code> , <code>order_id</code> , <code>product_id</code> , <code>quantity</code> , <code>unit_price</code>
<code>payments</code>	– <code>id</code> , <code>order_id</code> , <code>provider</code> , <code>amount</code> , <code>status</code> , <code>provider_ref</code>
<code>notifications_log</code>	– <code>id</code> , <code>order_id</code> , <code>user_id</code> , <code>channel</code> , <code>template</code> , <code>sent_at</code>

Exercise 1: Map Data Ownership

For each table, answer: - Which domain **writes** to this table? - Which domains **read** from this table (and for what purpose)? - Is there a **write conflict** — more than one domain writing to the same table?

Use this template:

Table	Owned By (writes)	Read By	Write Conflict?
users			
addresses			
products			
categories			
inventory			
orders			
order_items			
payments			
notifications_log			

What to look for: Tables written by more than one domain are coupling points. They are either poorly named (the domains are actually one domain) or they represent a seam problem that a split would need to resolve with events or API calls.

Exercise 2: Apply the Decision Table

For each domain, evaluate against the split criteria from 01-when-to-split.md:

Domain	Independent Deploy Needed?	Different Scaling Req?	Different Team Owner?	Different Failure Blast Radius?	Different Compliance Scope?	Split Justified Now?
Product catalog						
Inventory						
Orders						
Payments						
Notifications						
User accounts						

Fill in Yes/No/Partial with a one-line justification per cell. The final column should follow from the others — not from intuition about what “feels” like a service.

Exercise 3: Design the Modular Monolith

Before any service split, the first architectural move is making the module boundaries explicit inside the monolith.

Design the internal module structure:

1. Name the modules (map the 6 domains to 4–6 modules — some may merge)
2. For each module, define what it **owns** (its tables, its write path)
3. Define what it **exposes** to other modules (public interface: method calls, events, or read-only query objects)
4. Define what it **must not do** (the constraints: no direct cross-module table joins, no writing to another module’s tables)

Format:

```
Module: [Name]
Owns: [tables]
Exposes: [interface description]
Must not: [constraint list]
```

Aim for 4–6 modules. If you end up with 6 modules that mirror the 6 domains exactly, ask whether any two modules are so tightly coupled that splitting them creates more interface overhead than value.

Exercise 4: The One Split You Would Make First

Given: - 3 developers, shared on-call - 50k orders/month - Manual deployments, 2–3x/week - No queue, no cache

Name **one domain** you would extract to a separate service if forced to split today. Apply the decision table explicitly — write one sentence per criterion.

Then answer: what changes to team process are required before this split makes sense? (Deployment pipeline, monitoring, on-call rotation.)

Exercise 5: The Split You Would Not Make Yet

Name **one domain** that looks like a natural microservice candidate but should stay in the monolith for now. Explain: - What makes it tempting to split - What makes it premature given the current team/load/ops context - What condition would change your answer

Example Answer: Exercise 4

Domain to extract first: **Payments**

Criterion	Assessment
Independent deployment needed	Partial — payment provider integrations change on their own schedule, but currently there’s no deploy velocity problem
Different scaling requirement	No — payment volume tracks order volume; no independent scaling gap
Different team ownership	No — same 3 developers own everything
Different failure blast radius	Yes — this is the strongest argument. A bug in notification dispatch should not cause payment processing to fail. A misconfigured Stripe webhook handler should not bring down checkout.
Different compliance scope	Yes — PCI DSS scope. Isolating payment card handling to one service reduces PCI audit surface across the rest of the system.

Verdict: Two strong criteria met (failure isolation, compliance scope). This is the only domain in this system where both apply simultaneously.

Pre-conditions before split: - CI/CD pipeline capable of independent deployments (not manual coordination) - Monitoring and alerting on the payments service independently - API contract defined between Orders and Payments (Orders places a payment intent, Payments processes it and emits an event) - Decision on data: does Payments get its own Postgres schema/database, or is it isolated within the shared database first?

Data Ownership Matrix — Reference

Use this to check your Exercise 1 work. Disputed cells are intentional — they reflect real coupling in most monoliths.

Table	Most Likely Owner	Common Read-Crosses
users	User Accounts	Orders (billing), Notifications (recipient)
addresses	User Accounts	Orders (shipping address snapshot)
products	Product Catalog	Inventory (product_id FK), Orders (order_items)
categories	Product Catalog	Product Catalog only
inventory	Inventory	Orders (stock check at checkout)
orders	Orders	Payments (order_id FK), Notifications (order events)
order_items	Orders	Payments (amount verification)
payments	Payments	Orders (payment status display)
notifications_log	Notifications	Audit/reporting only

Note the coupling: `inventory` is read by Orders during checkout. If you split Inventory, the stock-check at checkout becomes a synchronous API call or an eventual consistency problem. This is why Inventory is not the first split to make.

Phase — 03-data-and-storage

Directory: 03-data-and-storage/

Source: 03-data-and-storage/01-storage-decisions.md

Storage Decisions

Storage is the hardest decision to reverse. Get it wrong and you carry the cost for years. Adding a new API endpoint is hours. Migrating 50M rows to a different storage engine is a multi-week project with downtime risk.

The Storage Decision Tree

Start with query patterns and consistency requirements. Don't start with technology preferences.

Storage Type	Choose When	Do Not Choose When
Relational (Postgres, MySQL)	ACID required, joins needed, schema is stable, writes and reads are balanced	Write throughput exceeds ~50k/s on a single node without sharding plan
Document (MongoDB, Firestore)	Schema varies significantly per record, document is the unit of read/write, no cross-document joins needed	You have relational data with real joins — you will reimplement SQL badly
Key-value (Redis, DynamoDB)	Simple lookups by known key, TTL expiry useful, latency requirements < 5ms, read-heavy	You need to query by anything other than the primary key
Search (Elasticsearch, Typesense)	Full-text search, faceting, ranking, fuzzy matching	You are using it as a primary data store — it is not one
Time-series (TimescaleDB, InfluxDB)	Append-only write pattern, aggregations over time windows, retention policies needed	Data is not time-ordered or you need to update individual records frequently

The actual decision process: 1. What are your query patterns? (List them: “find by user_id”, “full-text search on description”, “get latest 100 events for order_id”) 2. What consistency do you need per operation? (Strong vs eventual) 3. What is your write volume? (Proven, not projected) 4. How stable is your schema? (Has it changed 3 times in 6 months, or has it been stable for 2 years?)

Most systems need Postgres and Redis. Everything else is additive and carries a cost.

CAP Theorem, Practically

The theory: you cannot simultaneously guarantee Consistency, Availability, and Partition Tolerance. In practice, partition tolerance is not optional in any networked system — so the real choice is between consistency and availability under network partition.

What this means for your e-commerce system:

Component	Priority	Reasoning
Payments	CP — consistency over availability	A double charge or missed refund is a business liability. If the payment service is unavailable, the correct response is an error, not a stale result.
Order state machine	CP	Order status must be authoritative. “Fulfilled” and “Pending” cannot be simultaneously true in two nodes.
Product catalog	AP — availability over consistency	A customer seeing a price that is 30 seconds stale is acceptable. The catalog being down is not.
Inventory stock levels	Depends	For display purposes: AP (show approximate stock). For reservation at checkout: CP (prevent oversell). These are two different reads of the same data with different consistency requirements.
Session store (Redis)	AP	A user being logged out due to a consistency event is a bad UX. Stale session data for seconds is acceptable.
Search index	AP	Search is a read replica of source data. Eventual consistency is inherent to the pattern.

The practical question is not “which theorem applies.” The practical question is: “if this component returns stale data or returns an error, which is worse for the business?” That answer maps to AP vs CP.

Schema Evolution Without Downtime

`ALTER TABLE ADD COLUMN NOT NULL` on a table with 10M rows and active traffic will: 1. Acquire an exclusive lock on the table 2. Rewrite every row to add the column 3. Block all reads and writes for the duration

On a busy table, this can take minutes. This is how you cause an outage during a migration.

The four safe migration patterns:

1. Expand/Contract (Parallel Change)

- Phase 1 (Expand): Add the new column as nullable. Deploy code that writes to both old and new column.
- Phase 2: Backfill old rows with the new column value in batches (small transactions, not one giant UPDATE).
- Phase 3 (Contract): Deploy code that only reads/writes the new column. Drop the old column in a separate migration.
- Use when: renaming a column, changing a column type, splitting one column into two.

2. Shadow Writes

- Write to both the old store and the new store simultaneously.
- Read from the old store. Compare results in background.
- When confidence is high, flip reads to the new store.
- Flip writes to new store only, stop writing to old.
- Use when: migrating to a different storage engine or table structure.

3. Read From Both

- Application reads from both old and new location, merges results.
- Use when: splitting a table and old data hasn't been backfilled yet.
- Temporary — clean up as soon as backfill is complete.

4. Feature-Flagged Cut Over

- New schema/store is live but behind a flag.
- Enable for internal users → 1% of traffic → 10% → 100%.
- Rollback is flag disable, not schema revert.
- Use when: high-risk migrations where you need an abort path at any point.

The pattern for the spine project (Postgres):

For any column addition: always nullable first, backfill in batches of 1,000–10,000 rows with a small sleep between batches to avoid lock contention, then add NOT NULL constraint as a separate migration after backfill is verified.

Read Replicas and Their Actual Cost

A read replica is not free reads. It is eventual consistency with a lag.

What you are buying: - Read throughput offloaded from the primary - A node that can be promoted to primary in failover - Lower latency for read-heavy analytics queries

What you are getting in exchange: - Replication lag: typically milliseconds, but can be seconds under write pressure - Read-after-write inconsistency: a user creates an order, you route the next read to a replica, the replica hasn't caught up, the user sees their order is missing

Places where replica lag breaks your app:

Operation	Problem if Routed to Replica
After user registration	User logs in, replica hasn't replicated the new user row, login fails
After order creation	Redirect to order confirmation page, order not found
After payment	Payment confirmed, order status still shows "pending"
After inventory reservation	Stock count shows as unreserved

Mitigation patterns: - Read from primary for N seconds after a write (sticky reads) - Read from primary when the session has modified data in this request - Use Redis as the read surface for data that you just wrote, with a short TTL until replication catches up - Accept the lag only for genuinely read-only workloads (reporting, search indexing, analytics)

When Postgres Is Enough

Most systems at most scales need Postgres with proper indexing and a connection pool. The evidence for “Postgres can’t handle this” is almost always missing.

Before adding a second database type, prove:

Check	How
Query time with index	<code>EXPLAIN ANALYZE</code> the slow query. Is the index being used? Is it the right index?
Connection pool config	Are you exhausting connections? <code>pg_stat_activity</code> . Is PgBouncer in front?
N+1 query pattern	Are you running 1 query per row in a list? Fix that before adding a cache.
Table bloat	<code>pg_stat_user_tables</code> . Dead rows from high-UPDATE tables inflate table size and slow scans. Run <code>VACUUM</code> .
Query at scale	Does the query plan change at 10M rows vs 100k rows? Test it.

Actual Postgres limits (rough order of magnitude): - 10,000 writes/second: achievable on commodity hardware with proper indexing - 1B rows in a table: fine with correct indexes and partitioning - 100 concurrent connections: manageable with PgBouncer - Sub-millisecond reads on indexed lookups: normal

The benchmark you need before adding a second database: does `EXPLAIN ANALYZE` on your slow query show a sequential scan on a large table? If yes, add an index. If the index exists and is used and the query is still slow, now you have a real problem to solve.

Decision Table: Query Pattern → Storage

Query Pattern	Consistency Need	Write Volume	Schema Stability	Storage Choice
Lookup by primary key, ACID writes	Strong	Moderate	Stable	Postgres
Full-text search with ranking	Eventual OK	Low (index updates)	Moderate	Elasticsearch / Typesense + Postgres as source of truth
Session data, TTL expiry	Eventual OK	High	N/A	Redis
Append-only events, time-window aggregations	Eventual OK	Very high	Stable	TimescaleDB or Postgres with partitioning
Highly variable schema per document	Eventual OK	Moderate	Unstable	Document store — but verify joins aren’t needed first
Rate limiting, distributed counters	Eventual OK	Very high	N/A	Redis (INCR with TTL)
Job queue, task state	Strong	Moderate	Stable	Postgres (with advisory locks) or Redis (if Postgres queue adds latency)

Anti-Patterns

Adding a cache before profiling the slow query. Redis in front of an unindexed Postgres query means you now have two systems to operate and the underlying problem still exists. `EXPLAIN ANALYZE` first. Add an index. Then decide if a cache is still needed.

Using MongoDB because “schema flexibility.” If your schema has been stable for 6 months, the flexibility argument is moot. You’ve traded ACID guarantees and joins for a problem you don’t have. The real cost appears when you need to query across documents or enforce referential integrity.

Separate database per microservice when the microservices share transactions. If splitting orders from inventory requires a distributed transaction (saga pattern) to keep stock consistent, you’ve added enormous complexity. Shared transactions are a signal that the services are not truly independent and should either be merged or redesigned with eventual consistency explicitly accepted.

Šta da pitaš AI

- “Given these query patterns: [list them with approximate frequency]. What storage type fits each? What are the consistency tradeoffs if we use Postgres for all of them?”
- “We want to add [column definition] to [table name] which has [N] million rows and [X] writes per second in production. What migration strategy avoids downtime? Write out the steps.”
- “Our Postgres query takes [X]ms: [paste query]. Before we add a cache, what should we check? What does EXPLAIN ANALYZE output tell us about this query plan?”
- “We’re considering a read replica for [use case]. Which of our read operations are safe to route to the replica, and which must go to primary? Walk through each endpoint.”

Source: 03-data-and-storage/02-lab-storage.md

Lab: Storage Decisions — Job Marketplace

Two exercises. The first tests storage choice. The second tests migration design. Both use the decision framework from 01-storage-decisions.md.

Exercise 1: Storage Choice for a Job Marketplace

System: Job marketplace platform. Employers post jobs, applicants apply, both sides message each other, and the system generates recommendation scores per applicant/job pair.

Team: 5 engineers. Single Postgres database currently. Considering whether each entity needs a different storage strategy.

Entities and their access patterns:

Entity	Description
Job listings	~500k active listings. Employers create, update, close. Applicants search by title, location, salary, skills. Filtering and ranking on 10+ attributes.
Applications	Applicants apply to jobs. One applicant → one job = one application. Status transitions: applied → reviewed → interview → offer → rejected. Employers query by job_id. Applicants query by their own history.
Companies	~50k companies. Name, description, size, industry, logo URL. Rarely updated. Frequently read.
Users (applicants)	~2M users. Registration, login, profile (skills, experience, education), resume PDF reference.
Messages	Direct messages between applicant and employer per application. Append-only thread. ~5M messages/month. Query: load thread for application_id.
Recommendation scores	ML-generated score per (user_id, job_id) pair. Updated nightly in batch. ~100M pairs. Query: given user_id, return top-50 job_ids by score.

For each entity, fill in this table:

Entity	Primary Query Pattern	Secondary Query Pattern	Consistency Requirement	Write Pattern	Storage Choice	Justification
Job listings						
Applications						
Companies						
Users						
Messages						
Recommendation scores						

Guiding questions per entity: - What does the most common read look like? (key lookup, full-text search, range scan, join) - What happens if the read returns stale data by 5 seconds? 5 minutes? (consistency requirement) - Is the write pattern insert-only, update-heavy, or batch? - How many rows/documents at steady state?

Example Answer: Job Listings

Field	Value
Primary query pattern	Full-text search + faceted filtering: "software engineer in Berlin, €70–90k, full-time, Python required"
Secondary query pattern	Employer loads their own listings by company_id (key lookup)
Consistency requirement	AP — a listing that is 30 seconds stale in search results is acceptable. A listing being unindexed for 30 seconds after creation is acceptable.
Write pattern	Create/update by employer (~moderate volume). Status transitions (open → closed) are low volume.
Storage choice	Postgres as source of truth + Elasticsearch/Typesense as search index
Justification	Full-text search with ranking and multi-facet filtering is not efficient in Postgres. A search index gives sub-100ms ranked results on 500k documents. Postgres remains the authoritative store. Write to Postgres first, sync to search index asynchronously.

Note the gap: search index is eventually consistent with Postgres. A newly posted job will appear in Postgres immediately but may not appear in search results for up to ~10 seconds (depending on indexing pipeline). Acceptable for this use case. Not acceptable for a checkout stock check.

Complete the table for the remaining 5 entities. Pay particular attention to: - **Recommendation scores:** the scale (100M pairs) and the query pattern (top-N for a user) point strongly away from a naive relational approach. What is the right structure? - **Messages:** append-only and thread-scoped. Does this need a separate store or does Postgres handle it cleanly? - **Companies:** high read, low write, moderate size. Where does caching belong in the picture?

Exercise 2: Migration Design

Context: Production system. Postgres. Table `orders` with 10 million rows. High write volume: ~200 inserts/minute during peak hours. The table is read on every page load for order status.

Requirement: Add a column `fulfilled_at` `TIMESTAMP` that records when an order reached the `fulfilled` status. Business logic: for historical orders already in `fulfilled` status, the value should be set to `updated_at` (the closest proxy available). For future orders, it will be set at transition time.

The naive migration:

```
ALTER TABLE orders ADD COLUMN fulfilled_at TIMESTAMP NOT NULL DEFAULT NOW();
UPDATE orders SET fulfilled_at = updated_at WHERE status = 'fulfilled';
UPDATE orders SET fulfilled_at = NULL WHERE status != 'fulfilled';
```

This will cause an outage. Do not do this.

Design the correct migration. Answer these questions:

1. Which strategy applies? Choose from: Expand/Contract, Shadow Writes, Read From Both, Feature-Flagged Cut Over. Justify your choice.

2. Write out the steps in order.

For each step, specify: - What SQL or code change is involved - What the application code does at this point - Whether this step requires a deployment - Whether this step is reversible

3. What is the rollback plan?

At which steps can you roll back cleanly? What does rollback look like for each?

4. What do you tell the team?

Write the 3-sentence message you send to the engineering team before starting the migration. Cover: what is happening, what downtime risk exists, what the rollback plan is.

Reference: Migration Strategy Template

Use this template for Exercise 2 and future migrations:

Migration: [Name]
Table: [table name]
Row count: [estimated]
Write volume: [peak writes/minute]
Strategy: [Expand/Contract | Shadow Writes | Read From Both | Feature-Flagged]

Steps:

- Step 1: [SQL or code change]
- App behavior: [what application code does here]
 - Deployment required: Yes/No
 - Reversible: Yes/No — [how to reverse]
- Step 2: ...

Backfill plan:

- Batch size: [rows per transaction]
- Batch delay: [sleep between batches in ms]
- Estimated duration: [total time for backfill]
- Monitoring: [what to watch during backfill]

Rollback:

- Before step N: [rollback action]
- After step N: [rollback action or "point of no return"]

Communication:

- Team message: [what you tell engineers]
- Incident escalation: [who to call if it goes wrong]

Example Answer: Exercise 2 (Partial)

Strategy: Expand/Contract

This is a column addition with a backfill. The old column doesn't need to be dropped. The expand phase adds the column safely; the backfill runs in batches; the application starts writing to the new column at a defined deploy point.

Step 1 — Expand (add nullable column)

```
ALTER TABLE orders ADD COLUMN fulfilled_at TIMESTAMP NULL;
```

- This is a metadata-only change in Postgres 11+. No table rewrite. Near-instantaneous.
- App behavior: application ignores the new column. Still reads/writes existing columns only.
- Deployment required: No (schema migration only, app is unaffected)
- Reversible: Yes — `ALTER TABLE orders DROP COLUMN fulfilled_at;`

Step 2 — Backfill in batches

```
-- Run in batches, not as a single statement
UPDATE orders
SET fulfilled_at = updated_at
WHERE status = 'fulfilled'
  AND fulfilled_at IS NULL
  AND id BETWEEN :batch_start AND :batch_end;
```

- Batch size: 5,000 rows per transaction
- Sleep between batches: 100ms to avoid lock contention under write load
- Estimated duration at 10M rows, 5k/batch, 100ms delay: ~200 batches × 100ms = ~20 minutes for the fulfilled subset
- Monitor: `pg_stat_activity`, replication lag on replica, application error rate

Step 3 — Deploy application code to write `fulfilled_at` on transition - Application now sets `fulfilled_at = NOW()` when an order transitions to `fulfilled` status - Deployment required: Yes - Reversible: Yes — deploy previous version; column goes back to null for new transitions (backfill data is unaffected)

Step 4 — Add NOT NULL constraint (if required by application)

```
-- Only after backfill is complete and application is writing the column
ALTER TABLE orders ALTER COLUMN fulfilled_at SET NOT NULL;
```

- In Postgres 12+: if all rows have non-null values, this is a constraint-only change (no table scan)
- In Postgres < 12: this triggers a table scan — test on a production-sized copy first
- Reversible: Yes — `ALTER TABLE orders ALTER COLUMN fulfilled_at DROP NOT NULL;`

Rollback plan: - Before Step 3: drop the column — no application impact - After Step 3: deploy previous application version — fulfilled_at column stays but new transitions stop writing to it; not a data integrity problem - After Step 4: drop the NOT NULL constraint — instantaneous

Team message:

We are adding `fulfilled_at` to the orders table over the next 30 minutes. The migration runs in batches with no downtime — the `ALTER TABLE` step is instantaneous in Postgres 11+. If anything goes wrong during the backfill we can drop the column with no impact on the running application. I will post status here and @channel if anything requires attention.

Phase — 04-async-events-queues

Directory: 04-async-events-queues/

Source: 04-async-events-queues/01-when-async.md

When to Go Async

Async is not free. Every async boundary is a failure mode, a consistency problem, and an observability challenge. Add async because the problem demands it, not because it feels modern.

The Async Decision

Use async when **all three** hold: 1. The caller doesn't need the result to continue 2. The operation can fail without the user knowing immediately 3. The operation takes longer than the user should wait

Use sync when **any one** holds: - The user needs the result to proceed (payment confirmation, inventory check result) - The operation must succeed for the transaction to succeed - You need strong consistency — the write must be visible to the next read

The test: Can you show the user a success screen before this operation completes? If no, it's sync.

Event vs Command vs Query

Three message shapes. Choosing wrong causes cascading design problems.

Type	Semantics	Direction	Coupling	Result
Event	Something happened (<code>OrderPlaced</code>)	Fire-and-forget, fan-out	Publisher knows nothing about consumers	None expected
Command	Do this (<code>SendConfirmationEmail</code>)	Point-to-point, targeted	Publisher knows there's one consumer	Acknowledged eventually
Query	Give me data	Always synchronous	Tight — caller blocks	Immediate response

Events are past tense and immutable: `OrderPlaced`, `PaymentFailed`, `InventoryReserved`. The publisher has no opinion about what happens next.

Commands are imperative and directed: `SendEmail`, `ProcessRefund`. If nobody processes it, that's a bug — use a DLQ and alerting.

Wrong choice symptoms: - Using events for commands → you don't know if anyone processed it - Using commands when events are appropriate → tight coupling, single consumer bottleneck - Using async for queries → you have to poll or build callbacks for something that should be a function call

Ownership of Guarantees

Confusion here causes “the queue just lost our data” incidents.

Publisher owns: - Event schema and versioning - At-least-once delivery (the message was sent) - Valid payload (schema validation before publish) - Not publishing unless there's something to process

Consumer owns: - Idempotency — handling duplicate delivery - Ordering tolerance — messages may arrive out of order - DLQ monitoring — if your consumer is failing, you need to know - Own retry logic (not relying on publisher to resend)

The boundary: publisher guarantees the message exists and is valid. Consumer guarantees the effect happens exactly once despite multiple deliveries.

If your publisher is “making sure” the consumer processed it, you've inverted ownership and created coupling.

The Idempotency Requirement

Every queue consumer must be idempotent. A message **will** be delivered more than once. This is not a bug — it is the delivery guarantee (at-least-once).

Idempotency patterns:

Natural idempotency: The operation is inherently safe to repeat. Setting a status to `shipped` twice is fine. Use when possible.

Idempotency key: Consumer tracks a key (message ID, order ID + event type) in a processed-events table. Before processing, check if already handled. If yes, skip and ack.

```
-- In the same transaction as the effect
INSERT INTO processed_events (event_id, processed_at)
VALUES ($1, NOW())
ON CONFLICT (event_id) DO NOTHING
RETURNING event_id
-- If no row returned, already processed — skip
```

Conditional write: Only write if precondition still holds. `UPDATE orders SET status = 'shipped' WHERE status = 'paid'` — if already shipped, WHERE fails, no double-ship.

What breaks without idempotency: - Payment charged twice - Confirmation email sent twice - Inventory decremented twice - Analytics double-counted

DLQ as Required Infrastructure

Not optional. DLQ is the difference between silent message loss and recoverable failure.

Without DLQ: Consumer fails repeatedly → queue broker drops message after max retries → failure is invisible.

With DLQ: Consumer fails repeatedly → message moves to dead-letter queue → message accumulates → alert fires → engineer replays after fix.

DLQ operational checklist: - [] Every consumer queue has a corresponding DLQ - [] DLQ depth is monitored with alert threshold (> 0 for critical, > N for warning) - [] Messages in DLQ include original payload + failure reason + retry count - [] Replay tooling exists (don't manually requeue one at a time) - [] DLQ messages expire (don't accumulate forever — set TTL appropriate to recovery window)

DLQ monitoring = ops maturity signal. If your team doesn't know the DLQ depth right now, you don't have async ops maturity.

Outbox Pattern

Problem: You want to publish an event only if the database transaction committed.

Without outbox: 1. DB transaction commits (order saved) 2. Publish event to queue 3. If step 2 fails → order exists, event never sent → downstream never notified

Or inverted: 1. Publish event to queue 2. DB transaction commits 3. If step 3 fails → event sent, order doesn't exist → phantom event downstream

With outbox: 1. In the same DB transaction: write order record + write event to `outbox` table 2. Transaction commits atomically 3. Separate outbox worker reads `outbox` table, publishes to queue, marks as published 4. If publishing fails → outbox record stays unpublished → worker retries

The event is now guaranteed to be published if and only if the transaction committed. The outbox worker delivers at-least-once (retry on failure), so consumers still need idempotency.

When you need it: Any time you write to DB and publish an event in the same operation, and you care about consistency between them.

Decision Table

Caller needs result?	Op duration	Failure tolerance	Consistency requirement	Decision
Yes	Any	Low	Strong	Sync
No	< 200ms	Low	Strong	Sync
No	< 200ms	High	Eventual	Could go either way — sync is simpler
No	> 500ms	High	Eventual	Async
No	Any	High	Eventual, write + publish	Async + Outbox
No	Any	High	Eventual, fan-out	Event (not command)

Anti-patterns

Async for everything. Every operation becomes a debugging and observability problem. “Why did this not happen?” is hard to answer when there are 8 async hops. Reserve async for operations that genuinely fit.

No idempotency in consumers. Duplicate delivery is guaranteed. If your consumer doesn’t handle it, you will double-charge, double-ship, or double-send. Not if — when.

Publisher schema changes without consumer coordination. Adding a required field breaks all existing consumers. Removing a field silently breaks consumers that depend on it. Schema versioning and backward compatibility are publisher responsibilities.

No DLQ. Failed messages disappear. You will never know. You will discover the failure when a customer complains weeks later.

Using async to hide latency problems. “The DB query is slow so we’ll make it async.” The query is still slow. You’ve added a queue but not fixed the problem, and now you have both a slow query and async complexity.

Commands modeled as events. `EmailSendRequested` sounds like an event but it’s a command. If it ends up in DLQ and nobody notices, the user never gets their email. Model it as a command, own the result.

Šta da pitaš AI

For sync/async decision:

“This operation [describe it in detail: what it does, how long it takes, what depends on it succeeding]. Should it be sync or async? What failure modes does each choice introduce? What does the user experience look like if the async path fails?”

For idempotency design:

“Our queue consumer processes [describe: what the consumer does, what side effects it has, what data it writes]. What are the idempotency requirements? How do we handle duplicates? Show me the idempotency key design and the DB table I need.”

For event schema:

“We need an event schema for [domain event: what happened, what the downstream needs to know]. What fields should be in the stable contract vs what can evolve? How do we version this schema without breaking consumers?”

For outbox decision:

“We write to [table] and publish [event] in the same request handler. Do we need the outbox pattern? What breaks if we don’t use it? Show me the outbox table schema and worker pseudocode for our Symfony + Postgres + [queue] stack.”

Source: `04-async-events-queues/02-lab-async.md`

Lab: Async Classification — E-Commerce Checkout

Scenario: user places an order. The system must: (1) reserve inventory, (2) charge payment, (3) send confirmation email, (4) update analytics, (5) notify warehouse.

Work through each exercise before checking the reference tables.

Exercise 1: Classify Each Step

For each operation, decide: **sync** (must complete before 201 to user) or **async** (can happen after response).

Apply the three-part test to each: - Does the caller need the result to continue? - Can it fail without the user knowing immediately? - Does it take longer than the user should wait?

Write your classification before reading the reference table.

Reference: Sync/Async Classification

Step	Classification	Reason	Failure impact
Reserve inventory	Sync	User needs to know if item is available before payment	Without this, oversell — chargebacks and fulfillment failures
Charge payment	Sync	Transaction cannot succeed if payment fails; user needs confirmation of charge	Async payment = orders with no payment; impossible to reconcile
Send confirmation email	Async	User has their 201, email is a nice-to-have in the moment	Failed email: retry, don't roll back the order
Update analytics	Async	Analytics is reporting infrastructure, never user-critical	Failed analytics: silent catch-up, no user impact
Notify warehouse	Async	Warehouse processing is downstream fulfillment, not checkout	Failed notification: DLQ alert, manual or replay recovery

The rule confirmed by this table: sync operations are those where failure means the transaction itself failed. Async operations are effects that should happen but don't gate the core user transaction.

Exercise 2: Idempotency Ownership

For each async step, answer: - Who owns idempotency? (Name the service/consumer, not a team) - What is the idempotency key? - What happens on duplicate delivery?

Write your answers before reading the reference table.

Reference: Idempotency Ownership

Async step	Idempotency key	Duplicate handling	What breaks without it
Send confirmation email	<code>order_id + event_type('OrderPlaced')</code>	Check <code>sent_emails(order_id, type)</code> before sending; skip if exists	Customer gets 2+ confirmation emails; support tickets
Update analytics	<code>order_id + event_id</code>	Upsert on <code>(order_id, event_id)</code> — last write wins or dedup at read	Double-counted revenue, conversion rates; misleads decisions
Notify warehouse	<code>order_id</code>	Check <code>warehouse_notifications(order_id)</code> before dispatch; skip if exists	Warehouse picks and ships twice; inventory error + refund cost

Ownership rule confirmed: the consumer owns idempotency. The publisher (checkout service) emits `OrderPlaced` once (plus retries). Each consumer is independently responsible for handling it exactly once regardless of delivery count.

Exercise 3: DLQ Handling Design

For each async step, design the DLQ response: - What does a message in DLQ mean operationally? - Who gets alerted? - What is the recovery action?

Write your design before reading the reference table.

Reference: DLQ Handling

Consumer	Message in DLQ means	Alert recipient	Recovery action
Confirmation email	Email service down or template error	On-call engineer (email service)	Fix root cause, replay DLQ; customers get delayed email with apology if >1hr
Analytics	Analytics pipeline broken; events will have gap	Data team (non-urgent)	Replay DLQ after fix; no customer impact but timeline gap in reports
Warehouse notification	Warehouse integration broken; orders accumulating without fulfillment signal	On-call engineer (critical — SLA impact)	Fix integration ASAP; replay DLQ ordered by <code>created_at</code> to process in sequence

DLQ alert thresholds: - Confirmation email: alert at depth > 0, resolve within 30 min (customer-visible) - Analytics: alert at depth > 100 or age > 1hr (non-urgent, batch recovery acceptable) - Warehouse: alert at depth > 0, page on-call immediately (SLA and fulfillment at risk)

Recovery sequence for warehouse (order-sensitive): 1. Identify root cause (do not replay into a broken consumer) 2. Fix consumer 3. Deploy 4. Replay DLQ in order — warehouse needs to process in dispatch sequence 5. Verify warehouse received and acknowledged each notification 6. Close incident, document in post-mortem

Exercise 4: Where Do You Need the Outbox Pattern?

For each operation that involves both a DB write and a queue publish, decide whether outbox is required. Justify your answer.

Write your decisions before reading the reference.

Reference: Outbox Decision

Outbox decision rule:

Use the outbox pattern when: you write to the database AND publish an event in the same request, AND the downstream cannot tolerate phantom events or silent non-delivery.

Step	DB write?	Queue publish?	Outbox required?	Justification
Inventory reservation	Yes (<code>inventory_reservations</code>)	No (sync, no queue)	No	Sync operation, no queue involved
Payment charge	Yes (<code>orders</code> , <code>payments</code>)	No (sync, no queue)	No	Sync operation, no queue involved
Emit <code>OrderPlaced</code> event	Yes (<code>orders</code> committed by this point)	Yes → email, analytics, warehouse consumers	Yes	If publish fails after commit: email never sent, warehouse never notified. If publish happens before commit and transaction rolls back: phantom <code>OrderPlaced</code> for an order that doesn't exist

Without outbox on `OrderPlaced`:

Sequence that fails silently:

```
1. BEGIN TRANSACTION
2. INSERT INTO orders ...
3. INSERT INTO payments ...
4. COMMIT ← success
5. publish OrderPlaced to queue ← crash here
```

Result: order exists in DB, warehouse never notified, confirmation email never sent. Customer sees 201, order never fulfills.

With outbox:

```

1. BEGIN TRANSACTION
2. INSERT INTO orders ...
3. INSERT INTO payments ...
4. INSERT INTO outbox (event_type, payload, published=false)
5. COMMIT -- all or nothing
6. Outbox worker: SELECT unpublished FROM outbox
7. Publish to queue
8. UPDATE outbox SET published=true

```

If step 7 fails, outbox worker retries. Event is guaranteed to be published iff the transaction committed.

Outbox table (Postgres):

```

CREATE TABLE outbox (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type  VARCHAR(100) NOT NULL,
  payload     JSONB NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  published   BOOLEAN NOT NULL DEFAULT FALSE,
  published_at TIMESTAMPTZ,
  retry_count INT NOT NULL DEFAULT 0
);

CREATE INDEX ON outbox (published, created_at)
WHERE published = FALSE;

```

Summary Tables

Sync/Async Split

Step	Decision	Gate to user response?
Reserve inventory	Sync	Yes
Charge payment	Sync	Yes
Send email	Async	No
Update analytics	Async	No
Notify warehouse	Async	No

Idempotency Keys

Consumer	Key	Pattern
Email	order_id + event_type	Insert-or-skip
Analytics	order_id + event_id	Upsert
Warehouse	order_id	Insert-or-skip

Outbox Decision Rule

Condition	Outbox needed?
Sync operation (no queue)	No
Queue publish, DB write independent	No
Queue publish + DB write in same transaction, both must succeed or fail together	Yes

Phase — 05-reliability-failure

Directory: 05-reliability-failure/

Source: 05-reliability-failure/01-failure-thinking.md

Failure Thinking

Reliability is not “does it work?” — it is “how does it degrade when dependencies fail?” Every service will have its dependencies fail. The question is what your service does about it.

SLO/SLI Practically

SLI (Service Level Indicator): what you measure. A specific, observable metric. **SLO (Service Level Objective):** the target. The threshold your team agrees to operate to. **SLA (Service Level Agreement):** the external contract. Usually less strict than internal SLO to give you operating room.

Concrete SLIs for a Symfony API: - p99 request latency < 200ms (measured at the load balancer or application) - error rate < 0.1% (5xx responses / total responses, 5-minute window) - availability > 99.9% (successful health checks / total checks)

Do not set SLOs at theoretical maximums. Set them at what your team can actually defend and what users will actually notice if crossed. If users tolerate 500ms, don't set SLO at 200ms — you'll burn budget on noise.

SLO should sit below user expectations, leaving you budget to burn before users feel it.

Error Budget

Error budget = the unreliability you're allowed before you must stop shipping features and do reliability work.

SLO	Error budget (monthly)	Error budget (weekly)
99.9% availability	~44 minutes	~10 minutes
99.5% availability	~3.6 hours	~50 minutes
99.0% availability	~7.2 hours	~1.7 hours

The mechanism that forces tradeoffs: - Budget remaining → ship features, accept some instability - Budget depleted → feature freeze, reliability work only

Without error budget, reliability and feature velocity are both “important” and neither has leverage. Error budget makes the tradeoff explicit and gives the reliability team standing to say “no new features.”

Error budget consumption triggers: - Incident duration × proportion of traffic affected - Planned maintenance (counts against budget — incentivizes good change management) - Failed deploys that caused errors

Cascading Failure

The failure pattern that takes down systems, not services:

```
Service A slow (high latency)
→ Service B waits for A (threads held)
→ Service B thread pool exhausted (no threads to serve new requests)
→ Service B unavailable
  → Service C times out waiting for B
  → Service C thread pool exhausted
→ Total outage
```

This happens because of **unbounded waits**: B waits for A with no timeout. One slow dependency fills all threads.

Three controls:

1. Timeouts on every dependency call. Every HTTP call, every DB query, every Redis operation needs a timeout. The timeout should be lower than your own SLO response time. If your SLO is p99 < 200ms and you have 3 downstream calls, each dependency gets a ~50ms timeout — not “default” (which is usually infinite or 30 seconds).

2. Circuit breakers on high-fanout dependencies. When a dependency is failing, stop sending requests to it immediately — don’t wait for each request to time out. Circuit breaker opens after N consecutive failures, rejects requests fast (fast-fail vs slow timeout), half-opens after a cooldown to probe recovery.

Circuit breaker states: - **Closed** (normal): requests pass through - **Open** (failing): requests rejected immediately, no calls to dependency - **Half-open** (probing): one request allowed through; if it succeeds, close; if not, re-open

3. Bulkheads (separate thread pools per dependency). If all dependencies share one thread pool, one slow dependency starves all others. Dedicate thread pools per downstream service. Service B’s slowness cannot exhaust the thread pool used for Service C.

Degradation Posture

When a dependency fails, three options. Choose based on: how important is this data? How stale is acceptable?

Option	What it does	When to use
Fail fast	Return error immediately	Dependency is critical and no substitute exists; stale data is worse than no data
Degrade gracefully	Return cached or partial data	Stale data is acceptable; user experience is better with approximate answer than error
Queue for retry	Accept the request, process when dependency recovers	Write operation; losing the request is worse than delayed processing

Degradation is not a fallback you design last. It is a first-class design decision made when you design the happy path. For every dependency: “If this is down, what does my service do?”

Degradation responses in practice: - Search service → Elasticsearch down → return cached results from Redis with staleness indicator - Product page → pricing service slow → return last-known price with “price may have changed” note - Checkout → payment processor down → fail fast with “payment unavailable, try again shortly” - Order write → DB slow → queue for retry (cannot lose the write)

The Minimum Monitoring Set

Four categories. Without these, you cannot answer “is the service healthy?” or “what is breaking?”

1. Your SLIs - Request latency (p50, p95, p99) — histograms, not averages - Error rate (5xx / total, per endpoint) - Availability (uptime probe)

2. Dependency health - Response time and error rate for each downstream service - DB connection pool utilization (near-full pool is pre-failure signal) - Redis hit/miss rate and latency - Queue consumer lag (how far behind is the consumer from the producer?)

3. Queue depth and age - Current depth per queue - Age of oldest unprocessed message (depth can look fine if one old message is stuck) - DLQ depth (anything > 0 is a signal)

4. Error budget remaining - Current SLO attainment vs target - Budget consumed this period - Burn rate (if you’re burning 10x normal rate, you’ll exhaust budget before the period ends)

If you don’t have these four categories instrumented, start here before adding any other monitoring.

Decision Table: Degradation

Dependency fails	Data freshness requirement	User impact if degraded	Decision
Not critical path	Stale OK (minutes)	Minimal	Degrade gracefully — serve from cache
Not critical path	Stale OK (hours)	None	Queue for retry
Critical path	Must be fresh	High	Fail fast with clear error
Write operation	N/A — must persist	High if lost	Queue for retry
Critical path	Stale marginally OK	Medium	Degrade with staleness indicator

Anti-patterns

No timeout on database queries. One slow query holds a thread. 20 slow queries hold all threads. DB timeouts are not optional — they are the load-shedding mechanism that keeps your thread pool from being the bottleneck when DB degrades.

Circuit breaker with no fallback. You break the circuit and return 500. From the user's perspective, that's the same as the timeout you just avoided — you just made it faster. A circuit breaker without a degradation strategy is not a reliability improvement; it's a latency improvement. The user still gets an error.

SLO set at “100% uptime.” Impossible. Means nobody takes reliability seriously because the target can't be met. A 100% SLO also means there's no error budget — no tolerance for any failure anywhere, which paralyzes deployment and change management.

Monitoring only the happy path. Your dashboards show success rates and green latency while the queue is 10,000 messages deep and aging out. Monitor what breaks, not what works.

Timeout set equal to or longer than your SLO. If your SLO is 200ms and your downstream timeout is 30 seconds, the timeout never triggers before you've already violated your SLO. Timeouts must be tuned relative to your own latency budget.

Šta da pitaš AI

For cascade analysis:

“Service A calls services B, C, D. B is responding in [X ms] instead of [Y ms]. Walk through the cascade failure: what fills up at each layer? What does the user see? What metrics spike first? Then tell me what timeouts, circuit breakers, and bulkheads I should add and at which layer.”

For SLI/SLO instrumentation:

“Our SLO is [availability/latency/error rate target]. What are the specific SLIs I need to measure it? For a Symfony API with Postgres and Redis, what instrumentation do I add and where? Show me the Prometheus metrics and alert rules.”

For degradation design:

“Dependency [X] just went down. My service does [describe function]. Walk through each degradation option — fail fast, degrade gracefully, queue for retry. What does the user get in each case? What are the risks of each? Which would you recommend given [your constraints]?”

For error budget:

“Our SLO is [X]. We had an incident that caused [Y minutes] of [Z% error rate]. How much error budget did that consume? At our current burn rate, when do we exhaust the monthly budget? What does that trigger for our team?”

Source: `05-reliability-failure/02-lab-reliability.md`

Lab: Failure Scenarios and Degradation Design

Two exercises. Work through each before checking the reference answers.

Exercise 1: Cascade Failure Trace

Stack: Gateway → Symfony API → Go worker → Postgres → Redis

Scenario: Postgres starts returning queries in 8 seconds instead of the normal 20ms. Postgres is not down — it is responding, just slowly. No timeouts are currently configured.

Tasks: 1. Trace the cascade: what happens at each layer in sequence? 2. What does the user see and when? 3. What metrics spike and in what order? 4. What breaks first? 5. Design mitigations at each layer.

Work through this before reading the reference below.

Reference: Cascade Trace (No Mitigations)

Timeline:

Time	Layer	What happens
T+0s	Postgres	Queries start taking 8s (slow disk, lock contention, runaway query — root cause TBD)
T+0–8s	Symfony API	First requests to Postgres hang. PHP-FPM workers waiting for DB response.
T+8s	Symfony API	First requests return (slowly). p99 latency now ~8s. FPM worker count drops as each worker is occupied for 8s instead of 20ms — capacity drops ~400x.
T+10–30s	Symfony API	FPM pool fills. New requests queue behind waiting workers. Queue backs up at the Gateway.
T+30–60s	Gateway	Connection pool to Symfony API fills. Gateway starts returning 502/504 to users.
T+30–60s	Go worker	Worker polls Postgres (job queue or state reads). Worker goroutines hang waiting for DB. Worker throughput drops to near zero.
T+60s+	Full outage visible	Users get 502s or timeouts. No new requests are being processed.

What the user sees: - First ~10 seconds: normal (in-flight requests complete before the pool fills) - 10–30 seconds: requests are slow (8s response times) - 30s+: requests fail entirely (502, 504, connection refused)

Metrics that spike, in order: 1. `postgres_query_duration_p99` (first signal — if you have it) 2. `php_fpm_active_processes` approaching `php_fpm_max_children` 3. `http_request_duration_p99` at Symfony API 4. `gateway_upstream_errors` and `gateway_request_queue_depth` 5. `go_worker_goroutine_count` climbing (goroutines blocked) 6. `http_error_rate` (502/504) — this is what most teams alert on, but it's the last signal

What breaks first: Symfony API FPM pool. The connection pool is the first bounded resource that fills. Everything downstream of it (user responses, worker processing) degrades from there.

Reference: Mitigations by Layer

Layer	Mitigation	Mechanism	Effect
Symfony API → Postgres	DB query timeout	<code>PDO::ATTR_TIMEOUT</code> or Doctrine <code>defaultQueryTimeout</code> = 500ms	Queries fail fast instead of holding FPM workers for 8s. FPM pool freed.
Symfony API → Postgres	Connection pool limit + timeout	<code>max_connections</code> tuned to FPM pool size; connection acquisition timeout	Prevents connection exhaustion at Postgres level
Symfony API	FPM pool sizing	<code>pm.max_children</code> sized to handle burst, not unlimited	Bounded resource — explicit ceiling rather than implicit
Symfony API	Circuit breaker on DB	Open after N consecutive DB timeouts; fast-fail new requests	Stops hammering slow Postgres, gives it recovery time
Symfony API	Read from Redis cache	Cache hot data; serve stale reads when DB is slow	API continues serving read requests from cache
Gateway → Symfony API	Upstream timeout + retry	Gateway timeout < SLO; retry on 503, not on 500	Gateway fails fast rather than queuing indefinitely
Go worker	DB query timeout	Per-query context with timeout	Worker goroutines don't accumulate
Go worker	Backoff on DB failure	Exponential backoff with jitter when DB queries fail	Reduces Postgres load during degradation, aids recovery
Postgres	Query timeout at DB level	<code>statement_timeout</code> = 2000ms in Postgres config	Server-side safety net catches all clients
Postgres	Identify slow query	<code>pg_stat_activity</code> , <code>auto_explain</code>	Find the root cause (runaway query, missing index, lock)

After mitigations — same scenario: - Postgres slows to 8s - Symfony API queries time out at 500ms - Circuit breaker opens after 5 failures - API serves reads from Redis cache - API returns 503 (degraded) for writes — fast, not slow - FPM pool stays available - Go worker backs off, waits for circuit to close - User sees: “service degraded, read-only mode” or cached results — not a 502

Exercise 2: Degradation Posture Design

Service: Product search **Stack:** Symfony API with three data sources: - Elasticsearch (primary — full-text search, facets, relevance ranking) - Redis cache (recent query results, TTL 5 minutes) - Postgres (canonical product data — all fields, no search capability)

Task: Design the degradation posture for each failure scenario. For each: - What does the user get? - What do you log? - What is the alerting threshold? - What is the recovery path?

Work through all four cases before reading the reference.

Scenarios: 1. Elasticsearch is down (connection refused) 2. Elasticsearch is slow (responding in > 500ms, SLO is p99 < 200ms) 3. Redis is down (connection refused) 4. All three down simultaneously

Reference: Degradation Posture Table

Scenario	User gets	Logging	Alert threshold	Recovery path
ES down	Results from Redis cache (if hit). If cache miss: fallback to Postgres ILIKE search with note “limited search results.” If Postgres also fails: “Search unavailable, browse by category.”	ERROR <code>elasticsearch.unavailable</code> + <code>circuit_state=open</code> . Log every cache hit vs fallback vs total failure per request.	ES circuit breaker opens (immediately); alert ops. DLQ if search queries are queued.	ES recovers → circuit half-opens → probe → close. No replay needed (stateless read).
ES slow (>500ms)	Results from Redis cache if hit (TTL not expired). If cache miss: serve Postgres fallback with degradation note. New queries not sent to ES (circuit opens after N slow responses).	WARN <code>elasticsearch.latency</code> <code>p99=Xms</code> <code>threshold=500ms</code> . Log slowness + fallback type per request.	<code>p99 > 500ms</code> for 30s window → alert.	ES latency returns to normal → circuit half-opens → probes succeed → close.
Redis down	Results from Elasticsearch directly (normal search, slightly higher ES load). Cache writes silently dropped (don't fail the request for a write failure).	WARN <code>redis.unavailable</code> . Log every request that would have been a cache hit (to measure ES load increase).	Redis circuit opens → immediate alert (ES load will spike 3–10x without cache).	Redis recovers → circuit closes → cache warm-up naturally through traffic.
All three down	Static fallback: return featured/bestseller product list from in-memory config (last-known, compiled at deploy time). Clear degradation message: “Search is temporarily limited — showing popular products.”	CRITICAL <code>search.all_backends_unavailable</code> . Alert immediately, page on-call. Log every request hitting the static fallback.	Immediate page — this is a P1 incident.	Restore any one backend. Prioritize Redis (fastest recovery path to partial function). Then ES. Then Postgres (should never be down independently in this stack).

Example Worked: Case 1 (Elasticsearch Down)

Request arrives: `GET /search?q=running+shoes`

1. Check circuit breaker for Elasticsearch → OPEN (ES is down)
2. Skip ES entirely (fast-fail — no timeout needed, circuit is open)
3. Check Redis cache → cache HIT for this query
4. Return cached results
5. Log: `search.request{backend=redis_cache, es_state=circuit_open, cache_hit=true}`
6. Response time: ~5ms (Redis), not 200ms (ES)

Request arrives: `GET /search?q=obscure+product+no+one+searched+before`

1. Check circuit breaker for Elasticsearch → OPEN
2. Skip ES
3. Check Redis cache → cache MISS (never searched before)
4. Fallback to Postgres: `SELECT id, name, description FROM products WHERE to_tsvector('english', name || ' ' || description) @@ plainto_tsquery($1) LIMIT 20`
5. Return results with header: X-Search-Mode: degraded
Response body note: "Search results may be limited"
6. Log: `search.request{backend=postgres_fallback, es_state=circuit_open, cache_hit=false}`
7. Write result to Redis cache (so next identical query hits cache)
8. Response time: ~80ms (Postgres), not 200ms (ES)

What the Postgres fallback cannot do vs Elasticsearch: - Full-text relevance ranking (Postgres gives you match, not ranked match) - Faceted navigation (color, size, brand filters) - Fuzzy matching / typo tolerance - Synonym handling

The user gets results. They're less good. That is the correct tradeoff vs “search unavailable.”

Cascade Tracing Template

For any failure analysis, walk this structure:

Failing component: [name, what failure mode: down / slow / corrupted]
Bounded resources at each layer: [thread pools, connection pools, queue depth]

Layer 1 [component closest to failing dependency]:

- What fills up first?
- At what threshold does it become visible to the next layer?

Layer 2 [next component]:

- What does it see? (slow response? errors? connection failures?)
- What of its resources fills up?

Layer 3 [next component]:

- Same

User impact:

- First symptom (time, what user sees)
- Full failure (time, what user sees)

Metrics spike order:

1. [first indicator – usually internal to the failing component]
2. [second – usually latency at the next layer]
3. [third – usually error rate at the layer after that]
4. [last – usually what the dashboard shows]

Summary: Degradation Decision Matrix

Dependency	Data type	Stale OK?	Fallback exists?	Posture
Elasticsearch	Search index	Yes (minutes)	Redis cache + Postgres	Degrade gracefully
Redis	Cache	N/A (it is the cache)	Go to source	Remove cache layer, serve from source
Postgres	Canonical data	No	None	Fail fast for writes; serve from Redis/ES for reads
Payment processor	External	No	None	Fail fast with clear error
Email service	Notification	Yes (minutes to hours)	Queue for retry via DLQ	Queue + retry
Analytics service	Reporting	Yes (hours)	Drop and log	Silent drop, catch up later

Phase — 06-scaling-performance

Directory: 06-scaling-performance/

Source: 06-scaling-performance/01-scaling-decisions.md

Scaling Decisions

Scale the bottleneck, not the system. Scaling everything costs 10x and fixes nothing.

Bottleneck Identification First

Before scaling anything, find the constraint. Scaling a non-bottleneck wastes money and shifts the bottleneck somewhere harder to see.

The **USE Method** — apply per resource before any scaling decision:

Metric	What it measures	Alarm signal
Utilization	% of time resource is busy	>70% sustained
Saturation	Work waiting to be processed (queue depth)	Any queue growing over time
Errors	Failed operations on this resource	Any non-zero error rate

Resources to check in order:

1. CPU (application server, Postgres, Redis)
2. Memory (swap activity = saturation alarm)
3. Disk I/O (Postgres: `pg_stat_bgwriter`, `iostat`)
4. Network (bandwidth, packet loss, latency between services)
5. Connection pool (Postgres: `pg_stat_activity`, waiting clients)
6. Thread pool (PHP-FPM workers, Go goroutine queue)

Critical rule: the bottleneck changes after you fix it. Measure again after every intervention. The system you diagnosed last month is not the system you have today after 3x traffic growth.

Vertical Before Horizontal

Vertical scaling (bigger machine) is simpler, cheaper, and reversible. Do it first.

Dimension	Vertical	Horizontal
Implementation complexity	Low — resize and restart	High — load balancing, session affinity, distributed state
Cost	Linear until hardware ceiling	Linear but with coordination overhead
Reversibility	Easy	Hard — you've now built distributed infra
New failure modes introduced	None	Split-brain, network partition, clock skew
When you hit the ceiling	~32 cores, ~256GB RAM for most cloud instances	No hard ceiling

Horizontal scaling introduces distributed systems problems that cannot be undestroyed. Session state must be externalized. Caches must be shared or invalidated across nodes. Scheduled jobs must be deduplicated. Do not introduce these until vertical hits its ceiling.

Symfony + Postgres stack: a single 32-core RDS instance handles 5k-10k req/s for typical CRUD workloads. Most teams never need to go horizontal on Postgres.

Database Scaling Path

Work this list in order. Stop when the problem is solved.

1. **Add indexes** for slow queries. Check `pg_stat_statements` for top queries by total time. An unindexed foreign key on a 10M-row table is a ticking clock.
2. **Connection pooling via PgBouncer**. Postgres forks a process per connection. 500 direct connections = 500 processes. PgBouncer multiplexes 500 app connections into 20-50 Postgres connections. Free performance.
3. **Query optimization**. Run `EXPLAIN (ANALYZE, BUFFERS)` on slow queries. Look for sequential scans on large tables, hash joins with no statistics, N+1 in Doctrine (use `JOIN FETCH` or batch loading).
4. **Read replicas** for read-heavy load. Streaming replication to a hot standby, then route read-only queries there. Works for reporting, search, analytics. Does not help write-heavy workloads.
5. **Caching for hot reads**. Redis for data that is read frequently, changes infrequently, and can tolerate brief staleness. Requires a cache invalidation strategy (see below).
6. **Partitioning for large tables**. Partition by time range (logs, events) or list (tenant, status). Postgres 12+ declarative partitioning. Reduces index size, enables partition pruning.
7. **Sharding**. Split data across multiple Postgres instances by key. Rarely needed. Very expensive in ops complexity. If you need this, you probably already know it.

Most systems stop at step 3 or 4. Teams that jump to step 5 before doing step 1 spend weeks building cache infrastructure that a missing index would have made unnecessary.

Caching as Architectural Decision

Cache is not a performance patch. It is an architectural commitment. You now have two sources of truth.

Cache invalidation strategies:

Strategy	Consistency	Complexity	Use when
TTL	Eventual — data is stale until TTL expires	Low	Read-heavy, staleness acceptable (product catalog, static config)
Event-driven invalidation	Strong — invalidated on write	High — requires event publishing on every write	User-facing data where staleness is noticeable
Write-through	Strong — cache updated synchronously on write	Medium — write path is now two operations	Small hot datasets, low write volume
Cache-aside (lazy load)	Eventual — miss triggers DB read and cache fill	Low	General case, read-heavy, infrequent writes

What must be designed before implementation:

- What is the cache key structure? (`user:{id}:profile`, not `user_profile_{id}`)
- What invalidates this cache entry? A write to which table/event triggers invalidation?
- What is the TTL if event-driven invalidation fails? (Always have a TTL backstop)
- What happens on cache miss under high load? (Cache stampede — use probabilistic early expiration or locking)
- What does stale data look like to the user? Is it acceptable?

For the Symfony + Redis stack: use Symfony Cache with tag-based invalidation. Cache tags let you invalidate all keys associated with a domain object in one call. This is event-driven invalidation without building a separate event pipeline.

Capacity Envelope Calculation

Rough math to know if a design is in the right order of magnitude. Takes 5 minutes. Saves weeks of debugging.

Template:

```
Peak req/s = daily requests × peak multiplier / 86400
DB queries per request = (measure with profiler, estimate 5-20 for CRUD)
Peak DB queries/s = peak req/s × queries per request
Max Postgres queries/s = (connections in pool) / (avg query duration in seconds)
Is it enough? = max DB queries/s > peak DB queries/s
```

Worked example — job marketplace:

Daily searches: 50k
Peak multiplier: 10x
Peak searches/s = $50,000 \times 10 / 86,400 \approx 6 \text{ req/s (searches)}$
Daily applications: 5k → peak = $5,000 \times 10 / 86,400 \approx 0.6 \text{ req/s (writes)}$

Assume 5 DB queries per search, 3 per application write:
Peak DB queries/s = $(6 \times 5) + (0.6 \times 3) = 32 \text{ queries/s}$

PgBouncer pool of 20 connections, avg query 50ms:
Max queries/s = $20 / 0.05 = 400 \text{ queries/s}$

400 > 32 → single Postgres instance is fine with connection pooling.

If the math shows you’re within 2x of the ceiling, add a buffer and start the next scaling step. If you’re at 10% of ceiling, stop and ship.

Decision Table

Bottleneck	Traffic pattern	Team ops maturity	Recommended action
Slow queries (Postgres high CPU)	Steady, predictable	Any	Step 1: indexes + EXPLAIN
Connection exhaustion	Spiky	Any	Step 2: PgBouncer
Read-heavy, write-light	Any	Any	Step 4: read replica or Step 5: cache
Write-heavy, no read issues	Any	High	Optimize write queries, batch writes
App server CPU saturated	Growing	Medium	Vertical scale app server first
App server CPU saturated at ceiling	Growing	High	Horizontal scale app servers (stateless by design)
Go worker queue depth growing	Bursty	Any	Add worker replicas (workers are stateless, safe to scale)
Redis memory full	Any	Any	Eviction policy + key expiry audit before scaling Redis

Anti-Patterns

- **Cache before profiling:** adding Redis before identifying the slow query. You’ve now cached a slow query’s result instead of fixing the query. The query still runs, just less often.
- **Horizontal scale of stateful services:** scaling Symfony app servers when session state is in PHP files. New requests go to nodes without the session. Users get logged out.
- **“We’ll just add more servers”:** without knowing what’s saturated. If Postgres is the bottleneck, more app servers make it worse — more connections, more load.
- **Cache without invalidation strategy:** adding a 24-hour TTL to user profile data, then wondering why users see stale names after account updates.
- **Read replica without query routing:** setting up a read replica but routing all queries to primary because “it’s easier.” The replica is unused infrastructure.
- **Partitioning as first response:** partitioning a 500k-row table. Partitioning pays off at tens of millions of rows. Before that, it adds complexity with no benefit.

Šta da pitaš AI

- “Our [component] is showing [USE metric: e.g. 80% CPU utilization]. Walk me through the USE method to identify whether this is the actual bottleneck or whether something upstream is causing it.”
- “We have [X req/s] with [Y ms] average [operation] hitting [component]. Is this within the capacity envelope? Show the math and tell me at what point we’d need the next scaling step.”
- “We want to add caching for [read pattern: e.g. job listing detail pages]. Our consistency requirement is [e.g. updates visible within 60 seconds]. What are the cache invalidation options and their tradeoffs?”
- “We’re seeing slow queries in Postgres. Here’s the output of `pg_stat_statements` ordered by `total_time`: [paste]. What do I do first?”

- “We’re at step [N] in the database scaling path. Walk me through what step [N+1] requires operationally — not just the configuration, but what breaks, what to monitor, what rollback looks like.”

Source: 06-scaling-performance/02-lab-scaling.md

Lab: Scaling Decisions

No scaffolding. Work the exercises. Wrong answer with correct reasoning is better than right answer with no reasoning — show your math.

Exercise 1: Capacity Envelope

System: job marketplace on Symfony + Postgres + Redis.

Known numbers: - 100k active job listings in database - 50k daily searches - 5k daily job applications (writes) - Peak traffic: 10x daily average

Your job:

1. Calculate peak searches per second
2. Calculate peak applications per second
3. Estimate peak Postgres query load (assume 8 DB queries per search, 4 per application write)
4. Determine whether a single Postgres instance with PgBouncer (pool of 25, average query time 40ms) can handle this
5. If it cannot, what is the first thing to add? Why that and not something else?

Capacity math template:

```
Peak req/s = (daily requests × peak multiplier) / 86400

Peak DB queries/s = sum of (req type req/s × queries per request)

Max Postgres throughput = pool_connections / avg_query_duration_seconds

Verdict = max throughput vs peak load
```

Show your work. State your assumptions explicitly.

Exercise 2: Bottleneck Diagnosis

Situation: production API for the job marketplace.

Symptoms: - 3 months ago: average API response time 50ms - Today: average API response time 800ms - Traffic grew 3x over the same period - No error rate increase — 200s, just slow - No recent deployments that changed query patterns

Your job:

Walk through the diagnosis in sequence. For each bottleneck layer, state: 1. Which USE metric to check 2. How to check it (specific tool/query/command) 3. What you expect to find given the symptoms 4. What the fix is if this layer is the bottleneck

Bottleneck layers to evaluate in order:

```
Layer 1: Application server (PHP-FPM workers / Symfony)
Layer 2: Database connections (connection pool saturation)
Layer 3: Database query performance (query execution time)
Layer 4: Database I/O (disk, buffer cache hit rate)
Layer 5: Network (between app and DB)
Layer 6: Redis (if caching layer exists)
```

Hint: 3x traffic growth with no code changes and no errors usually points to one of two places. You should identify which two and explain why.

USE Method Checklist

Run this before any scaling conversation.

Resource: _____

Utilization

- Current value: _____%
- How measured: _____
- Alarm threshold: 70% sustained

Saturation

- Queue depth / wait time: _____
- How measured: _____
- Alarm threshold: Any non-zero growth over time

Errors

- Error rate: _____
- How measured: _____
- Alarm threshold: Any non-zero

Verdict:

- ☐ This is the bottleneck – address first
- ☐ This is not saturated – check next resource
- ☐ Inconclusive – need more data (specify what)

Database Scaling Path — Quick Reference

1. Indexes → `pg_stat_statements`, `EXPLAIN (ANALYZE, BUFFERS)`
2. PgBouncer → `pg_stat_activity`, waiting connection count
3. Query optimize → `EXPLAIN` plan, N+1 detection, Doctrine profiler
4. Read replica → `pg_stat_replication`, replica lag
5. Redis cache → cache hit rate, invalidation strategy defined
6. Partitioning → table size > 10M rows, sequential scan on date range
7. Sharding → last resort, requires full architecture change

Worked Example: Connection Pool Math

Scenario: Symfony app, 20 PHP-FPM workers per server, 3 app servers.
Direct connections to Postgres: $20 \times 3 = 60$ connections.
Postgres `max_connections`: 100 (default).
At peak traffic, 60 connections is fine. At 5 app servers: 100 connections = hitting the ceiling.

With PgBouncer (transaction pooling):

- App → PgBouncer: $20 \times N$ connections per server (cheap, PgBouncer handles these)
- PgBouncer → Postgres: 20 real connections (configurable)

Throughput at 20 real connections, 40ms avg query:
 $20 / 0.04 = 500$ queries/s

This is 8x more throughput than without pooling, from configuration alone.

Self-Check

After completing both exercises, verify:

- ☐ Exercise 1: Did you show the arithmetic, not just the conclusion?
- ☐ Exercise 1: Did you state whether single Postgres is sufficient before recommending what to add?
- ☐ Exercise 1: Did you recommend step 1 of the database scaling path, not step 5?

- [] Exercise 2: Did you check connection saturation before assuming slow queries?
- [] Exercise 2: Did you specify the exact tool/query for each layer, not just “check the database”?
- [] Exercise 2: Did you identify the two most likely bottlenecks given 3x traffic growth with no code changes?

Reference Numbers for Estimation

Component	Rough throughput ceiling	Notes
Single Postgres (8-core RDS)	2k-5k simple queries/s	Drops fast with complex joins
PgBouncer (20 Postgres connections)	~500 queries/s at 40ms avg	Adjust for your query profile
Redis	100k ops/s single thread	Rarely the bottleneck
PHP-FPM worker	1 request at a time	Worker count = max concurrency
Go worker (goroutine)	Thousands concurrent	Bottleneck is usually downstream, not the worker
Network (same datacenter)	<1ms latency	If you're seeing >5ms, investigate

Phase — 07-communication-decisions

Directory: 07-communication-decisions/

Source: 07-communication-decisions/01-adr-and-stakeholders.md

ADRs and Stakeholder Communication

An architecture decision no one understands will be reversed by the next engineer who touches it.

ADR Anatomy

The format that actually gets read and accepted. Every field has a reason.

```
# ADR-{number}: {Decision verb + subject}
# Example: "ADR-007: Use outbox pattern for order event publishing"
# Not: "ADR-007: Event publishing approach"

**Status:** Proposed | Accepted | Superseded by ADR-{N}

## Context
What is the situation that requires a decision?
What constraints exist — technical, organizational, compliance?
What is the cost of not deciding?

## Decision
One clear sentence. "We will use X to solve Y."
Not: "We've decided to evaluate options for..."
Not: "We should consider..."

## Consequences
**Better:** What improves because of this decision.
**Worse:** What gets harder or more expensive. Be honest — a whitewashed ADR is useless.
**Required:** What must now be built, learned, or changed because of this decision.

## Alternatives Considered
For each alternative:
- What it is
- Why we did not choose it (specific, not vague)

An ADR without "Alternatives Considered" is an announcement, not a decision record.
```

What makes an ADR get accepted vs ignored:

- Title is a decision, not a topic. "Use X for Y" forces clarity. "Approach to Y" defers it.
 - Context explains the forcing function. What happens if we don't decide? ADRs with no urgency get deferred.
 - Consequences section is honest. If you only list upsides, engineers distrust it — they know there are tradeoffs.
 - Alternatives section shows you thought it through. "We considered X but it has [specific cost we cannot absorb]" builds trust faster than "we chose the best option."
-

Example ADR

```
# ADR-012: Use PgBouncer for Postgres connection pooling

**Status:** Accepted

## Context
The job marketplace API runs on 4 PHP-FPM servers with 20 workers each (80 direct connections).
At current growth trajectory, we will hit Postgres max_connections (100) within 6 weeks.
Direct connection scaling requires either raising max_connections (increases RAM per connection)
or adding PgBouncer as a multiplexing layer. Raising max_connections is a stopgap with
diminishing returns; PgBouncer is the standard solution for this class of problem.

## Decision
We will deploy PgBouncer in transaction pooling mode in front of Postgres, targeting
20 real Postgres connections from PgBouncer regardless of application connection count.

## Consequences
**Better:** Application servers can open as many connections as needed without Postgres load.
Estimated throughput ceiling increases from ~80 concurrent queries to ~500 queries/s.
**Worse:** Prepared statements and advisory locks do not work in transaction pooling mode.
Any code using these must be refactored before deployment.
**Required:** Audit of Doctrine configuration for prepared statement usage. Update deployment
runbook. Add PgBouncer metrics to monitoring dashboard.

## Alternatives Considered
- **Raise max_connections to 200:** Postgres allocates ~10MB RAM per connection. At 200
  connections that's 2GB RAM consumed by idle connections. This is a stopgap that doesn't
  solve the scaling path — we'd be back here at 10 app servers.
- **Switch to async PHP (Swoole/RoadRunner):** Would reduce connection count by reusing
  connections across requests. Estimated 2-3 sprint effort to migrate, high risk of
  incompatibility with existing Symfony middleware. Disproportionate cost for this problem.
```

Stakeholder Narrative

The same decision explained differently for different audiences. Same facts, different emphasis. This is not spin — it's communication competence.

Decision being communicated: Migrating from synchronous DB writes for order events to the outbox pattern.

For engineers:

The current approach writes the order record and publishes the event in separate operations. If the app crashes between the two, the event is lost and downstream services don't get notified. The outbox pattern writes both to Postgres in the same transaction, then a separate process reads the outbox table and publishes to the queue. This guarantees at-least-once delivery. You'll need to write idempotent consumers — that means checking event IDs before processing, because the same event may arrive twice during retry. The outbox poller will be a new Go worker. Doctrine entity writes stay the same; you add an `OutboxEvent` row to the same transaction.

For the engineering manager:

We have a data integrity risk: order events can be silently lost under failure conditions, causing downstream services (fulfillment, notifications) to miss orders. This has happened twice in staging, not yet in production. The fix (outbox pattern) requires 1 sprint to implement, 1 sprint to test and roll out. The tradeoff is a small increase in write latency (~5ms) and a new Go worker to operate. The alternative — living with the risk — means we'll eventually have a production incident where orders are processed but fulfillment isn't triggered.

For the CTO or VP Engineering:

We have a reliability gap: order processing can succeed while order fulfillment silently fails. At current scale this is recoverable manually, but at 10x volume it becomes a support and refund cost. We're addressing it this sprint with an industry-standard

pattern (transactional outbox). The cost is 2 sprints of engineering time. The risk of doing nothing scales with our order volume. This closes a known gap before it becomes a customer-visible incident.

The 5-Minute Pitch Format

When you need a decision in a meeting, not a document.

"We have [problem].
It's causing [observable business impact – not technical impact].
We have [N] options: [A], [B], [C].
I recommend [X] because [specific reason tied to our constraints].
The main risk is [Y] and we'd mitigate it by [Z].
I need [decision / resource / timeline] by [date]."

What this format does: gives the decision-maker what they need without requiring them to understand the technical details. The business impact sentence is the load-bearing sentence — if you can't state it, you're not ready to ask for the decision.

Example:

"We have connection exhaustion in Postgres. It's causing request failures at peak traffic, currently in staging — production is 6 weeks away at current growth. We have three options: raise max_connections, add PgBouncer, or reduce PHP-FPM workers. I recommend PgBouncer because it solves the problem permanently without capping our concurrency. The main risk is that it doesn't support prepared statements, and we'd mitigate that by auditing our Doctrine config this week before deploying. I need a decision today so we can start the work this sprint."

Defending Under Pressure

When someone challenges your decision in a meeting.

Challenge	Response pattern	What not to do
"Why didn't you consider X?"	"We did. X costs [specific thing] that we can't absorb because [constraint]."	"We didn't think of that" or dismissing it without engaging
"This is overengineered"	"The constraint that requires this is [X]. Without it, [failure mode] occurs."	Getting defensive, restating the solution without addressing the concern
"Can't we just do Y?"	"Y gives up [specific guarantee] that is non-negotiable for [reason]."	"Y won't work" without explaining what it costs
"This will take too long"	"The estimate is [N]. We can descope [optional part] to [N/2] if needed."	Defending the estimate without offering a path
"I've read that [other approach] is better"	"For [use case], yes. Our constraint is [X], which changes the tradeoff because [reason]."	Dismissing the source or getting into a credibility contest

What to never do: get defensive, dismiss the challenge, defer without committing to when you'll have an answer. A challenge is information. Treat it as such.

Losing Gracefully

When the decision goes against you.

1. **State your objection on the record.** In the ADR under "Dissenting views" or in the meeting notes. Not to relitigate — to document that the risk was surfaced.
2. **Commit to executing the agreed decision.** Disagreement in discussion, alignment in execution. An architect who undermines decisions they lost is a team hazard.
3. **Note the condition under which you'd revisit.** "If we see [specific signal — error rate, incident type, metric threshold], I'll raise this again." This converts your concern into a monitoring condition, not a standing grievance.

Šta da pitaš AI

- “Here’s the context for an architecture decision: [paste context]. Help me write an ADR. Challenge me if the Consequences section is too optimistic or the Alternatives section is too thin.”
- “I need to explain [technical decision] to a CTO who doesn’t have a technical background. What business impact framing would make this land? What am I probably leaving out?”
- “I’m preparing to defend [decision] in a meeting. What are the three most likely challenges I’ll face, and what’s the strongest response to each?”
- “Here’s an ADR I wrote: [paste]. What would make an experienced engineer distrust this? What’s missing from the Alternatives section?”
- “I lost a decision in a meeting. The decision was [X] and I recommended [Y]. Write the ‘dissenting view’ paragraph for the ADR that records my concern without sounding adversarial.”

Source: 07-communication-decisions/02-lab-communication.md

Lab: Communication and Decisions

Write the artifacts. Generic answers are wrong answers — use the system context (Symfony API + Go worker + Postgres + Redis + queue).

Exercise 1: Write an ADR

Decision to document: Adopt the outbox pattern for the `order-placed` event in the e-commerce system.

Background facts to incorporate:

- Currently: `OrderService::placeOrder()` writes to Postgres and publishes to RabbitMQ in two separate operations
- The failure window between the two operations has caused missed events in staging (twice in the last sprint)
- The queue is the source of truth for downstream services: fulfillment, notifications, analytics
- An order that succeeds but whose event is never published causes a silent failure — no error, just a missing fulfillment record
- Alternatives you considered: two-phase commit (XA transactions), dual-write with reconciliation job, change data capture (Debezium)

Write the full ADR using this template:

```
# ADR-{number}: {Title as a decision, not a topic}

**Status:** {Proposed | Accepted | Superseded}

## Context
{What is the situation? What are the constraints? What is the cost of not deciding?}

## Decision
{One sentence.}

## Consequences
**Better:** {What improves}
**Worse:** {What gets harder — be honest}
**Required:** {What must now be built, learned, or changed}

## Alternatives Considered
- **{Option A}:** {Why not chosen — specific}
- **{Option B}:** {Why not chosen — specific}
- **{Option C}:** {Why not chosen — specific}
```

Grading your own ADR:

- ☐ Title is a decision (“Use X for Y”), not a topic (“Approach to Y”)
- ☐ Context states the forcing function — what breaks if we don’t act?
- ☐ Decision is one sentence, no hedging
- ☐ Consequences section includes something honest in “Worse”

- [] Each alternative has a specific reason for rejection, not “it wasn’t the best fit”

Exercise 2: Stakeholder Translation

Decision to communicate:

“We need to migrate from RabbitMQ to Kafka because our consumer group scaling is hitting RabbitMQ’s architectural limits at our projected message volume. RabbitMQ deletes messages after acknowledgement; Kafka retains them, allowing replay and multiple independent consumers without queue fan-out configuration. At 500k messages/day with 6 downstream consumers, the operational overhead of RabbitMQ exchange/queue topology is becoming a liability.”

Write 3 versions. Each version: 3-5 sentences.

Version 1 — For engineers:

Cover: what technically changes, what they need to learn, what breaks, what gets better. Be specific about the operational implications (consumer group config, offset management, message retention policy).

Version 2 — For the engineering manager:

Cover: cost, timeline, risk, what the team needs to learn, what happens if we don’t do this. No jargon. The EM does not need to know what a consumer group is — they need to know what the risk is and what it costs to fix it.

Version 3 — For CTO or VP Engineering:

Cover: business risk of not acting, cost horizon of acting, what we’re committing to, what we’re foreclosing. One sentence on the tradeoff. No queue topology discussion.

Check: Could each version stand alone without the others? Would a person in that role understand it without asking follow-up questions?

Exercise 3: Meeting Pressure Drill

Scenario:

You recommended PostgreSQL with read replicas over MongoDB for a new feature: a candidate profile search with filtering by skills, location, and availability. You chose Postgres because the data model is relational (candidates have applications, which have statuses, which link to jobs), the team knows SQL, and read replicas give you sufficient read scaling.

Your CTO says in the architecture review:

“I’ve read that MongoDB scales much better for this kind of flexible document data. Why are we not using it?”

Part A: Write your response (3-5 sentences)

Requirements for the response: - Acknowledge the CTO’s point before disagreeing (don’t open with “actually”) - State the specific constraint that makes MongoDB a worse choice for this system - Do not say MongoDB is bad — say why Postgres fits better given what we have - End with something that closes the challenge, not opens a debate

Part B: Write the ADR addition

Write 3-5 sentences to add to the “Alternatives Considered” section of your ADR that would have pre-empted this question.

Format:

```
- **MongoDB:** {Why not chosen – specific to this system's constraints}
```

Example ADR (Reference — Different Decision)

Use this to calibrate format, not content. Write your own ADR for Exercise 1.

ADR-009: Deploy Redis as the session store for Symfony API

****Status:**** Accepted

Context

Symfony API runs on 3 horizontally scaled app servers. PHP file-based sessions are local to each server. Sticky sessions (load balancer session affinity) are currently used as a workaround, but this creates uneven load distribution and breaks during deployments when instances are replaced. Redis is already in the stack for caching.

Decision

We will configure Symfony to store sessions in Redis using the existing Redis cluster, with a 2-hour TTL matching the current session lifetime.

Consequences

****Better:**** Sessions survive app server restarts and deployments. Load balancer can use round-robin without sticky sessions. Deployment downtime for sessions eliminated.
****Worse:**** Redis becomes a hard dependency for authenticated requests. Redis failure means all sessions are lost. Must ensure Redis is included in the high-availability plan.
****Required:**** Symfony session handler configuration change. Redis connection pool sizing review (sessions add ~1KB per active user). Runbook update for Redis failure scenario.

Alternatives Considered

- ****Sticky sessions (status quo):**** Does not survive deployments – sessions lost on instance replacement. Causes load imbalance when one user has a heavy session. Not viable at scale.
- ****Database session store (Postgres):**** Works but adds load to Postgres for every authenticated request. Redis is faster and already in the stack for this type of workload.
- ****JWT stateless auth:**** Would eliminate session storage entirely. Estimated 3-sprint migration effort. Token revocation requires a blocklist (same Redis infrastructure anyway). Disproportionate cost for a deployment friction problem.

Self-Check

Exercise 1: - ☐ ADR title is a decision, not a topic - ☐ Context includes the two staging incidents as the forcing function - ☐ “Worse” includes the latency cost and the new worker to operate - ☐ Two-phase commit alternative explains specifically why it was rejected (hint: Postgres and RabbitMQ don’t share a transaction coordinator)

Exercise 2: - ☐ Engineer version mentions consumer group offset management or message retention — something technically specific - ☐ EM version has no acronyms that weren't explained - ☐ CTO version is one business risk sentence, one cost sentence, one commitment sentence — no topology discussion

Exercise 3: - ☐ Part A opens by acknowledging the point before disagreeing - ☐ Part A mentions the relational join requirement or existing team SQL knowledge as the specific constraint - ☐ Part B is written as “MongoDB: [specific reason]” not “MongoDB was not chosen because it wasn’t the right fit”

Phase — 08-capstone-drills

Directory: 08-capstone-drills/

Source: 08-capstone-drills/01-scenario-drills.md

Capstone Drills: Scenario-Based Practice

Six scenarios drawn from the full spine (Symfony API + Go worker + Postgres + Redis + queue). Each asks you to produce specific outputs before reading the good answer. Self-evaluate: if your answer matches the structure and reasoning, not just the words, you're on track.

Scenario 1: “The CTO Wants Microservices”

Situation Your Symfony monolith handles 200 req/s at peak. Team has 4 engineers. Response times are fine. The CTO read an article about Netflix and wants to “modernize the architecture with microservices.” You think this is the wrong move.

Produce 1. The 3 constraints you'd state that push back on this. 2. A counter-proposal: what you'd suggest instead. 3. The one condition under which you'd agree to split.

What to ask AI

We have a Symfony monolith, 4 engineers, 200 req/s peak, no scaling or ownership pain. Leadership wants microservices. Give me the three strongest constraint-based arguments against splitting now, framed for a non-technical executive.

What specific operational costs does a team of 4 incur when moving from a monolith to microservices? Be concrete: CI/CD, observability, on-call, deployment coordination.

Good answer includes - Conway's Law stated explicitly: 4 engineers will produce a distributed monolith, not independent services, because communication paths don't map to service ownership - Operational cost itemized: separate deploys, distributed tracing, inter-service auth, network failure modes — each adds work without team to absorb it - The problem microservices solve (independent scaling, independent deployability per domain, separate failure domains) is not a problem you currently have - Counter-proposal: modular monolith — enforce domain boundaries inside the codebase via namespace conventions and no cross-domain direct DB access - The one condition: a single domain needs to scale independently AND has a clearly separate ownership boundary. Not “because the CTO read an article.”

Scenario 2: “Postgres Is Too Slow”

Situation An engineer escalates: “Postgres is our bottleneck. We need to switch to Cassandra.” No profiling data exists. The Go worker that processes orders is experiencing latency spikes under load.

Produce 1. The 5 things you'd measure before agreeing to any infrastructure change. 2. The diagnosis sequence using the USE method (Utilization, Saturation, Errors). 3. The earliest point at which Cassandra becomes the right answer.

What to ask AI

A Postgres instance is showing latency spikes under load. I have no profiling data yet. Give me a prioritized diagnostic checklist, starting from the cheapest checks (query analysis) before any infrastructure change. Include specific queries or tools for each step.

Apply the USE method to Postgres latency diagnosis. For each of Utilization, Saturation, and Errors, what metric do I look at and what threshold signals a problem?

Good answer includes - Measure first: (1) slow query log / `pg_stat_statements` — are specific queries responsible? (2) index coverage — run `EXPLAIN ANALYZE` on the hot queries (3) connection pool saturation — are workers waiting for connections? (4) disk I/O — are you hitting read amplification? (5) table bloat / dead tuples — when was `VACUUM` last run? - USE sequence: Utilization = CPU %, I/O wait, connection count vs `max_connections`; Saturation = query queue depth, replication lag; Errors =

lock waits, deadlocks, checkpoint warnings - Cassandra is the right answer only when: query patterns are genuinely write-heavy at scale where Postgres write amplification is proven, data is wide-row or time-series shaped, and you've exhausted read replicas, connection pooling (PgBouncer), and index tuning. Not before. - "Switching databases" is never the first answer. Missing index is.

Scenario 3: "We Lost Messages in the Queue"

Situation 3am. Orders were placed but fulfillment never triggered. Root cause: the Go worker was restarted mid-processing — it had consumed messages off the queue but hadn't committed the fulfillment action to Postgres before the restart.

Produce 1. Immediate triage steps (what do you do right now). 2. Root cause hypothesis: what exactly broke and why. 3. The architectural change that prevents recurrence. 4. What you'd write in the postmortem.

What to ask AI

A queue consumer was restarted mid-processing. Messages were dequeued but downstream actions were not completed. Walk me through a root cause analysis and the architectural patterns that prevent this failure class.

Explain the transactional outbox pattern. When does it apply, what does it guarantee, and what are its implementation costs in a Symfony + Postgres + RabbitMQ stack?

Good answer includes - Triage: identify which orders are in the gap (placed_at timestamp range where queue consumer was down), check Postgres for fulfillment records vs order records, replay or manually process missing orders - Root cause: at-least-once delivery was not implemented — the worker ACKed or auto-ACKed before processing was durable. Restart lost in-flight work. - Architectural change: (1) move to manual ACK — only ACK after the fulfillment write commits to Postgres; (2) implement idempotency key on fulfillment so replayed messages don't double-process; (3) add a Dead Letter Queue for messages that fail repeatedly so they don't disappear; (4) for the full solution, transactional outbox — write event to outbox table in same transaction as the originating change, separate relay publishes to queue - Postmortem: state the failure (not blame), identify the assumption that broke (processing is atomic with consumption), name the mitigation, give timeline for implementation

Scenario 4: "New Feature That Breaks the Architecture"

Situation Product wants real-time inventory counts on the product listing page. Current architecture: inventory stored in Postgres, page queries DB on load. Requirement: inventory updates visible across 50k product listings within 100ms of a stock change.

Produce 1. Why the current architecture cannot satisfy this requirement. 2. Three options with tradeoffs. 3. The option you'd recommend and why.

What to ask AI

I need to push inventory updates to 50k product listing pages within 100ms of a stock change. Current setup: Postgres source of truth, pages query on load. Give me 3 architectural options with explicit tradeoffs on consistency, infrastructure cost, and implementation complexity.

Compare Redis pub/sub, SSE (Server-Sent Events), and WebSockets for pushing real-time inventory updates to a browser. I have a Symfony backend and 50k concurrent product pages. What breaks at scale for each?

Good answer includes - Why current architecture fails: polling at 100ms interval across 50k pages = 500k queries/second. DB will not survive it. Even 1s polling is 50k q/s. This is a push problem, not a query optimization problem. - Option A — Redis cache + event invalidation: inventory change writes to Postgres and publishes invalidation event, Symfony reads from Redis cache on page load, Go worker updates cache on event. Tradeoff: cache staleness window, added complexity, but solves DB pressure. Read path stays fast. - Option B — SSE from Symfony: browser holds open connection, server pushes inventory deltas on change. Tradeoff: connection count at 50k concurrent is real infrastructure cost, but low latency and no polling. Correct for truly real-time. - Option C — WebSockets via dedicated service: same push semantics as SSE but bidirectional. Overkill unless you already need bidirectional comms. Higher complexity for same result. - Recommendation: Redis cache with invalidation for inventory reads (solves the query pressure), SSE for the real-time push to browser only on pages actively being viewed. Don't hold 50k connections open for users who aren't looking at the page. - The wrong answer: "we'll add an index and query faster." This is a throughput problem at the DB layer, not a latency problem.

Scenario 5: “Stakeholder Doesn’t Trust the Decision”

Situation You’ve proposed the transactional outbox pattern to ensure reliable event publishing. In the design review, a senior engineer says: “This is over-engineering. We’ve shipped this system for two years without it. You’re solving a problem we don’t have.”

Produce 1. Your in-meeting response (3-5 sentences, out loud). 2. What you’d add to the ADR. 3. The condition under which the senior engineer is right.

What to ask AI

Help me write a non-defensive response to "this is over-engineering" when proposing the transactional outbox pattern. The audience is a senior engineer who hasn't seen the failure this pattern prevents. I need to name the specific failure clearly.

Write the "Considered Alternatives" and "Consequences" sections of an ADR for adopting the transactional outbox pattern. Include the explicit failure case the pattern prevents and the cost of implementing it post-incident vs now.

Good answer includes - In-meeting response: name the specific failure (not “reliability” — “an event fails to publish after the DB write commits, because they’re not in the same transaction; the order exists in Postgres but the fulfillment queue never sees it”). Acknowledge it hasn’t happened. State the cost to implement now (one outbox table, one relay process) vs after the incident (data reconciliation, customer impact, incident cost). Don’t argue. Give the engineer a way to agree without losing face. - ADR addition: “Rejected alternative: fire-and-forget publish after DB commit. Risk: partial failure leaves system in inconsistent state with no recovery path. Cost of mitigation post-incident exceeds implementation cost now.” - When the senior engineer is right: if the service publishes events that are informational only (no downstream state change depends on them), at-least-once delivery with occasional loss is acceptable. The outbox is overhead if the subscriber can tolerate gaps. State this in the ADR explicitly.

Scenario 6: “Design a New System from Scratch”

Situation Design a job application tracking system. Employers post jobs. Applicants apply. Both sides can message each other. Rough scale: 10k new jobs/day, 100k applications/day, high-volume messaging.

Produce 1. Constraints you’d gather before designing anything. 2. Components and their boundaries. 3. Storage decision for each data type. 4. The one async boundary you’d introduce first and why. 5. The SLO you’d propose for job search.

What to ask AI

Before designing a job application tracking system, what constraints should I gather from stakeholders? Give me a list organized by: scale, consistency requirements, operational constraints, and non-functional requirements.

I'm designing a system with 10k jobs/day, 100k applications/day, and high-volume messaging between employers and applicants. Identify the three subsystems most likely to have independent scaling or failure characteristics, and explain why splitting them makes sense at this scale.

Compare Postgres, Elasticsearch, and a dedicated search service for job search at 10k new documents/day. What query patterns drive the decision, and at what scale does Postgres full-text search become insufficient?

Good answer includes - Constraints to gather: read/write ratio per subsystem (search is read-heavy, messaging is write-heavy), consistency requirement per operation (is a missed message acceptable? Is a double-application a problem?), existing infrastructure, team size and operational capacity, SLA per user-facing feature - Components: (1) Job service — CRUD for job listings, search index; (2) Application service — tracks application state machine; (3) Messaging service — high-volume, append-only, separate from structured data; (4) Notification service — fan-out of events to email/push - Storage decisions: jobs in Postgres + Elasticsearch for search; applications in Postgres (structured, relational, low write volume relative to read); messages in Postgres partitioned by conversation or a dedicated append-only store (messages are immutable, high write volume, accessed by range); Redis for session, rate limiting, notification deduplication - Async boundary: application submission triggers downstream processing (employer notification, matching, analytics) via queue. This is the highest-volume write path and the downstream steps do not need to be synchronous with the HTTP response. Decoupling here prevents application submission latency from being coupled to notification infrastructure. - Search SLO: p99 < 200ms for job search queries, 99.5% availability.

Justify why: search is the primary acquisition path; degraded search = users leave. Index staleness acceptable at 60s (new job visible within 1 minute of posting). - Demonstrates constraints-first: you cannot design the storage until you know the query patterns. You cannot design the async boundary until you know the consistency requirement per operation.

Source: 08-capstone-drills/02-what-to-ask-ai.md

What to Ask AI: Architect-Level Prompt Reference

Before using this sheet: AI gives you options and analysis. You make the decision. The quality of your output equals the quality of your constraints — if you don't give AI the real constraints, you get generic advice. Every prompt below has a [placeholder]. Fill it with your actual situation before sending.

System Design from Scratch

Use these to generate a structured option space before committing to any design.

I'm designing [system name]. Constraints: [team size], [traffic estimate], [existing stack], [operational maturity], [consistency requirement]. Before proposing any design, list the questions you'd ask to clarify the architecture drivers.

Given these constraints: [list constraints], generate 3 architectural options for [system]. For each option, state: what problem it solves best, what it makes harder, and at what scale it breaks down.

I'm designing [system]. What are the top 5 failure modes I should design against, given [key characteristic: high write volume / external integrations / eventual consistency / stateless workers]? For each failure mode, name a mitigation.

What assumptions am I implicitly making in this design: [paste your design sketch]? List each assumption and the risk if it's wrong.

Evaluating a Tradeoff

Use these when you have two or more options and need to make a decision with justification.

I'm choosing between [Option A] and [Option B] for [problem]. My constraints are: [list constraints]. Evaluate each option against each constraint explicitly. Do not recommend until you've completed the matrix.

What does [Option A] cost to operate at [scale], given a team of [size]? Include: deployment complexity, observability requirements, on-call burden, and failure recovery procedures.

I'm leaning toward [option] for [reason]. What's the strongest argument against this choice, given [constraint]? I want the counterargument, not confirmation.

At what scale or load does [Option A] stop being the right choice and [Option B] become necessary? Give me a concrete threshold, not a vague "it depends."

Failure Diagnosis

Use these during incidents or in post-incident design review.

[System] is exhibiting [symptom: latency spike / dropped messages / 503s]. Walk me through a diagnosis sequence starting from the cheapest observable signal and escalating to infrastructure changes only if the cheaper checks don't explain the problem.

Apply the USE method (Utilization, Saturation, Errors) to [component: Postgres / RabbitMQ / Go worker / Redis]. For each dimension, what metric do I look at, what tool surfaces it, and what threshold indicates a problem?

A cascade failure occurred: [describe sequence of events]. Trace the failure path backward. What was the proximate cause, what was the contributing condition, and what architectural assumption broke?

Here is our postmortem draft: [paste draft]. What is missing from the "contributing factors" section? What mitigation is implied but not stated? What would prevent this class of failure, not just this specific incident?

Storage Decisions

Use these before choosing a database or proposing a migration.

My application has these query patterns: [list read patterns], [list write patterns]. Data shape: [describe]. Volume: [rows/day or GB/month]. Evaluate Postgres, Redis, and [alternative] against these patterns. Be specific about where each breaks.

I'm considering migrating from [current store] to [target store]. What are the migration risks specific to this transition? Include: data consistency during cutover, rollback complexity, and the operational burden on a team of [size].

At what data volume or query rate does Postgres full-text search become insufficient for [use case]? What are the specific signals that indicate I've hit that ceiling?

I need to store [data type: time-series events / messages / user sessions / audit log]. What are the top 3 storage options for this access pattern? For each, state: what it optimizes for, what it sacrifices, and what the operational cost is.

Async and Event Design

Use these when designing queues, consumers, or event-driven flows.

I'm designing a [consumer type] that processes [event type] from [queue]. What are the idempotency requirements? Walk me through how to implement an idempotency key for this specific operation: [describe operation].

Design a Dead Letter Queue strategy for [queue/consumer]. Include: what triggers DLQ routing, how I alert on DLQ depth, and the replay procedure when the underlying cause is fixed.

My event schema is [describe current schema]. I need to add [new field / change field type / remove field]. What schema evolution strategy applies here? What breaks for consumers that haven't updated yet?

Explain the transactional outbox pattern for [stack: Symfony + Postgres + RabbitMQ]. What does it guarantee, what does it not guarantee, and what is the implementation cost? When is it overkill?

Communication and Decisions

Use these to translate technical decisions into stakeholder language or to pressure-test an ADR.

I've decided to [technical decision]. My audience is [role: CTO / product manager / senior engineer who disagrees]. Write a 3-sentence explanation of why this decision was made, what it prevents, and what it costs.

Review this ADR: [paste ADR]. What is missing from the "Considered Alternatives" section? What implicit assumption in the decision is not stated? What would make a future engineer question this decision?

A stakeholder said "[objection]" about [technical decision]. Generate the strongest version of their argument. Then give me a response that addresses the substance without being defensive.

I need to explain [technical risk: message loss / cache inconsistency / cascading failure] to a non-technical stakeholder. Use an analogy. Then state the cost of the mitigation in non-technical terms.

Phase — 09-api-design

Directory: 09-api-design/

Source: 09-api-design/01-api-decisions.md

API Design Decisions

Architects sign off on API contracts. A bad API contract costs years.

REST vs gRPC vs GraphQL

Decision table. Pick the column that matches your situation.

Factor	REST	gRPC	GraphQL
Consumer type	External/public, broad clients	Internal service-to-service	Multiple consumers with divergent data needs
Performance	Adequate for most	High-throughput, low-latency critical	Acceptable overhead
Contract rigidity	Loose (JSON schema optional)	Strict (Protobuf required)	Flexible by design
Human-readable	Yes	No (binary)	Yes
Browser support	Native	Requires proxy (grpc-web)	Native
Tooling	Ubiquitous	Language-specific, good Go/Java	Client-side query complexity
Streaming	Awkward (SSE/polling)	First-class (bidirectional)	Subscriptions supported
Schema drift risk	High	Low (breaking at compile time)	Medium
Symfony support	Native	Requires bridge	Bundle available

Decision rules: - Default to REST unless you have a specific reason not to. - Use gRPC for Symfony → Go worker communication when call volume is high or latency matters. - Use GraphQL only when multiple consumers (mobile, web, third-party) genuinely need different shapes of the same data. Not because it's modern. - Do not expose gRPC to public consumers without a REST/HTTP gateway in front. - Do not use GraphQL for internal service-to-service. You pay schema flexibility overhead for no benefit.

API Contract as a Commitment

Once an API is consumed externally, breaking changes cost more than the feature was worth.

Breaking changes — these break existing consumers silently or loudly:

Change	Why it breaks
Remove a field from response	Consumers fail on missing key
Rename a field (<code>user_id</code> → <code>userId</code>)	Consumers read null or error
Change field type (<code>int</code> → <code>string</code>)	Consumers fail type assertions
Change HTTP status code (200 → 201)	Consumers checking exact status break
Change URL structure (<code>/orders/{id}</code> → <code>/order/{id}</code>)	All bookmarked/hardcoded URLs 404
Remove an endpoint	All consumers of that endpoint break
Make optional field required	Consumers not sending it get 422

Non-breaking changes — safe to ship without a version bump:

Change	Why it is safe
Add a new optional field to response	Consumers ignore unknown fields
Add a new optional request parameter	Existing requests still valid
Add a new endpoint	Consumers don't call it unless they opt in
Add a new HTTP status code for new error cases	Consumers already handle unknown errors (if well-written)
Expand enum values (with caution)	Only safe if consumers handle unknown enum values

Practical rule: If you are unsure whether a change is breaking, assume it is. Check consumer code before shipping.

Versioning Strategies

Strategy	Example	Pros	Cons
URL versioning	GET /v1/orders	Obvious, debuggable, cacheable, easy to route in Nginx/Symfony	URL "pollution", version in wrong layer semantically
Header versioning	API-Version: 2	Cleaner URL design	Invisible in browser, harder to test, harder to route
Content negotiation	Accept: application/vnd.api+json;version=2	RFC-correct	Complex, poor tooling support, nobody does it well

Verdict: URL versioning. It wins on debuggability and operational simplicity. Header versioning is architecturally cleaner but the operational cost is real — logs, routing rules, client testing all become harder.

Pick one strategy. Enforce it at the gateway level. Never mix URL and header versioning in the same API.

Symfony implementation: Route prefix per version in `config/routes/`. Each version gets its own controller namespace. Shared logic extracted to services, not shared between controllers.

Deprecation Process

Steps to retire a version without breaking consumers:

- 1. Announce sunset date** — add `Sunset` and `Deprecation` headers to all responses from the deprecated version. `Sunset: Sat, 31 Dec 2025 23:59:59 GMT Deprecation: Mon, 01 Jul 2025 00:00:00 GMT Link: <https://api.example.com/v2/docs>; rel="successor-version"`
- 2. Publish migration guide** — diff between v1 and v2 response shapes, with code examples for major consumers.
- 3. Monitor usage** — log requests to deprecated endpoints. Track by consumer (API key, user agent). Do not cut off consumers still actively using it.
- 4. Notify consumers directly** — email/Slack to known API key holders. Do not rely on headers alone.
- 5. Hard cutoff** — return 410 Gone after sunset date. Not 404. 410 signals permanent removal.

Minimum sunset windows: - External/public API: 6 months - Partner integrations: 6 months - Internal services you own: 3 months - Internal services other teams own: 3 months, with a team sign-off

Do not let sunset windows slip. A slipped sunset date signals to consumers that deadlines are not real.

Pagination Architecture

Strategy	How it works	Stable on insert?	Stateless?	Use when
Offset	?page=2&per_page=20	No — inserts shift pages	Yes	Small static datasets, admin panels
Cursor	?after=eyJpZCI6MTIzfQ==	Yes	Yes	High-volume, frequently updated data
Keyset	?after_id=123	Yes	Yes	Simple cases, single sort key

Offset problem in practice: - User loads page 2 of job listings. While they read, 5 new jobs are inserted at the top. - Page 3 now contains items that were on page 2. User sees duplicates or misses items. - At 100k rows with frequent inserts, offset pagination is unreliable by default.

Default: cursor-based pagination. Encode the cursor as base64 JSON (`{"id": 123, "created_at": "..."}`). Opaque to consumers, cheap to implement, stable under inserts.

Symfony + Doctrine: Implement as a `PaginatorService` with `WHERE id > :cursor ORDER BY id ASC LIMIT :limit`. Return `next_cursor` in response envelope.

Idempotency Keys

POST requests that must not duplicate need idempotency keys. Non-negotiable for financial operations, order creation, or any operation with real-world side effects.

Flow:

Client	Server	Redis / DB
-- POST /orders		
Idempotency-Key: uuid-1234		
	-- GET idempotency:uuid-1234 -->	
	<-- null (not found) -----	
	-- process order	
	-- SET idempotency:uuid-1234 -->	
	(result, TTL: 24h)	
<-- 201 Created (order) -----		
-- POST /orders (retry)		
Idempotency-Key: uuid-1234		
	-- GET idempotency:uuid-1234 -->	
	<-- cached result -----	
<-- 201 Created (same order) ---		

Rules: - Client generates the key (UUID v4). Server does not generate it. - Store result keyed by `{endpoint}:{idempotency_key}` to prevent cross-endpoint collisions. - TTL: 24 hours is standard. Align with your retry window. - Return the same HTTP status code and body on replay. - If the original request is still in flight, return 409 Conflict (not a duplicate, not ready yet). - Redis is the right store. Not Postgres — you need fast lock semantics and TTL.

Anti-Patterns

A-1: Breaking API changes in a patch release. “We just renamed the field — it’s a small change.” Renames are breaking changes. Version bump required. No exceptions for external consumers.

A-2: No versioning strategy until the first breaking change is needed. By then you have consumers in production and no clean path. Versioning must be designed before the first public release, not retrofitted.

A-3: GraphQL for internal service-to-service communication. You pay the overhead of client-driven queries between services you control. Use gRPC (typed, fast) or REST (simple). GraphQL’s value is consumer flexibility — you have no unknown consumers internally.

A-4: Offset pagination on high-volume, frequently-updated datasets. Job listings, product catalogs, feed data. Offset is a trap. Cursor from day one.

A-5: Idempotency keys as optional. “We’ll add it later.” Payment processing, order creation, and anything that charges money or allocates resources does not get a “later.” It ships with idempotency or it does not ship.

A-6: Sunset dates with no enforcement. Announcing a date and then not cutting off the endpoint teaches consumers to ignore your deprecation notices. If you set a date, honor it.

What to Ask AI

- “We have [describe consumers: external mobile app, third-party integrations, internal Go workers]. Should this be REST, gRPC, or GraphQL? What are the tradeoffs for our specific case?”
- “We need to make this breaking change to our API: [describe change, e.g., rename `user_id` to `userId` in the orders response]. What is the safest migration path for [external consumers / internal services]?”
- “Design the pagination strategy for [endpoint description: 100k job listings, updated frequently, sorted by `posted_at`]. When would offset break and why? Show me the cursor implementation for Symfony + Doctrine.”
- “Our Symfony API calls a Go worker service 500 times per second. Should this be REST or gRPC? What do I need to add to Symfony to support gRPC clients?”

- “We have a v1 API in production with 3 known external consumers. We need to ship a breaking change. Write the deprecation announcement headers and migration guide template.”
- “Review this idempotency key implementation [paste code]. What edge cases does it miss?”

Source: `09-api-design/02-lab-api.md`

Lab: API Design Decisions

Reference: Breaking vs Non-Breaking Changes

Change	Breaking?	Affects External?	Affects Internal?
Remove field from response	Yes	Yes	Yes
Rename field (<code>user_id</code> → <code>userId</code>)	Yes	Yes	Yes
Change field type (<code>int</code> → <code>string</code>)	Yes	Yes	Yes
Change HTTP status code	Yes	Yes	Yes
Change URL path	Yes	Yes	Yes
Remove endpoint	Yes	Yes	Yes
Make optional field required	Yes	Yes	Yes
Add new optional response field	No	No	No
Add new optional request param	No	No	No
Add new endpoint	No	No	No
Add new enum value	Depends	Only if consumer rejects unknown values	Depends
Change error message text	No	No	No
Change error code within same status	Yes	Yes	Yes

Deprecation Timeline Template

```
Version: v1
Deprecated: 2025-07-01
Sunset: 2025-12-31
Migration target: v2
Migration guide: https://docs.example.com/migrate/v1-to-v2

Timeline:
  T+0   (2025-07-01): v2 released. v1 marked deprecated.
                        Deprecation + Sunset headers added to all v1 responses.
                        Migration guide published.
                        All known consumers notified directly.
  T+30  (2025-08-01): Usage report. Follow up with consumers above 10% of v1 traffic.
  T+90  (2025-10-01): Second notice to consumers still on v1.
  T+150 (2025-11-28): Final notice. 30 days remaining.
  T+183 (2025-12-31): Hard cutoff. v1 returns 410 Gone.

Response headers on v1 from T+0:
  Deprecation: Tue, 01 Jul 2025 00:00:00 GMT
  Sunset: Wed, 31 Dec 2025 23:59:59 GMT
  Link: <https://api.example.com/v2/docs>; rel="successor-version"
```

Idempotency Key Flow

POST /orders (first attempt)
Headers: Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

Client	Symfony API	Redis
--POST /orders ----->		
Idempotency-Key: uuid		
	--GET orders:uuid ----->	
	<--null (miss)-----	
	[process order]	
	[write to Postgres]	
	--SET orders:uuid ----->	
	value: {201, body}	
	TTL: 86400s	
<--201 Created -----		

POST /orders (retry – network timeout, client unsure if it landed)
Headers: Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

Client	Symfony API	Redis
--POST /orders ----->		
Idempotency-Key: uuid		
	--GET orders:uuid ----->	
	<--{201, body} (hit)----	
<--201 Created (replay)----		
[same order, no dup]		

POST /orders (concurrent – first request still in flight)
Headers: Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

Client	Symfony API	Redis
--POST /orders ----->		
Idempotency-Key: uuid		
	--GET orders:uuid ----->	
	<--LOCKED (in flight)---	
<--409 Conflict -----		
[retry after delay]		

Exercise 1: API Contract Review

Proposal: “Rename `user_id` to `userId` in the orders response for consistency with our new frontend convention.”

Work through each question before reading the answer scaffold.

Question 1: Is this breaking? For whom?

Consider: - Which consumers call `GET /orders` or any endpoint returning order objects? - What happens to code that reads `response['user_id']` when the field becomes `userId`? - Does “consistency with the frontend” justify breaking server contract for all other consumers?

Answer scaffold: - Yes, this is a breaking change for any consumer that reads `user_id` by name. - External consumers (mobile apps, third-party integrations) break silently — they read null or throw a key error. - Internal consumers (other services, the Go worker if it reads order data) break depending on their deserialization. - The frontend convention is irrelevant to the API contract. Frontend can alias the field.

Question 2: Migration path if external consumers exist

Design the migration: - Can you ship both fields (`user_id` and `userId`) simultaneously during a transition window? - What version do you create for the renamed field? - How long is the transition window? - What headers do you add to the old field?

Answer scaffold: 1. Create v2 with `userId`. Keep v1 with `user_id` unchanged. 2. Do not ship the rename to v1. v1 contract is frozen. 3. Announce v2 with migration guide. Set sunset on v1 (minimum 6 months for external). 4. Optionally add both fields to v1 for a transition window: `user_id` (deprecated) + `userId` (new). Remove `user_id` only in v2. 5. Monitor v1 traffic. Cut off at sunset date.

Question 3: Migration path if only internal consumers exist

Design the migration: - Are internal consumers different enough to change the process? - What is the minimum sunset window for internal services? - Who needs to sign off?

Answer scaffold: 1. Still a breaking change — internal consumers have code that reads `user_id`. 2. Coordinate with each consuming team directly. Do not assume they will notice header changes. 3. Minimum 3 months. Shorter if all consuming teams confirm they have migrated. 4. Require explicit sign-off from each team before cutting over. 5. Option: deploy both field names, confirm all internal consumers updated, remove `user_id`. Faster than a full version bump for internal-only.

Question 4: What would you add to the ADR for this decision?

Draft the key sections:

- **Context:** Frontend wants camelCase field names. Orders API currently returns `user_id`.
 - **Decision:** Field rename treated as breaking change. v2 endpoint introduced with `userId`. v1 maintained until sunset.
 - **Rejected alternatives:** Rename in v1 directly (breaks external consumers). Ship both fields indefinitely (schema debt).
 - **Consequences:** v1 sunset window starts [date]. External consumers must migrate. Monitoring added to track v1 usage. Sunset date: [date + 6 months].
 - **Sunset criteria:** v1 traffic below 1% of total orders traffic, or all known consumers confirmed migrated.
-

Exercise 2: Versioning Decision

System: Job marketplace API. Currently no versioning. v1 consumers in production.

New consumers coming: - Employer dashboard (different data shape — needs aggregate stats, not raw listings) - Mobile app (bandwidth-sensitive — needs stripped-down responses) - Third-party integrations (strict contracts — they pin to specific response shapes)

Question 1: Versioning strategy choice with justification

Consider: - You have three consumers with genuinely different needs. - Third-party integrations require strict contracts. - Mobile needs smaller payloads. - Employer dashboard needs different data, not just less data.

Design your answer: - URL versioning (`/v1/` , `/v2/`) or header versioning? Why? - Do different consumers get different endpoints or different versions? - Where do you enforce versioning — gateway, Symfony routing, both?

Answer scaffold: - **URL versioning.** Three consumer types, all with different operational teams/organizations. URLs are debuggable in logs, browser, curl. Header versioning fails when a third-party integration sends a ticket saying “it’s broken” and you cannot see the version in the URL. - Introduce `/v2/` for employer dashboard (different data shape = new version, not a new endpoint option). - Mobile bandwidth: sparse fieldsets via `?fields=id,title,salary` added to v1 — this is not a breaking change. - Third-party integrations: lock to `/v1/`. They freeze there. Future changes go to `/v2/` , `/v3/` . - Enforce via Symfony route prefix. Add a middleware that logs version usage per API key.

Question 2: Which endpoints need idempotency keys and why

Go through the likely endpoints:

Endpoint	Idempotency key needed?	Why
POST /jobs (employer posts a job)	Yes	Duplicate job listings are real-world harm and billing events
POST /applications (candidate applies)	Yes	Duplicate applications corrupt employer view, may trigger emails
POST /employers (account creation)	Yes	Duplicate accounts on retry = support burden
GET /jobs	No	Read-only, idempotent by definition
PUT /jobs/{id}	No	PUT is idempotent by HTTP semantics
DELETE /jobs/{id}	No	DELETE is idempotent by HTTP semantics
POST /payments (if billing exists)	Yes	Non-negotiable

Implementation: Idempotency-Key header on all POST endpoints listed above. Store in Redis keyed by {endpoint_path}:{idempotency_key}. TTL 24h.

Question 3: Pagination strategy for job listings (100k jobs, updated frequently)

Work through the choice:

- 100k rows. Offset at page 50 (OFFSET 1000) requires scanning 1000 rows to discard.
- Jobs are posted and expired continuously. Page 2 you loaded 10 minutes ago is now different.
- Mobile clients need to resume a scroll session after backgrounding the app.

Design: - Cursor or offset? What cursor field? - What is the response envelope? - How does the mobile app handle resuming a session?

Answer scaffold: - **Cursor-based pagination.** Encode {id, posted_at} as base64 JSON cursor. - Sort by posted_at DESC, id DESC (stable secondary sort on id for same-timestamp rows). - Query: WHERE (posted_at, id) < (:cursor_posted_at, :cursor_id) ORDER BY posted_at DESC, id DESC LIMIT 20. - Response envelope: json { "data": [...], "pagination": { "next_cursor": "eyJwb3N0ZWRFYXQiOiIyMDI1LTA1LTlwIiwiaWQjEjE3M30=", "has_more": true } } - Mobile resume: client stores next_cursor locally. On app resume, sends cursor in next request. Gets correct next page regardless of how many jobs were posted while app was backgrounded. - Offset is acceptable only for the employer admin panel listing their own jobs (small bounded set, rarely updated).

Phase — 10-security-boundaries

Directory: 10-security-boundaries/

Source: 10-security-boundaries/01-security-architecture.md

Security Architecture

Architects design where security checks live. Security engineers implement them. If the architect misplaces the checks, no amount of careful implementation fixes it.

Trust Boundaries

Every component boundary is a potential trust boundary. A trust boundary is any point where data or requests cross from one execution context to another — browser to API, API to worker, service to database.

The core question at each boundary: what do you verify, and what do you assume?

External boundaries — browser, mobile client, third-party integrations. Assume nothing. Verify everything: authentication, input format, size, content type.

Internal boundaries — Symfony API to Go worker, API to Postgres, API to Redis. The mistake: trusting internal traffic implicitly because “it’s behind the firewall.”

Why that fails: internal compromise is real. A misconfigured Redis, an exploitable dependency in the Go worker, a compromised container — once an attacker is inside the network, implicit trust becomes lateral movement. If Symfony trusts any message that arrives on the queue without verification, a compromised producer writes arbitrary jobs.

What to do instead: - Service identity: each internal service has a verifiable identity (mTLS certificates, service account tokens) - Message integrity: queue messages are signed or carry a short-lived token scoped to that operation - Least privilege: Go worker connects to Postgres with credentials that only allow the operations it needs — not `SUPERUSER`, not the same credentials as the API

Boundary inventory: when reviewing a design, list every boundary explicitly. For each one: what does the consumer send? What does the receiver verify before acting?

Auth/Authz Architecture

Authentication and authorization are different problems. Architects who conflate them create security gaps.

Authentication — who are you? Answers the question: is this request from who it claims to be?

JWT tokens: self-contained, stateless. The token carries claims signed by a known key. Validation requires the public key and checks: signature valid, not expired, issuer correct, audience correct. If any check fails, reject before doing anything else.

OAuth2: delegates authentication to an identity provider. The API receives an access token — it must still validate that token (signature, expiry, scope). OAuth2 does not remove the need to validate; it moves where the identity decision is made.

Where authentication validation lives:

Approach	Where Validation Happens	Tradeoff
Centralized (API gateway)	Gateway validates JWT, passes claims downstream	Consistent enforcement, single point of failure, services can trust gateway-injected headers
Decentralized (each service)	Every service validates the token independently	Resilient, no single choke point, each service must maintain validation logic and key rotation
Hybrid	Gateway validates, services re-verify on sensitive operations	Balanced — services don’t blindly trust headers for write operations

The typical choice for a Symfony API + Go worker system: **gateway handles initial validation and injects a trusted header** (e.g., `X-User-ID`, `X-User-Roles`). Symfony trusts those headers only from the gateway (verify the request came from the gateway, not from the open internet). The Go worker, triggered by a queue message, carries its own signed payload — it does not rely on headers.

Authorization — what can you do? Answers the question: does this authenticated identity have permission to perform this operation on this resource?

RBAC (Role-Based Access Control): assign roles to users, permissions to roles. Simple to implement, simple to audit, hard to granulate. `admin`, `editor`, `viewer` works until you need “editor of their own content only.”

ABAC (Attribute-Based Access Control): policies evaluate attributes of the subject, resource, and environment.

`user.department == resource.department AND time.now < resource.embargo_date`. Flexible, powerful, significantly more complex to reason about and audit.

Where authorization lives: in the service that owns the resource. Always.

Who checks what	Correct
Gateway validates JWT signature	Yes
Gateway checks token expiry	Yes
Gateway decides if user can access <code>/orders/123</code>	No — gateway doesn't own orders
Symfony checks if authenticated user owns order 123	Yes
Go worker checks if the job was issued for a valid user	Yes

The rule: **gateway handles authentication, services handle authorization**. Mixing them creates gaps — either the gateway is making decisions it doesn't have context for, or services are duplicating authentication logic they can't do correctly.

Secrets Management

Secrets are: database passwords, API keys, JWT signing keys, OAuth client secrets, encryption keys.

Where secrets must not live: - Source code (even in private repos — git history is forever) - `.env` files committed to version control - Plain environment variables set in deployment configs visible in CI logs - Docker image layers

Where secrets must live: a secret manager — AWS Secrets Manager, HashiCorp Vault, GCP Secret Manager. Secrets are injected at runtime: the application fetches them from the secret manager on startup, or the deployment platform injects them as environment variables from the secret manager (not from plain config).

Rotation architecture: the system must handle secret rotation without a restart.

Design requirement: applications that require a restart to pick up a new DB password or JWT signing key create downtime windows during rotation. Architect around this: - Fetch secrets with a TTL and re-fetch periodically (every 5–15 minutes) - JWT signing: support multiple valid signing keys simultaneously (current + previous) during rotation window - Database passwords: use connection pooling middleware (PgBouncer) where the secret rotation is applied at the pool level, not per-connection

Threat Modeling Basics (STRIDE Checklist)

You don't run a formal STRIDE exercise on every feature. You use STRIDE as a checklist when reviewing a design, evaluating a PR, or deciding where a new component fits.

For each component or feature, ask:

Letter	Threat	Question to ask
S	Spoofing	Can an attacker pretend to be a legitimate user, service, or component?
T	Tampering	Can data be modified in transit or at rest without detection?
R	Repudiation	If something goes wrong, is there an audit trail? Can a user deny an action they took?
I	Information Disclosure	Can an attacker read data they shouldn't? Logs, error messages, API responses?
D	Denial of Service	Can an attacker make this component unavailable? Is there rate limiting?
E	Elevation of Privilege	Can an attacker gain more access than they were granted?

Apply this checklist at: new API endpoints, new queue consumers, new integrations with third-party services, new data export features, any component that touches PII or financial data.

Audit Log as Architecture

Audit logging is not a feature added later. It is architectural. The schema and storage must be designed upfront.

Rule: any action that changes state or accesses sensitive data needs a log entry with: - Who (authenticated user ID, service identity) - What (action taken, resource affected, resource ID) - When (UTC timestamp, millisecond precision) - From where (IP address, service name, request ID) - Result (success, failure, error code)

The schema question: structured log (JSON in a log aggregator) or a dedicated audit log table? For compliance (GDPR, PCI, SOC2), the answer is almost always a dedicated table — log aggregators can be cleared or rotated, and you need to prove the log hasn’t been tampered with. Immutable append-only storage.

What does not need an audit log: read operations on non-sensitive data. `GET /products` does not need an audit entry. `GET /users/123/payment-methods` does.

Decision Table: Where Security Checks Live

Component	Consumer Type	Data Sensitivity	Authentication Check	Authorization Check
API Gateway	External browser/mobile	Any	Gateway validates JWT	Not here — pass to service
Symfony API endpoint	Via gateway (trusted)	Low (public catalog)	Trust gateway header	None needed
Symfony API endpoint	Via gateway (trusted)	High (orders, PII)	Trust gateway header	Service checks ownership
Symfony admin endpoint	Internal admin users	High	Gateway validates admin JWT	Service checks admin role
Go worker	Queue message	Medium	Verify signed job payload	Worker checks job belongs to valid user
Postgres	Go worker, Symfony	High	DB credentials (least privilege)	Schema-level grants per service
Redis	Symfony API	Medium (session/cache)	Redis AUTH token	Application-level key namespacing

Anti-patterns

Anti-pattern 1: Authorization logic in the API gateway `if role == "admin" { allow }` in the gateway. The gateway does not have resource context. It doesn’t know if admin is allowed to access *this specific* resource. Authorization belongs in the service that owns the resource.

Anti-pattern 2: JWT tokens without expiry or rotation strategy Short-lived tokens (15 minutes) plus refresh tokens is the baseline. No expiry means a stolen token is valid forever. No rotation strategy means a compromised signing key requires emergency key replacement with downtime.

Anti-pattern 3: Secrets in .env files committed to git Even “accidentally” committed secrets are compromised. Assume git history is public. Rotate immediately, audit access logs, fix the architecture.

Anti-pattern 4: No audit log until compliance forces it Retrofitting audit logging onto an existing system is expensive — you have to reconstruct schemas, backfill historical gaps, prove to auditors that the log is complete. Design it in from the start.

Anti-pattern 5: “Internal network is trusted” Internal compromise is the dominant attack vector in post-breach analyses. Without service identity (mTLS or equivalent) and per-service least-privilege credentials, internal lateral movement is trivial.

Network-Level Security Posture

Architecture decisions about network topology are security decisions.

Ingress: only the API gateway should be publicly accessible. Symfony, Go worker, Postgres, Redis — none of these should accept connections from the public internet. They live in a private network (VPC, private subnet) and are reachable only from within it.

Common mistake: a developer exposes Postgres on port 5432 to 0.0.0.0/0 for convenience during early development, then forgets to restrict it before production. The check: “what is the security group / firewall rule on this port? Who is allowed to connect?” belongs in the architecture review, not the ops runbook.

Egress: outbound traffic should also be restricted. The Go worker does not need to make arbitrary HTTP requests. If its only external call is to object storage, restrict egress to that endpoint. Egress restriction limits the blast radius if the worker is compromised — it cannot beacon out to an attacker-controlled server or exfiltrate data to an arbitrary endpoint.

Service mesh and mTLS: in a multi-service system, mTLS means every service-to-service connection is mutually authenticated — both sides present a certificate, both sides verify the other. This eliminates the “trust by network location” assumption. The cost is certificate management (issuance, rotation, distribution). At small scale, a service mesh (Istio, Linkerd) manages this. At very small scale (two services), manually issued certificates with a short TTL work. The decision: do you need mTLS? If the answer is “we can’t enumerate who is allowed to call each service,” the answer is yes.

Rate limiting and throttling: rate limiting is a DoS mitigation and a cost control, but it is also a security control. Without rate limiting, a brute-force attack on a login endpoint, a credential-stuffing attack, or an enumeration attack on user IDs can run indefinitely. Rate limiting belongs at the gateway for external traffic and per-service for internal traffic on write-heavy endpoints.

Input Validation as a Security Boundary

Input validation is not just a data quality concern — it is a security boundary. Every point where external data enters the system is an attack surface.

Where validation lives: - At the boundary where data enters the system (the API endpoint, not deep in the service layer) - Validated once, as early as possible — not repeatedly in different layers with inconsistent rules - On both structure (is this valid JSON? does the field exist?) and semantics (is this user ID within a valid range? is this file type allowed?)

What to validate at each entry point: - Data type and format (not just “is this a string” but “is this a valid UUID / ISO date / enum value”) - Size and length limits (unbounded input = DoS vector) - Allowed values (allowlist, not blocklist — ["csv", "pdf"] not “anything except .../etc/passwd”) - Ownership (before acting on a resource ID, verify the caller owns that resource)

The mistake: validating at the service layer but not at the API layer. A malformed payload that gets deep into the system before rejection can trigger unexpected behavior, expose stack traces in errors, or create partial state.

The Go worker receives job payloads from a queue. Even though the queue is internal, validate the payload schema at the start of the worker handler. Treat queue messages as untrusted input — because the producer might be compromised, misconfigured, or simply have a bug.

Šta da pitaš AI

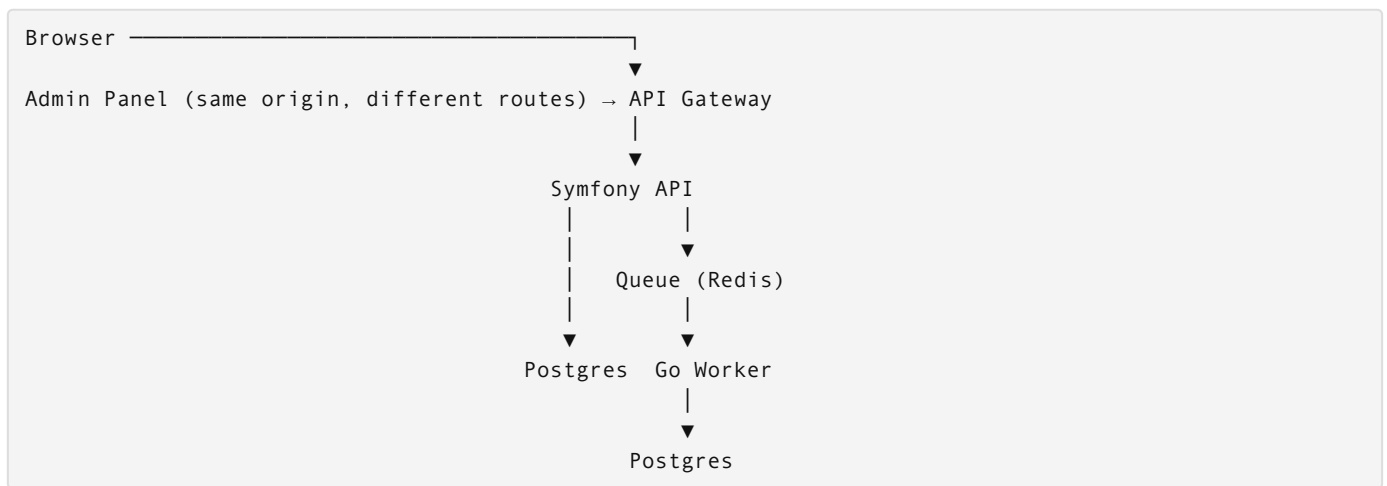
- “Draw the trust boundaries in this architecture: [describe components]. Where should authentication checks live? Where should authorization checks live?”
 - “We store [describe sensitive data]. What audit log events are required? What schema would you use for an immutable audit log in Postgres?”
 - “We use JWT for auth. What is the token rotation strategy if a signing key is compromised? Walk me through the rotation sequence without downtime.”
 - “Here is our secrets management approach: [describe]. What are the risks and what should we change?”
 - “Apply the STRIDE checklist to this feature: [describe]. For any risk you identify, what is the architectural mitigation?”
 - “Where in this service does input validation happen? What inputs are not validated at the entry point and why is that a risk?”
-

Source: 10-security-boundaries/02-lab-security.md

Lab: Security Boundaries

Exercise 1: Trust Boundary Mapping

Architecture:



Consumers: - Regular users: browser clients authenticating with JWT via OAuth2 - Admin users: internal admin panel, same Symfony API, separate route prefix `/admin/` - Go worker: triggered by jobs placed on a Redis queue by the Symfony API

Questions

1. Where does authentication happen for each consumer type?

Map each consumer to the point where their identity is verified and what token/credential is used.

Consumer	Entry Point	Token/Credential Type	Where Validated
Browser user	API Gateway	?	?
Admin panel user	API Gateway	?	?
Go worker (queue)	Redis queue message	?	?

What is different about the Go worker's authentication compared to browser users? It never goes through the gateway. What does that mean for how you verify it?

2. Where does authorization happen?

For each of these operations, identify where the authorization check should live and why:

- Regular user reads their own order: `/api/orders/123`
- Regular user reads another user's order: `/api/orders/456` (they don't own it)
- Admin user exports all orders: `/admin/orders/export`
- Go worker processes a job to generate a PDF for user 789

For each: is the check at the gateway, in Symfony, in the Go worker, or somewhere else? What information does the check need that the gateway does not have?

3. Which internal boundaries need additional trust checks?

List each internal hop and decide: is the trust implicit, or does it need an explicit check?

Boundary	Implicit Trust OK?	What check is needed if not?
API Gateway → Symfony API	?	?
Symfony API → Postgres	?	?
Symfony API → Redis (queue write)	?	?
Redis queue → Go Worker (queue read)	?	?
Go Worker → Postgres	?	?

Hint: "implicit trust OK" has a high bar. What would an attacker need to do to exploit implicit trust at each boundary?

4. The Go worker's Postgres credentials

The Go worker connects to the same Postgres instance as the Symfony API.

Questions to answer: - Should the Go worker use the same database credentials as the Symfony API? - What tables does the Go worker actually need access to? (Assume it processes order export jobs: reads orders, writes to an exports table, updates a job_status table.) - What Postgres role would you define for the Go worker? What grants would it have? - If the Go worker's credentials are compromised, what is the blast radius with least-privilege credentials vs shared credentials?

Reference: Trust Boundary Diagram Template

Use this text format when diagramming trust boundaries in design reviews:

```
[Component A] ==EXTERNAL BOUNDARY==> [Component B]
  Verify: JWT signature, expiry, audience
  Pass: user_id, roles (as trusted header)

[Component B] —internal—> [Component C]
  Verify: signed message payload (HMAC or short-lived token)
  Pass: job_id, user_id, operation_type

[Component C] —internal—> [Database]
  Verify: credentials (least-privilege role)
  Pass: parameterized queries only
```

For each boundary, document: what is verified, what is passed downstream, and what happens on verification failure (reject vs degrade vs alert).

Exercise 2: Threat Model Lite

Feature: Users can export their order history as a CSV download.

Flow: 1. Authenticated user clicks “Export my orders” 2. Symfony API places a job on the Redis queue: `{job_type: export_orders, user_id: 123, requested_at: ...}` 3. Go worker picks up the job, queries Postgres for all orders where `user_id = 123` 4. Go worker writes a CSV file to object storage (S3-compatible) 5. Go worker calls back to Symfony API with a pre-signed download URL 6. Symfony API stores the URL, sends user a download link via email 7. User clicks the link, downloads the CSV

Apply the STRIDE checklist. For each letter: is there a risk in this feature? If yes, what is the architectural mitigation?

STRIDE Checklist Table

Fill in: Risk Present (yes/no), Where in the flow, Mitigation.

Threat	Question	Risk Present?	Where in Flow	Architectural Mitigation
Spoofing	Can someone claim to be a user they are not?	?	?	?
Tampering	Can the job payload or CSV data be modified?	?	?	?
Repudiation	If a user denies requesting the export, is there proof?	?	?	?
Information Disclosure	Can a user download another user’s export?	?	?	?
Denial of Service	Can an attacker exhaust resources via export requests?	?	?	?
Elevation of Privilege	Can an attacker get data beyond their own orders?	?	?	?

Worked example for S (Spoofing): Risk: yes. If the queue message carries only `user_id: 123` and the Go worker trusts it without verification, a compromised producer could forge any `user_id`. Mitigation: the queue job is signed (HMAC with a shared secret, or a short-lived token issued by Symfony at job creation time). The Go worker verifies the signature before processing.

Work through the remaining five letters using the same format.

Audit Log Schema Template

For the order export feature, every audit-worthy event needs a log entry. Use this schema as a starting point:


```

CREATE TABLE audit_log (
    id                BIGSERIAL PRIMARY KEY,
    event_time        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    actor_id          TEXT NOT NULL,           -- user_id, service name, or 'anonymous'
    actor_type         TEXT NOT NULL,           -- 'user', 'service', 'admin'
    action            TEXT NOT NULL,           -- 'order.export.requested', 'order.export.completed'
    resource_type      TEXT NOT NULL,           -- 'order_export', 'order'
    resource_id        TEXT,                   -- specific record ID if applicable
    source_ip          INET,                   -- originating IP
    service_name        TEXT NOT NULL,          -- 'symfony-api', 'go-worker'
    request_id         TEXT,                   -- correlation ID for distributed tracing
    result             TEXT NOT NULL,           -- 'success', 'failure', 'denied'
    result_detail       JSONB                   -- error code, reason, additional context
);

-- Index for compliance queries: "all exports for user X in the last 90 days"
CREATE INDEX idx_audit_actor_action ON audit_log (actor_id, action, event_time DESC);

-- Index for incident response: "all events from IP X in the last 24 hours"
CREATE INDEX idx_audit_source_ip ON audit_log (source_ip, event_time DESC);

```

Events to log for the export feature:

Event	actor_id	action	result_detail
User requests export	user_id	order.export.requested	{count_requested: N}
Worker picks up job	go-worker	order.export.job.started	{job_id, user_id}
Worker queries orders	go-worker	order.export.data.read	{user_id, row_count}
Export file written	go-worker	order.export.file.created	{storage_path, size_bytes}
Download URL accessed	user_id	order.export.downloaded	{source_ip, user_agent}
Unauthorized access attempt	attacker / user_id	order.export.access.denied	{attempted_resource, reason}

Storage rule: the audit log table must not be truncated or deleted by application code. Application database role does not have `DELETE` or `TRUNCATE` on this table. Retention and archiving are operational concerns, handled outside the application.

Exercise 1: Reference Answers

Use these after completing your own answers. If your answer differs, ask why — not all differences are wrong.

Authentication (Q1):

Consumer	Entry Point	Token/Credential Type	Where Validated
Browser user	API Gateway	JWT (access token, 15-min expiry)	Gateway validates signature, expiry, audience
Admin panel user	API Gateway	JWT (separate audience claim <code>aud: admin</code>)	Gateway validates; Symfony also checks admin role claim
Go worker (queue)	Redis queue message	HMAC-signed payload or short-lived scoped token	Go worker verifies signature before processing

The Go worker never passes through the gateway. It must self-validate the job payload. If it trusts the queue implicitly, any process that can write to Redis can inject arbitrary jobs.

Authorization (Q2):

Operation	Authorization Location	Reason
User reads own order <code>/api/orders/123</code>	Symfony service layer	Gateway doesn't know who owns order 123
User reads another's order	Symfony — deny, 403	Same: ownership is a data-level check
Admin exports all orders	Symfony — checks admin role	The role claim is present, but Symfony validates the resource scope
Go worker processes PDF job for user 789	Go worker — verifies job user_id matches signed payload	Worker must not trust that any job in the queue is legitimate

Internal boundaries (Q3):

Boundary	Implicit Trust OK?	What check is needed
API Gateway → Symfony API	Only if Symfony is not reachable directly from the internet	Symfony must reject requests not from the gateway (verify source IP or shared secret header)
Symfony API → Postgres	No	Dedicated role with table-level grants; parameterized queries only
Symfony API → Redis (queue write)	No	Redis AUTH; job payloads signed by Symfony
Redis queue → Go Worker	No	Worker verifies payload signature before acting
Go Worker → Postgres	No	Separate role from Symfony's role; minimum grants

Go worker Postgres credentials (Q4): The Go worker should have its own Postgres role distinct from Symfony's role. If the worker only needs to read `orders`, write to `exports`, and update `job_status`, those are the only grants it gets. If the worker's credentials are compromised, blast radius is limited to those three tables. With shared credentials, the attacker has everything the API has — which typically includes write access to users, sessions, and payment data.

Šta da pitaš AI (Lab)

- “Here is the trust boundary diagram for our export feature: [describe]. What trust checks are missing? What would an attacker exploit first?”
- “Apply STRIDE to this queue-based worker architecture. For each threat, what is the specific risk given that the worker never goes through the API gateway?”
- “We need a Postgres role for our Go worker. It reads from `orders` and `order_items`, writes to `exports`, updates `job_status`. Write the `CREATE ROLE` and `GRANT` statements.”
- “Our audit log is in the same Postgres database as our application. What are the risks of that, and how would you architect the audit log for a high-compliance environment?”

Phase — 11-observability

Directory: 11-observability/

Source: 11-observability/01-observability-design.md

Observability Design

Observability is not “we have logs.” It is: can you answer “what is broken and why” without SSH-ing into a server? If the answer is no, you do not have observability — you have noise.

The Three Pillars

Pillar	What it answers	Strengths	Weaknesses
Logs	What happened in this request	Debugging specific requests, audit trail, error detail	Useless aggregated at scale, no performance trends, expensive to store unstructured
Metrics	How is the system behaving over time	Dashboards, alerting, SLO tracking, cheap to store	Cannot debug why one specific request failed
Traces	How did this request move through the system	Latency attribution, dependency mapping, cross-service correlation	Expensive to collect + store, requires instrumentation in every service

The rule: start with metrics (cheapest, most useful for SLOs), add structured logs with request ID correlation, add traces only when you have multiple services and cannot find where latency originates.

Do not add distributed tracing on day 1. Traces without multiple services answer questions you do not yet have.

The Four Golden Signals

These four signals tell you **if** something is wrong. Everything else tells you why.

Signal	What it measures	Example metric	Alert threshold
Latency	How long requests take — p50, p95, p99 (not average)	http_request_duration_seconds{p99}	p99 > SLO budget
Traffic	How many requests per unit time	http_requests_total rate	Sudden drop = silent failure
Errors	How often requests are failing	http_requests_total{status=~"5.."} rate	> 1% of total requests
Saturation	How full is the most constrained resource	Queue depth, connection pool utilization, memory %	Growing trend, not point-in-time

Why not average latency: averages hide tail latency. A p99 of 2000ms on an API that averages 50ms means 1 in 100 users gets a broken experience. p99 is user-facing. Average is not.

Applied to the spine — API Gateway → Symfony API → Go worker → Postgres + Redis:

Component	Latency	Traffic	Errors	Saturation
API Gateway	Request duration p50/p99	Requests/sec per route	4xx/5xx rate	Concurrent connections
Symfony API	Endpoint duration p99	Requests/sec per endpoint	Exception rate, 5xx rate	PHP-FPM pool active workers
Go worker	Job processing duration p99	Jobs consumed/sec	Job failure/retry rate	Queue depth (unconsumed jobs)
Postgres	Query duration p99	Queries/sec	Error rate, deadlock count	Connection pool active/waiting
Redis	Command duration p99	Commands/sec	Error rate, eviction rate	Memory used / max memory

Structured Logging Design

Logs must be queryable. Plain text logs are useless at scale — you cannot aggregate, filter, or correlate them across services. Structured logging means JSON with fixed fields.

Minimum required schema:

```
{
  "timestamp": "2026-05-25T10:32:01.123Z",
  "level": "error",
  "service": "symfony-api",
  "request_id": "req_01HXYZ...",
  "user_id": "usr_9182",
  "endpoint": "POST /jobs/{id}/apply",
  "duration_ms": 234,
  "status_code": 500,
  "message": "Database connection timeout",
  "context": {}
}
```

Field rules:

Field	Required	Notes
timestamp	Always	ISO 8601, UTC. Not local time.
level	Always	debug, info, warning, error, critical
service	Always	Fixed string per service. Enables cross-service filtering.
request_id	Always	The spine of cross-service correlation. See propagation below.
user_id	When authenticated	Enables “show me everything this user experienced”
duration_ms	On request completion	Enables latency percentile queries in log tooling
status_code	HTTP services	Enables error rate queries in logs as fallback
message	Always	Human-readable. Not structured data — that goes in context.
context	When relevant	Machine-readable key-value pairs for filtering

What not to put in logs: PII beyond user ID (no email, no name), passwords, tokens, credit card numbers. Log the ID; look up the detail when needed.

Request ID Propagation

This is architectural, not implementation. If request_id is not propagated across service boundaries, you cannot reconstruct a distributed request. You have isolated logs, not correlated logs.

Propagation path for this spine:

```
API Gateway
  → generates request_id (UUID or ULID) if none present
  → injects as X-Request-ID header
  → logs with request_id

Symfony API
  → reads X-Request-ID from incoming request
  → attaches to all log statements (via Monolog processor)
  → passes X-Request-ID when calling internal APIs
  → writes request_id to job payload when enqueueing

Go worker
  → reads request_id from job payload
  → attaches to all log statements
  → passes as header if calling downstream services

Postgres / Redis
  → query logs tagged with request_id via application-layer logging (not native DB logs)
```

The architectural decision: request_id must be in the job message schema from day 1. Adding it later means modifying all job producers, consumers, and message schemas under production load.

Alerting Design

Alert on symptoms, not causes.

Type	Example	Problem
Symptom alert	API error rate > 1%	User-facing. Something is broken right now.
Cause alert	Postgres CPU > 80%	Infrastructure signal. May not yet affect users.

Start with symptom alerts. Add cause alerts only when you have learned which causes predictably lead to which symptoms — and even then, cause alerts are for early warning, not paging.

What to page on: - User-facing error rate exceeds threshold - API p99 latency exceeds SLO budget - Queue depth growing with no consumption (workers are down or stuck)

What to notify (not page) on: - Postgres connection pool near saturation (early warning) - Redis memory > 80% (before eviction starts) - Deploy completed (context for correlating metric changes)

SLO-Based Alerting: Burn Rate

Point-in-time alerting (“error rate > 1% for 5 minutes”) pages on noise. A 5-minute spike in errors that recovers is not an incident. An error rate that has been 1.5% for 3 hours has consumed a significant fraction of your monthly error budget — that is an incident, even if the rate looks “low.”

Error budget burn rate:

monthly error budget = (1 - SLO) × total requests for the month
burn rate = actual error rate / (1 - SLO)

e.g. SLO = 99.9%, error budget = 0.1%
if current error rate = 0.2%, burn rate = 2x
at 2x burn rate, 30-day budget is consumed in 15 days

Burn rate thresholds (Google SRE model):

Burn rate	Action	Window
> 14.4x	Page immediately	1 hour window
> 6x	Page (business hours)	6 hour window
> 3x	Create ticket	3 day window
> 1x	Monitor	Weekly review

At 14.4x burn rate, the entire monthly budget is consumed in 2 hours. That is always page-worthy.

Decision Table: Which Pillars to Implement First

System complexity	Team size	SLO requirement	Start with
Single service	Small (1-5)	None / informal	Structured logs + 4 golden signal metrics
2-3 services	Small-medium	Informal SLO	Metrics + structured logs + request_id propagation
3+ services	Any	Formal SLO with budget	Metrics + logs + burn rate alerting, then traces
3+ services	Medium-large	Strict SLO (99.9%+)	All three pillars, traces from week 1
Microservices (10+)	Large	Strict SLO	Traces are mandatory — latency is otherwise unattributable

For this spine (API + Go worker + Postgres + Redis): metrics + structured logs + request_id propagation. Add traces when you have a latency problem you cannot locate with logs alone.

Observability Cost

Pillar	Storage cost	Instrumentation cost	Query cost
Metrics	Low — aggregated time series	Low — counters and histograms	Low — pre-aggregated
Logs	Medium — grows with request volume	Medium — schema design + propagation	Medium — depends on indexing
Traces	High — one trace per request, cross-service	High — all services must be instrumented	High — sampling required at scale

Design for cost from day 1: - Do not log at DEBUG level in production. DEBUG is for development. Production logs should be INFO and above. - Sample traces at the gateway. 100% sampling at 1000 req/s is 86M traces/day. Sample at 10% or sample only slow/error requests. - Set log retention by tier: error logs 90 days, info logs 30 days, debug logs 3 days maximum.

Anti-Patterns

- **Logging everything at DEBUG level in production:** costs money, generates noise, makes signal impossible to find. Logs must be filtered before they are useful — do not create the filtering problem yourself.
- **No request ID propagation:** the most expensive oversight to fix retroactively. You have logs per service but cannot join them into a request story. Every cross-service incident becomes manual detective work.
- **Alerting on infrastructure metrics only:** Postgres CPU at 80% does not tell you whether users are experiencing failures. You can have a saturated database and happy users (slow, but succeeding). Always have a user-facing error rate alert.
- **Adding distributed tracing before 2+ services:** single-service traces add overhead and answer no questions you could not answer with logs. The value of tracing is attribution across service boundaries.
- **No SLO → no alerting discipline:** without an error budget, every alert is a judgment call. Budget burn rate turns alerting into math.
- **Alerting on every error:** individual errors are noise. Rate over window is signal.

Šta da pitaš AI

- “We have [describe system: components, request flow, expected traffic]. What are the four golden signals for each component? What specific metrics would we collect, and what would we alert on?”
- “Design the structured log schema for [service type, e.g. async job processor]. What fields must be present for cross-service correlation? What should never appear in logs?”
- “Our SLO is [X% availability over 30 days]. What burn rate alerting thresholds make sense? At what burn rate do we page vs. just notify? Show the math.”
- “We have [N] services. We have metrics and structured logs but no traces. Our p99 latency SLO is violated but we cannot find the cause. What is the minimum tracing setup to locate the latency? What do we instrument first?”
- “Our observability costs are growing linearly with traffic. We are logging at DEBUG in production and sampling traces at 100%. What is the prioritized list of changes to cut cost without losing diagnostic ability?”

Source: `11-observability/02-lab-observability.md`

Lab: Observability Design

Two exercises. Design before implementation. Know what you are building before you build it.

Reference: Four Golden Signals per Component

Component	Latency	Traffic	Errors	Saturation
API Gateway	Request duration p50 / p99	Requests/sec (total + per route)	4xx rate, 5xx rate	Concurrent open connections
Symfony API	Endpoint duration p99 per route	Requests/sec per endpoint	Exception rate, HTTP 5xx/s	PHP-FPM pool: active / max workers
Go worker	Job processing duration p99	Jobs consumed/sec	Job failure count, retry count	Queue depth (unconsumed message count)
Postgres	Query duration p99 (read / write separately)	Queries/sec	Error count, deadlock/s, rollback/s	Connections: active / pool max, replication lag
Redis	Command duration p99	Commands/sec	Error count, eviction count/s	Memory used / maxmemory, hit rate

Reference: Structured Log Schema

```
{
  "timestamp": "2026-05-25T10:32:01.123Z",
  "level": "error",
  "service": "symfony-api",
  "request_id": "req_01HXYZ4K2M3N5P6Q7R8S",
  "user_id": "usr_9182",
  "endpoint": "POST /jobs/{id}/apply",
  "method": "POST",
  "status_code": 500,
  "duration_ms": 234,
  "message": "Database connection timeout on job application write",
  "context": {
    "job_id": "job_4421",
    "db_host": "postgres-primary",
    "query_type": "INSERT"
  }
}
```

Field presence by service:

Field	API Gateway	Symfony API	Go worker
timestamp	Required	Required	Required
level	Required	Required	Required
service	Required	Required	Required
request_id	Generated here	Propagated from header	Propagated from job payload
user_id	Optional (if JWT parsed at gateway)	Required when authenticated	Required (from job payload)
endpoint	Required	Required	N/A — use job_type
job_type	N/A	N/A	Required
duration_ms	Required (total)	Required (per request)	Required (per job)
status_code	Required	Required	N/A — use job_status
job_status	N/A	N/A	success, failed, retrying

Reference: Alert Design

Alert	Signal type	Threshold	Action	Why
API error rate high	Symptom	5xx rate > 1% over 5 min	Page	User-facing. Every 5xx is a broken user request.
API p99 latency breach	Symptom	p99 > [SLO budget]	Page	Users experiencing slow responses at scale
Worker queue growing	Symptom	Queue depth > 1000 and not decreasing	Page	Workers are down, stuck, or overwhelmed — jobs not processing
Job failure rate high	Symptom	Job failure rate > 5% over 10 min	Page	Downstream dependency failing or data quality issue
PHP-FPM pool saturated	Cause	Active workers > 90% of pool max	Notify	Early warning before Symfony starts queuing or rejecting
Redis evictions started	Cause	evicted_keys counter > 0	Notify	Cache is full — hot data being evicted, hit rate will fall
Postgres connections near limit	Cause	Active connections > 80% of pool max	Notify	Connection exhaustion means next spike fails

Exercise 1: Observability Design

System: API Gateway → Symfony API → Go worker → Postgres + Redis

Work through the four questions in order. Each answer constrains the next.

1. Metrics per Component (Four Golden Signals)

Using the reference table above, extend it with: - The specific metric name you would use (Prometheus naming convention: `service_unit_operation_suffix`) - The label dimensions you would add (e.g. `{route="/jobs/{id}", method="POST", status="5xx"}`)

Required output format:

Component	Signal	Metric name	Labels	Why this label
Symfony API	Errors	http_requests_total	{route, method, status_class}	Route separation reveals which endpoint is broken
Go worker	Saturation	worker_queue_depth	{queue_name}	Multiple queues may have different failure modes
...	...	...	...	...

Constraints to answer: - Which Symfony API metrics must be per-route (not just aggregate)? Why? - What is the minimum Go worker metric that tells you workers are not processing? - What Postgres metric tells you that the application, not the database, is the bottleneck?

2. Structured Log Schema

Using the reference schema, design the complete schema for:

a) Symfony API — HTTP request log (emitted at request completion)

Required fields: identify which five fields are mandatory for cross-service diagnosis if you had nothing else.

b) Go worker — job completion log (emitted when a job finishes, success or failure)

Note: Go worker has no HTTP context. What replaces `endpoint`, `status_code`, `method`? How does `request_id` arrive in the worker?

c) The propagation contract:

Complete this sentence for each boundary: - “API Gateway passes `request_id` to Symfony API via: ” - “**Symfony API passes requestid to Go worker via:** ” - “Go worker passes `request_id` to its own logs via: ”

If `request_id` is not in the **job message payload schema from day 1**, what happens when you try to add it later?

3. First Two Alerts

Choose the two alerts you would implement first from the reference alert table. For each:

Decision	Answer
Alert name	
Signal type (symptom / cause)	
Why this one before the others	
What observability is required for this alert to exist	
What this alert cannot tell you	

Forcing constraint: one of your two alerts must be a symptom alert. If both are cause alerts, you have no user-facing signal. Explain what gap that leaves.

4. Request ID: Generation and Propagation

Answer in architecture terms, not implementation terms:

- 1. **Where is request_id generated?** One place only. Why the gateway and not the API?
- 2. **What is the format?** UUID vs. ULID — what is the operational difference? (Hint: sort order under log query.)
- 3. **What happens if Symfony API receives a request with no X-Request-ID header?** (Internal call, health check, misconfigured client.) Generate a new one? Reject? Log a warning?
- 4. **How does Go worker receive the request_id for jobs enqueued during an HTTP request?** Draw the data flow: HTTP request → Symfony API → job payload → Go worker log.
- 5. **What is the one place request_id propagation silently fails?** (Hint: what happens when a Symfony service calls an external API without forwarding the header?)

Exercise 2: Incident Diagnosis

Setup: You have implemented the observability from Exercise 1.
Alert fired: “API error rate > 2% for last 5 minutes” — paged at 02:14.

Walk through the diagnosis in sequence. At each step, name the specific metric or log query you run and what answer you are looking for.

Step 1: First Metric to Check

You have one dashboard open. What is the first panel you look at and why?
It is not “all metrics.” It is one signal that tells you whether this is a spike (transient) or a sustained degradation (incident).

Question	Metric / query	What the answer means
Is the error rate still > 2%?	<code>rate(http_requests_total{status_class="5xx"}[5m]) / total</code>	Confirms alert is still firing, not already resolved
Is traffic volume normal?	<code>rate(http_requests_total[5m])</code>	Sudden traffic spike → resource saturation. Sudden drop → upstream failure
When did it start?	Time-series view of error rate, 1h window	Correlates with deploy, config change, or traffic event

Step 2: Narrow to Which Endpoint

The error rate is confirmed: 2.3%, still rising. The system has 12 endpoints. You do not check all 12.

Query	What you are looking for
<code>rate(http_requests_total{status_class="5xx"}[5m]) by (route)</code>	Which route accounts for the majority of errors
<code>histogram_quantile(0.99, rate(http_request_duration_bucket[5m])) by (route)</code>	Whether the erroring route is also slow (latency spike = different cause than errors alone)

Result: errors are concentrated on `POST /jobs/{id}/apply` . Latency on that route is also elevated (p99: 4200ms, normal: 180ms).

What does the combination of high error rate + high latency on a write endpoint suggest before you open a single log?

Step 3: Find the Failing Requests in Logs

You know the endpoint. Now you need the specific error.

Log query pattern:

```
service="symfony-api"
AND endpoint="POST /jobs/{id}/apply"
AND level="error"
AND timestamp > [alert_time - 10 min]
ORDER BY timestamp DESC
LIMIT 50
```

What you extract from the first matching log entry: - `request_id` — use this to pull the full request trace across services - `message` — the error class - `context.query_type` and `context.db_host` — whether this is a database operation

Cross-service correlation:

```
request_id="req_01HXYZ4K2M3N5P6Q7R8S"
AND service IN ("api-gateway", "symfony-api", "go-worker")
ORDER BY timestamp ASC
```

This reconstructs the full lifecycle of one failing request across all services. Without `request_id` propagated through all services, this query returns partial results — you see the API error but not what the worker did with the job, or whether the API error was caused by the worker failing to confirm.

Step 4: Determine if It Is a Downstream Dependency

You have the error message: “Database connection timeout on job application write.”

Check	Metric / query	What you are looking for
Is Postgres query latency elevated?	<code>histogram_quantile(0.99, rate(pg_query_duration_bucket[5m]))</code>	Yes → Postgres is slow, not just the app
Is connection pool exhausted?	<code>pg_connections_active / pg_connections_max</code>	> 90% → connection exhaustion causing timeouts
Is Postgres error rate elevated?	<code>rate(pg_errors_total[5m])</code>	Yes → queries failing, not just slow
Is Redis involved in this endpoint?	Check structured log <code>context</code> for Redis commands	If Redis timeout is in the trace, the cause may be Redis, not Postgres

Confirming the blast radius:

```
service="symfony-api"
AND context.db_host="postgres-primary"
AND level="error"
AND timestamp > [alert_time - 10 min]
GROUP BY endpoint
```

If multiple endpoints are showing Postgres errors → it is a Postgres incident, not a code regression on `/jobs/{id}/apply` .

Observability Gaps: What Makes This Diagnosis Impossible

For each gap, name what you cannot answer and what work-around you are forced to do instead:

Missing observability	Question you cannot answer	Forced work-around
No per-route error rate metric (only aggregate)	Which endpoint is failing	Check application logs manually, no metric to filter by
No request_id in Go worker logs	Whether a job enqueued by the failing request also failed	Check worker logs by time range and hope job_id is logged
No Postgres query duration metric	Whether the timeout is from slow queries or connection exhaustion	SSH into Postgres, run <code>pg_stat_activity</code> — manual, delayed, requires access
No structured logs (plain text only)	Filter logs by endpoint, user, or error type without parsing	<code>grep + awk</code> on raw files — cannot do in a dashboard at 02:14
No <code>duration_ms</code> in logs	Whether this is a timeout (slow) or an immediate error (fail-fast)	Only <code>status_code</code> — cannot distinguish timeout from crash
No Go worker queue depth metric	Whether jobs from failing requests are piling up	Log into queue management UI or message broker console manually

The critical gap: no `request_id` propagation into Go worker means you can confirm the API is failing but cannot confirm whether the jobs enqueued before the failure were processed, lost, or are retrying. The write endpoint failing after enqueue means partial state — the job may or may not be in the queue. Without cross-service correlation, you do not know.

Phase — 12-deployment-architecture

Directory: 12-deployment-architecture/

Source: 12-deployment-architecture/01-deployment-decisions.md

Deployment Decisions

Deployment strategy is an architectural decision. It determines blast radius of bad releases, downtime windows, rollback cost, and how fast you can ship safely. Architects sign off on this — not because they run pipelines, but because the strategy constrains everything else: database migration sequencing, rollback feasibility, infrastructure cost, and ops burden.

Deployment Strategies

Strategy	Mechanism	Downtime	Rollback speed	Extra cost	Key constraint
Rolling	Replace instances one by one	Zero	Minutes (redeploy old image)	None	Both versions run simultaneously — must be backward compatible
Blue-green	Two identical environments, switch traffic atomically	Zero	Seconds (flip LB back)	Double infra during switch	DB migrations must be compatible with both versions
Canary	Route N% of traffic to new version first	Zero	Immediate (drain canary, route to stable)	Traffic splitting infra	Requires metrics comparison and automated or manual promotion gate
Feature flags	Deploy code dark, enable per user/segment	Zero	Immediate (disable flag)	Flag management system, dead code	Separates deploy from release; flags must be cleaned up

When to choose what

Factor	Lean toward
Low ops maturity, low traffic	Rolling
Need fast rollback, acceptable infra cost	Blue-green
High traffic, complex change, blast radius matters	Canary
Major feature, A/B testing, gradual user rollout	Feature flags
Non-backward-compatible DB migration	Blue-green (but migration must still be backward compatible)
Rollback SLA < 60 seconds	Blue-green or feature flag

Decision matrix: four factors

Traffic volume	Rollback SLA	DB migration complexity	Ops maturity	Strategy
Low	> 1 hour	Simple additive	Low	Rolling
Low	< 15 min	Simple additive	Low	Blue-green
Medium	< 15 min	Backward compatible	Medium	Blue-green
Medium	< 15 min	Requires expand/contract	Medium	Rolling with strict 3-step sequence
High	< 5 min	Any	High	Canary
High	< 60 sec	None (code-only change)	High	Feature flag
Any	Immediate	None	Any	Feature flag (per-user rollout)
Any	Any	Not backward compatible	Any	Block deploy — fix migration first

The last row is the most important. “Not backward compatible” is not a deployment strategy decision — it is a blocker. Fix it before choosing a strategy.

Operational requirements per strategy

Strategy	Requires
Rolling	Backward-compatible code + schema; image tagging; health checks
Blue-green	Two full environments; LB traffic switching; migration runs before switch
Canary	Traffic splitting (LB weights or service mesh); metrics pipeline; promotion gate
Feature flags	Flag management service (LaunchDarkly, Unleash, or custom); flag cleanup discipline

Database Migrations in Deployments

This is the hardest part. The constraint: **a migration must be backward compatible with the currently running version of the application.**

Rolling or canary deployments mean old code and new code run simultaneously. If the migration breaks old code, you have an incident the moment it runs.

Expand/contract pattern

Never deploy migration and code simultaneously. Instead:

Step 1 — Expand (safe migration first) Deploy the migration that adds the new column. Old code ignores the new column. System continues working.

Step 2 — Deploy new code New code reads and writes the new column. Old column still exists. Both work. No simultaneous deploy.

Step 3 — Contract (cleanup migration) Once all instances run new code and old column is confirmed unused, deploy the migration that removes the old column.

This takes three deploys. That is the correct number.

What breaks with simultaneous deploy

Deploy code and migration at the same time: during the rolling update, some instances run old code against the new schema. If new schema removes a column old code reads — query fails. If new schema renames a column — same. “Works in staging” because staging is a single instance that flips atomically.

Migration backward compatibility checklist

- Adding a nullable column: safe
- Adding a column with a default: safe
- Dropping a column: unsafe unless no running code references it
- Renaming a column: unsafe — treated as drop + add
- Adding a NOT NULL column without default: unsafe — old inserts fail
- Adding an index: safe (use `CONCURRENTLY` in Postgres to avoid table lock)
- Adding a foreign key: check if old code can insert rows that would violate it

Rollback Design

Rollback must be designed before deployment, not after the incident.

Stateless services (Symfony API, Go worker)

Rollback = redeploy previous image. Tag every image with git SHA. Keep the previous N images. Rollback is a pipeline trigger.

Database changes

Only rollbackable if the migration was backward compatible. If it was — the old code still works against the current schema. If it wasn't — rollback means running a compensating migration under time pressure at 3am. That is a system design failure, not an ops problem.

Queue consumers (Go worker)

Queue messages were written by old code or new code. After rollback, old worker processes messages written by both. Options:

- **Version-aware message processing**: worker checks message version field and handles both
- **Separate queues per version**: new code writes to new queue; rollback leaves old queue intact
- **At-minimum**: ensure old worker does not corrupt state when it encounters messages it doesn't understand (dead-letter, not silent ignore)

Rollback decision tree

Bad deploy detected

- └ Stateless service bug, no DB migration
 - └ Redeploy previous image → done
- └ DB migration was backward compatible
 - └ Redeploy previous image (migration stays) → done
- └ DB migration was not backward compatible
 - └ Cannot rollback automatically
 - └ Compensating migration required → incident, not rollback
- └ Queue messages corrupted
 - └ Stop consumer → assess corrupted records → manual recovery required

Zero-Downtime Deployment Requirements

Three requirements. If any is missing, you will have downtime or errors during deploy.

- 1. Load balancer health checks** New instance must pass health check before receiving traffic. Health check endpoint must verify: app is ready (not just started), dependencies are reachable (DB connection pool initialized, Redis connected). Without this: LB sends traffic to an instance that is still warming up — connection refused or 500s.
- 2. Graceful shutdown** When a SIGTERM arrives (deploy replacing this instance): stop accepting new connections, finish in-flight requests, then exit. Set a drain timeout (30s is common). Without this: in-flight requests are killed mid-processing — data corruption, failed transactions, broken responses.
- 3. Connection pool warmup** Database and Redis connection pools take time to reach target size. New instance getting full traffic immediately overwhelms the pool — connections queue, timeouts spike. Solutions: pre-warm pool before going live (health check waits until pool is ready), or ramp traffic slowly (canary/rolling gives natural warmup time).

Environment Parity

Production incidents from environment differences are not bad luck — they are a configuration management failure.

Dev/staging must match production in:

- Config structure (not values — secrets differ, structure must not)
- Infrastructure topology (if prod has a Redis sentinel cluster, staging must too — not a single Redis)
- Dependency versions (Postgres minor version, Redis version, PHP version)
- Queue configuration (same queue names, same message formats)

The failure mode: “it worked in staging” because staging ran a Redis version that doesn’t enforce a behavior that Redis 7.x changed. Or staging had no queue and the job just ran inline.

Parity is not about identical data. It is about identical behavior.

Parity checklist for this stack

Component	What must match
Symfony API	PHP version, extension set, env var structure, queue transport config
Go worker	Go version, queue connection config, message schema version
Postgres	Major version, extensions (uuid-ossdp, pg_trgm), connection pool limits
Redis	Version, eviction policy, sentinel vs. standalone topology
Queue	Broker (AMQP/Redis Streams), queue names, dead-letter config, message TTL

Anti-patterns

- Blue-green with non-backward-compatible DB migrations** — you switch traffic instantly, but the old environment still points at the same database. The “fast rollback” switches traffic back to the old app running against the new schema. It fails.
- No health check endpoint** — load balancer has no signal. It sends traffic to the new instance the moment the process starts. Connection pools aren’t warm. First requests fail.
- Rollback plan that requires manual DB intervention at 3am** — if your rollback procedure includes “run this SQL manually,” it will be wrong under pressure. Compensating migrations must be pre-written and tested.
- Feature flags never cleaned up** — dead code accumulates. Every flag evaluation adds a branch. After 18 months, nobody knows which flags are live. Removing the flag is now risky. Cost: complexity that compounds.
- “We’ll test it in production” without canary** — this is a canary without the safety mechanism. Route 5% of traffic, not 100%.
- Simultaneous code and migration deploy** — explained above. Never.

Šta da pitaš AI

- “We need to deploy [describe change including DB migrations]. Which deployment strategy is safe? What are the steps in sequence? What must happen before each step?”
- “We have this database migration: [describe]. Is it backward compatible with the running version? What would break if old code runs against this schema?”
- “Design the rollback procedure for [describe deployment]. What are the steps? What state cannot be automatically rolled back and why?”
- “Our health check endpoint currently just returns 200. What should it actually check to be useful for zero-downtime deployments with our stack (Symfony + Go worker + Postgres + Redis)?”
- “We use rolling deployments. Our Go worker processes queue messages. During a rolling deploy, both old and new workers run simultaneously. What message compatibility requirements does this impose on us?”

Source: 12-deployment-architecture/02-lab-deployment.md

Lab: Deployment Decisions

Reference: Deployment Strategy Decision Table

Traffic volume	Rollback SLA	DB migration complexity	Ops maturity	Recommended strategy
Low	Hours	Backward compatible	Low	Rolling
Low	Hours	Not backward compatible	Low	Blue-green (migration still must be compat with both envs)
Medium	< 5 min	Backward compatible	Medium	Blue-green
High	< 60 sec	Any	High	Canary + feature flags
Any	Immediate (user-level)	None	Any	Feature flags

Reference: Expand/Contract Migration Sequence Template

Deploy 1 – Expand migration

Action: Add new column (nullable or with default)
Code: Old code (does not reference new column)
Risk: Zero – old code ignores new column
Verify: Migration applied, old app still passes health check

Deploy 2 – New application code

Action: No schema change
Code: New code reads/writes new column; also reads old column during transition
Risk: Low – schema is compatible with both versions
Verify: New behavior works; old column still readable

Deploy 3 – Contract migration

Action: Remove old column (only after 100% instances on new code)
Code: New code (does not reference old column)
Risk: Zero if step 2 is complete everywhere
Verify: Old column gone; no errors; query plans unaffected

Reference: Rollback Checklist Template

Pre-incident (design time):

- ☐ Previous image tag recorded (git SHA)
- ☐ Rollback pipeline tested in staging
- ☐ DB migration assessed: backward compatible? Y/N
- ☐ Queue message format: versioned? Y/N
- ☐ Compensating migration pre-written if needed

During incident:

- ☐ Stop traffic to new instances (or canary drain)
- ☐ Trigger rollback pipeline (previous image)
- ☐ Verify health checks pass on old version
- ☐ Verify error rate returns to baseline
- ☐ Assess: any queue messages processed by new code that old code cannot handle?
- ☐ Assess: any DB state written by new code that is invalid under old schema?

Post-rollback:

- ☐ Document what could not be rolled back automatically
- ☐ List affected records requiring manual recovery
- ☐ Root cause: what made this non-rollbackable?
- ☐ Fix plan before re-deploying

Exercise 1: Deployment Strategy Choice

Feature: Add `fulfilled_at` timestamp column to the `orders` table. Expose it in the orders API response (`GET /orders/{id}`). System runs rolling deployments. Current volume: medium traffic, rollback SLA is 15 minutes.

Work through the following:

1. Choose deployment strategy

Given: medium traffic, 15-minute rollback SLA, new nullable column (backward compatible migration), no removal of existing columns.

Choose a strategy and justify it. Consider: does the migration constrain your choice? Does the rollback SLA?

2. Migration sequence

Design the full expand/contract sequence. For each deploy, specify: - What schema change runs (if any) - What version of the Symfony API is deployed - What the running old version of the API does with the schema at that point - What a rollback looks like from that step

Step 1 — Expand migration: - Schema: `ALTER TABLE orders ADD COLUMN fulfilled_at TIMESTAMPTZ NULL` - API deployed: old version - Old code behavior: reads and writes orders without `fulfilled_at`, column exists but is ignored, all queries work - Rollback from here: `ALTER TABLE orders DROP COLUMN fulfilled_at` (safe, no code references it yet)

Step 2 — New API code: - Schema: no change - API deployed: new version (returns `fulfilled_at` in response, sets it on fulfillment) - During rolling deploy: some instances are old (return response without `fulfilled_at`), some are new (return it). Clients must tolerate optional field. - Rollback from here: redeploy old image. `fulfilled_at` column stays, old code ignores it. Safe.

Step 3 — (Optional) Remove old column if refactoring: - Only applicable if you had an old column you were replacing. In this case, no old column was removed — this step does not apply.

3. Rollback procedure

Rollback is needed 10 minutes after Step 2 deploy — new API has a bug in the `fulfilled_at` response serialization.

- Action: trigger pipeline, redeploy previous image tag
- Schema: `fulfilled_at` column remains — old code ignores it, no harm
- In-flight orders: any `fulfilled_at` values already written stay in the DB, ignored by old code
- Queue: Go worker is not involved in this change — no action needed
- Time to rollback: depends on rolling deploy speed, but no manual DB intervention required

4. What breaks if you deploy migration and code simultaneously

Scenario: Step 1 and Step 2 are combined into one deploy (migration + new code in same pipeline step, applied to all instances at once).

During rolling deploy: - New instances: running new code against new schema — works - Old instances still running: running old code against new schema — `fulfilled_at` exists but is ignored, this is actually safe in this specific case

This scenario is accidentally safe because the new column is nullable with no NOT NULL constraint and old code doesn't reference it. But this is the exception, not the rule.

Now change the scenario: new code also removes the old `notes` column from the API response, and the migration drops it simultaneously.

- Old instances running old code: `SELECT notes FROM orders` — column is gone — query fails — 500 errors until old instances are replaced
- This is why simultaneous deploy is dangerous: it works until the migration removes something old code depends on

Lesson: simultaneous deploy is only safe if you can guarantee the migration is a pure addition with no removals. That guarantee is fragile across teams. The discipline is: always separate.

Exercise 2: Rollback Design

Incident: New version of the Go worker has a bug. When processing fulfillment messages, it subtracts inventory twice instead of once (double-decrement). Deployed 30 minutes ago. 500 orders processed. Inventory counts for affected SKUs are now wrong by -500 units each (assuming 1-unit orders).

1. Immediate mitigation — stop the bleeding

- Stop the Go worker (kill the deployment / scale to 0 replicas)
- Do not process any more fulfillment messages
- Messages remain in the queue (not lost — queue is durable)
- Order processing will stall, but data corruption stops now
- Notify downstream: inventory service data is unreliable for the next N minutes
- Do not run any inventory reorder logic or fulfillment automation until counts are corrected

2. Rollback procedure for the Go worker

- Identify previous image tag (git SHA from deploy log or image registry)
- Trigger rollback pipeline: deploy previous image
- Verify health checks pass
- Do not resume queue processing yet — queue has messages that were correctly dequeued but not reprocessed; assess state first
- Once confirmed: resume queue processing with old worker
- Monitor: inventory decrements are now correct (subtract once)

3. What you cannot roll back automatically and why

Inventory counts — the database already has wrong values. Redeploying old code does not fix the data. The old worker will process future messages correctly, but the 500 corrupted records remain wrong.

Why: rollback restores behavior, not state. State mutations caused by the buggy code are permanent until explicitly corrected. There is no automatic undo for a committed database write.

Queue messages already processed — the 500 fulfillment messages were successfully dequeued and acknowledged. They are gone from the queue. The old worker cannot reprocess them because they no longer exist in the queue.

Why: at-least-once delivery means messages are acked on processing. The worker marked them done. Rollback does not restore them to the queue.

Time — 30 minutes of incorrect state may have already triggered downstream effects: reorder alerts, reporting snapshots, customer-facing inventory displays showing wrong counts.

4. Recovery steps for the 500 affected orders

Identify affected records: - Query: all orders fulfilled in the 30-minute window by the new worker version - Cross-reference with inventory audit log (if one exists) or reconstruct from order line items

Calculate correct inventory delta: - Each affected order decremented inventory once too many times - Correct adjustment:

`UPDATE inventory SET quantity = quantity + 1 WHERE sku_id = ? AND order_id was processed in window` - Or: aggregate by SKU — total over-decremented units per SKU = count of affected orders per SKU

Apply compensating update: - Write a migration/script: corrects inventory counts for each affected SKU - Run in a transaction with a dry-run first (log expected changes before committing) - Verify: sum of corrections matches expected (500 orders × affected SKUs)

Audit trail: - Record the correction as a named event in the audit log — do not silently patch - Include: timestamp, cause (bug in worker version X), orders affected, SKUs corrected, delta applied

Downstream notifications: - If inventory counts were consumed by other systems in the window: alert those systems - If reorder triggers fired: cancel or review them - If customer-facing counts were cached: invalidate cache

Key takeaways from both exercises

Expand/contract is not optional on rolling deployments — it is the only safe sequencing when old and new code run simultaneously.

Rollback restores behavior. It does not restore state. Any system that modifies durable state (DB writes, queue acks, external API calls) accumulates changes that survive a rollback. Design for this — compensating actions must be pre-planned, not improvised.

The 3am rollback plan must be executable by someone who did not write the code. If it requires domain knowledge to figure out which records to fix — it is not a rollback plan, it is a hope.

Phase — 13-network-layer

Directory: 13-network-layer/

Source: 13-network-layer/01-network-for-architects.md

Network Layer for Architects

Architects don't configure networks — they decide where components live and what flows between them. You need enough mental model to make the right call and ask the right question. Leave the CIDR blocks to the network engineers.

Load Balancers

L4 vs L7

Dimension	L4 (TCP/UDP)	L7 (HTTP/HTTPS)
Operates at	Transport layer — sees IP + port	Application layer — sees HTTP headers, paths, cookies
Routing decisions	IP address + port only	Path, host, header, cookie values
TLS	Pass-through or terminate	Terminate (and optionally re-encrypt)
Throughput	Higher — no payload parsing	Slightly lower — must parse HTTP
Use case	Non-HTTP protocols, raw throughput, database proxying	Web services, APIs, microservices

Default choice for web services: L7. Use L4 when you have non-HTTP protocols (Postgres TCP proxy, SMTP) or need raw throughput where HTTP parsing overhead matters.

Where TLS Terminates

Three patterns:

Terminate at load balancer (most common) - LB holds the cert, decrypts inbound traffic, forwards plain HTTP internally - Requires internal network to be trusted - Simple cert management — one cert at one place - Right choice for most systems on private VPC/cloud network

End-to-end TLS (re-encrypt) - LB terminates, re-encrypts to backend - Traffic encrypted even on internal network - Higher ops overhead: backend certs must also be managed and rotated - Choose when compliance requires encryption in transit even on internal segments

mTLS everywhere - Every service presents and validates a client cert - Correct for zero-trust architecture - Expensive to operate: cert rotation, service mesh required (Istio, Linkerd), debugging harder - Choose when: zero-trust mandate, highly sensitive data, regulated environment

Health Checks

Every service behind a load balancer needs a `/health` endpoint: - Returns `200` only when the instance is fully ready to serve traffic - Not “process is running” — “ready to serve”: DB connection established, migrations complete, warm-up done - LB polls health check on an interval (e.g. every 10s); after N consecutive failures, removes the instance from rotation - Without this, LB sends traffic to instances that are starting up, draining, or degraded

What `/health` should check for Symfony API: - Database connection available - Redis connection available - No critical config missing

What `/health` should NOT do: - Run slow queries - Check downstream services (that creates cascading failures — if Stripe is down, your instances aren't down)

Sticky Sessions

LB routes the same user (by cookie or IP) to the same backend instance.

- Required when session state is stored locally on the instance (in-memory, local filesystem)
- **Architectural smell** — it means your service is stateful in a way that breaks horizontal scaling

- Failure mode: if the pinned instance goes down, that user's session is lost
- Fix: move session state to Redis, remove sticky sessions, make service stateless

DNS

TTL as a Deployment Tool

DNS TTL controls how long resolvers cache a record before re-querying.

Scenario	TTL Strategy
Normal operation	300s (5 min) — standard tradeoff of freshness vs resolver load
Before planned traffic shift	Lower to 30–60s, at least 2× current TTL in advance
After cutover complete	Raise back to 300s
Emergency failover to new IP	Low TTL was required before the emergency — you can't retroactively fix it

Blue-green cutover via DNS: if your current TTL is 300s and you lower it right before cutover, you'll still have resolvers that cached the old value for up to 5 minutes. Pre-warm: lower TTL 24–48 hours before the cutover window.

Internal Service Discovery vs DNS

Approach	Mechanism	When to Use
Plain DNS	Services register A records, clients resolve by name	Kubernetes services (kube-dns), simple cloud setups
Service registry (Consul, AWS Cloud Map)	Services register on startup, deregister on shutdown, health-aware	Larger systems, need metadata, health-aware routing
Service mesh (Istio, Linkerd)	Sidecar proxies handle discovery + routing + mTLS	Complex microservices, zero-trust, canary routing needed

Architect decision: complexity budget. DNS is almost always sufficient to start. Add a service registry when you need health-aware routing or cross-datacenter. Add a mesh when you need policy enforcement at the network layer.

CDN

A CDN caches content at edge nodes close to users. Reduces round-trip latency from 100–200ms (origin) to 5–20ms (edge). Reduces origin load for cacheable traffic.

What Architects Decide

Cache policy per path — set by `Cache-Control` response headers: - `Cache-Control: public, max-age=86400` — cache for 24h, any proxy can store - `Cache-Control: private, no-store` — do not cache anywhere - `Cache-Control: public, max-age=3600, stale-while-revalidate=60` — cache 1h, serve stale while revalidating

Purge strategy — how to invalidate cached content when it changes: - Tag-based purge: assign cache tags to responses, purge by tag (e.g. purge all `product-42` when product 42 changes) - Path purge: invalidate a specific URL - Global purge: nuclear option — clears everything, origin gets hit hard

Origin shield — an intermediate caching layer between CDN edge nodes and your origin server. When 100 edge nodes all miss cache simultaneously, only the shield node contacts origin. Reduces origin load dramatically for high-traffic spikes.

When NOT to Cache

Path Type	Cache?	Reason
Authenticated responses	No	Per-user content — caching leaks data across users
Real-time data (prices, inventory)	No	Must be fresh on every request
POST/PUT/DELETE responses	No	Mutations should never be cached
Static assets (CSS, JS, images)	Yes	Content-addressed (hash in filename), long TTL safe
Public, infrequently changing pages	Yes	With appropriate max-age and purge on publish

TLS and Certificates

Certificate Rotation Without Downtime

Requirements: 1. **Dual-cert support during rotation** — load balancer or server must accept both old and new cert simultaneously during the transition window 2. **Automated renewal** — Let’s Encrypt (certbot), AWS ACM auto-renews, no manual process 3. **Expiry monitoring** — alert at 30 days, page at 7 days. Certificate expiry is an operational failure, not a surprise.

Rotation failure mode: cert expires in production, HTTPS breaks, users see security errors. This is entirely preventable and entirely an ops/monitoring failure.

Decision Table

Traffic type	Security requirement	Session requirement	L4/L7	TLS termination	Sticky sessions
HTTP API, web	Standard	Stateless (Redis session store)	L7	At LB	No
HTTP API, web	Compliance: encrypt in transit	Stateless	L7	Re-encrypt at LB	No
HTTP API, web	Zero-trust mandate	Stateless	L7 + service mesh	mTLS end-to-end	No
Database TCP proxy	N/A	N/A	L4	Pass-through	No
Legacy app, local sessions	Standard	Stateful (fix this)	L7	At LB	Yes (temporary)
High-throughput non-HTTP	Standard	N/A	L4	At LB or pass-through	No

Anti-Patterns

No health check endpoint — LB sends traffic to instances that are starting up, shutting down, or in a degraded state. Every service needs `/health` that reflects actual readiness.

Sticky sessions masking stateful service — you think you fixed the problem (it works), but you’ve hidden a scaling and reliability problem. One instance failure = user sessions lost.

High DNS TTL on services you need to failover quickly — if TTL is 3600 and something goes wrong, you have up to an hour of traffic going to the dead endpoint. Establish TTL hygiene before you need it.

CDN caching authenticated responses — a bug in cache key configuration and User A sees User B’s data. If a response depends on session/auth, `Cache-Control: private, no-store`.

mTLS everywhere on day one — operationally expensive, debugging is harder, slows down development. Earn it. Start with TLS termination at LB + trusted internal network. Add mTLS when you have the operational maturity for it.

PoC that tests “does the load balancer work” — LBs work. Test your specific failure modes: health check behavior during rolling deploy, sticky session failover, cert rotation under load.

Šta da pitaš AI

- “We have [describe traffic flow, protocol, auth model]. Where should TLS terminate? What are the security tradeoffs of each option for our setup?”
- “Our Symphony API currently uses sticky sessions. Walk me through the failure modes and the migration path to stateless sessions with Redis.”
- “We need to do a zero-downtime DNS cutover from IP A to IP B. Our current TTL is [X]. What is the exact TTL strategy and timing?”
- “Design the CDN cache policy for these paths: [list paths with descriptions]. For each: cacheable or not, Cache-Control header, purge trigger.”

- “We’re adding a Go worker that communicates with Symfony API internally. Does this traffic need TLS? What are the options and their tradeoffs?”

Source: `13-network-layer/02-lab-network.md`

Lab: Network Layer

Exercise 1: Architecture Diagram with Network Layer

Starting point

```
Browser → ??? → Symfony API (3 instances) → Postgres + Redis
```

Fill in the network layer. Answer each question in writing, then draw the diagram using the template below.

Questions to answer

1. What sits between Browser and Symfony?

Choose from: L4 load balancer, L7 load balancer, API gateway + L7 LB, CDN + L7 LB. State which and why. Consider: Symfony API is HTTP, needs path-based routing in future, team is 4 engineers.

2. Where does TLS terminate?

State the termination point and what happens to traffic after it. State what internal traffic looks like (encrypted or plain). State what assumption you’re making about network trust.

3. What health check does each component need?

Define `/health` behavior for: - Symfony API instance: what does it check, what does it return, what makes it return non-200 - Postgres: does it need a health check exposed to the LB? How does the LB know if Postgres is degraded? - Redis: same question

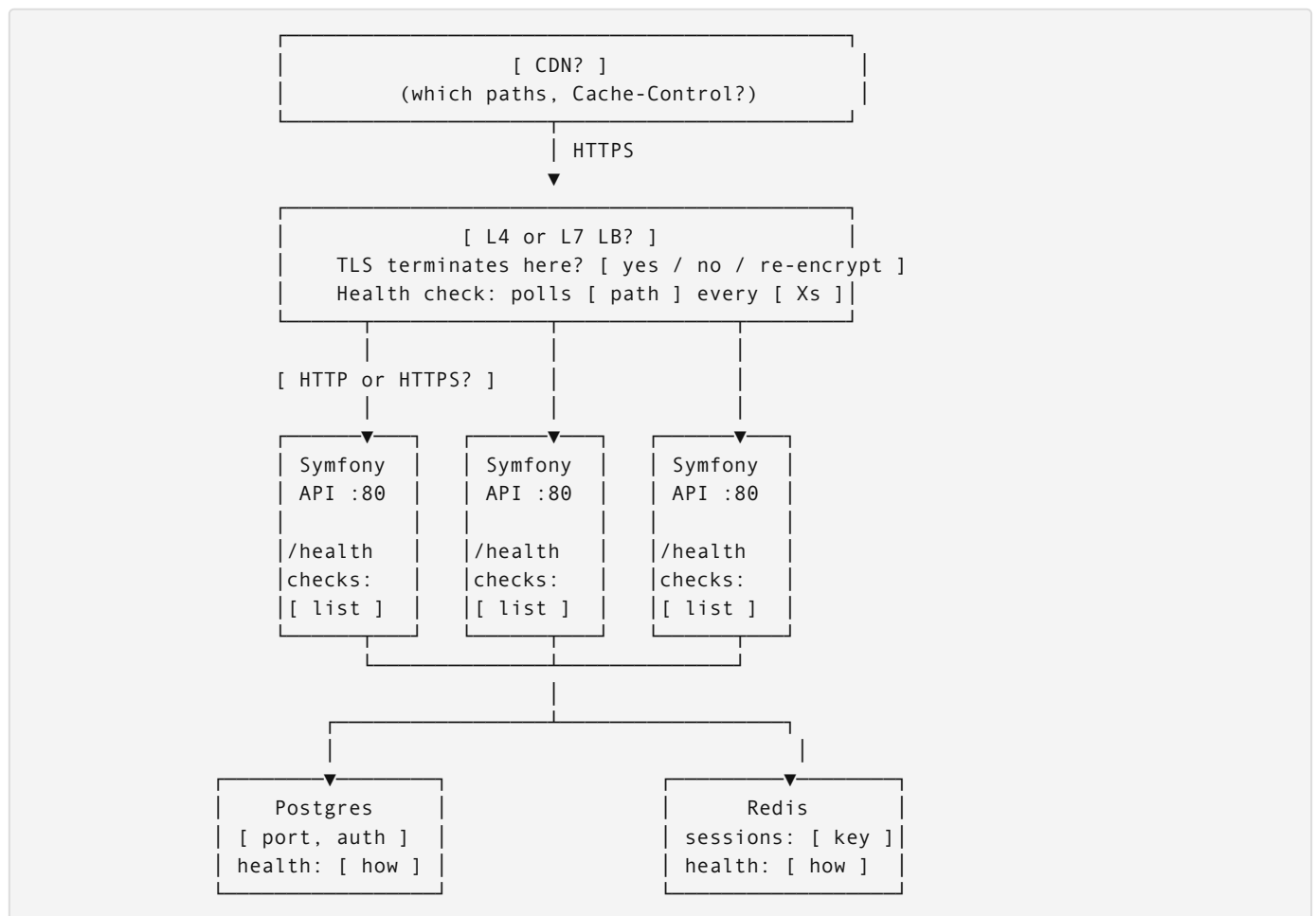
4. How is session state handled without sticky sessions?

The existing Symfony app stores PHP sessions in the local filesystem. You are removing sticky sessions.

State: (a) what changes in Symfony config, (b) what Redis key structure you’d use for sessions, (c) what the failure mode is if Redis goes down, and (d) whether you’d use a Redis cluster or single instance for sessions and why.

Diagram template

Fill in each `[ ]`:



Reference answer (review after completing)

Expand after you've written your own answer

****Between browser and Symfony:**** L7 load balancer (e.g. AWS ALB, nginx, HAProxy). L7 because HTTP traffic needs path-based routing capability as the system grows, SSL termination, health check by HTTP status. No CDN needed yet unless there are static assets or the team is already managing one — defer CDN until latency or origin load is a problem. ****TLS termination:**** At the L7 LB. LB holds the TLS cert. Internal traffic (LB → Symfony instances) is plain HTTP on a private network/VPC. Assumption: internal network is trusted — instances are on the same VPC and not reachable from the internet. This is correct for most cloud deployments. ****Health checks:**** - Symfony `/health`: checks DB connection (`SELECT 1`), Redis ping. Returns 200 only if both pass. Returns 503 if either fails. Does NOT call external services (Stripe, email — those failing doesn't mean your instance is unhealthy). - Postgres: LB doesn't health-check Postgres directly. Symfony's `/health` includes the DB check — if DB is down, Symfony instances return 503 and LB stops sending traffic. Postgres itself monitored separately (CloudWatch, `pg_stat`, Prometheus `postgres_exporter`). - Redis: same pattern — Symfony `/health` includes Redis ping. Redis monitored separately. ****Sticky sessions → Redis:**** - Symfony config: `session.save_handler = redis`, `session.save_path = tcp://redis:6379`. Remove any `session.sticky` LB config. - Key structure: `symfony_session:{session_id}` — Symfony handles this automatically via the Redis session handler. - Redis failure mode: if Redis is unavailable, session reads fail → users get logged out or see errors. Mitigation: Redis with replica + Redis Sentinel or Redis Cluster for HA. Alternatively, treat session loss as acceptable (re-login) and add graceful degradation. - Single instance vs cluster: for sessions, Redis Sentinel (1 primary + 1 replica + sentinel for automatic failover) is sufficient for most loads. Full cluster adds complexity without benefit unless you're partitioning session data across many nodes for write throughput.

Exercise 2: CDN Decision

Context

E-commerce site. Origin: Symfony API behind an L7 LB. CDN sits in front of the LB.

Paths to evaluate

Path	Description
/	Homepage — marketing content, hero image, featured products. Updated by marketing team a few times per week.
/products	Product catalog. Prices and stock levels update hourly via a cron job. No auth required.
/checkout	Multi-step checkout flow. User must be authenticated. Cart contents are session-specific.
/static/app.css, /static/app.js	Compiled frontend assets. Filename includes content hash (e.g. app.a3f9b2.css).
/api/orders	REST endpoint returning authenticated user's order history. JSON response.
/api/products/search	Product search. Public. Results are the same for all users for a given query. Can be slightly stale.

Your task

For each path, complete this table:

Path	CDN cacheable?	Cache-Control header	Purge trigger	Notes
/				
/products				
/checkout				
/static/				
/api/orders				
/api/products/search				

Reference answer (review after completing)

Expand after you've written your own answer

Path	CDN cacheable?	Cache-Control header	Purge trigger	Notes
/	Yes	`public, max-age=3600, stale-while-revalidate=300`	On marketing publish event	1h cache, serve stale while revalidating. Purge when content is updated — not on a schedule.
/products	Yes	`public, max-age=3600, stale-while-revalidate=60`	After hourly cron job completes	Max-age matches update frequency. Don't cache longer than update interval or users see stale stock/price.
/checkout	No	`private, no-store`	N/A	Session-specific. Must not be cached anywhere. Any caching here leaks one user's cart to another.
/static/	Yes	`public, max-age=31536000, immutable`	Never (content-addressed)	Hash in filename = new filename when content changes. Cache forever. `immutable` tells browser not to revalidate.
/api/orders	No	`private, no-store`	N/A	Authenticated, user-specific. Same as checkout.
/api/products/search	Yes	`public, max-age=300, stale-while-revalidate=60`	On product update	5-min cache is acceptable if "slightly stale" per requirements. Vary by query string (CDN must key on full URL including `?q=...`).

Key point on `/api/products/search`: Ensure CDN cache key includes the full query string. If the CDN treats `?q=shoes` and `?q=boots` as the same URL, it will serve the wrong cached response to all search queries.

Key point on `/static/`: The `immutable` directive tells browsers not to revalidate even after `max-age` would normally expire. Only safe because the filename changes when the file changes. Never use `immutable` on URLs that can serve different content at the same path.

Checklist before moving on

- ☐ You can explain L4 vs L7 in one sentence each and name a scenario where you'd choose L4 over L7
- ☐ You can describe where TLS terminates in your reference architecture and what assumption that makes about the internal network
- ☐ You can define what a correct `/health` endpoint checks and what it deliberately does not check
- ☐ You can explain why sticky sessions are an architectural smell and the migration path
- ☐ You can fill in a CDN decision table for any path given its auth requirement and freshness requirement
- ☐ You know what DNS TTL strategy to use before a planned cutover

Phase — 14-technology-evaluation

Directory: 14-technology-evaluation/

Source: 14-technology-evaluation/01-evaluating-technology.md

Evaluating Technology

Technology decisions outlast the people who made them. A bad choice compounds for years — migration costs, expertise gaps, operational toil, and the slow drag of a system that wasn't designed for the problem it's solving. "It's popular" and "we used it before" are not evaluation criteria. Neither is "the team wants to learn it."

The Evaluation Framework

Step 1: Define constraints first

Constraints eliminate options before you compare. They are non-negotiable: things that cannot change regardless of how good a technology looks on paper.

Common constraints: - **Team skills** — if no one on the team knows Kafka, operational complexity of Kafka is dramatically higher than its paper complexity - **Existing stack** — switching databases mid-product is a migration project, not a technology evaluation - **Compliance and security** — data residency requirements eliminate cloud-native options in some regions - **Ops maturity** — a 4-person team cannot operate Kafka, Elasticsearch, Cassandra, and Kubernetes simultaneously - **Budget** — fully managed services cost more in hosting but less in engineering time; make this tradeoff explicit

List your constraints. Anything that violates a constraint is eliminated. Don't evaluate eliminated options — it wastes time and creates false debate.

Step 2: Define selection criteria with weights

Criteria are what you're optimizing for. They vary by decision.

Criterion	Weight (1–5)	Notes
Throughput		Matters for message queues, not for admin dashboards
Operational complexity		Higher weight for small teams
Ecosystem maturity		Client libraries, community answers, known failure modes
Cost (TCO)		Hosting + engineering time + migration
Learning curve		Time before team is productive
Feature fit		Does it actually solve the problem without workarounds

Weight them. If you don't weight, every criterion has equal weight, which is almost never true. A team of 4 should weight operational complexity at 5; a platform team of 20 can afford to weight it lower.

Step 3: Generate at least 3 options including "stay with current"

"Stay with current" is always an option. It has known failure modes, known operational cost, and zero migration cost. If you can't articulate why it's insufficient for the new requirement, you shouldn't switch.

Step 4: Score each option per criterion

Not "Kafka is better." Score: Kafka scores 5/5 on throughput, 2/5 on operational complexity for a team of 4, 4/5 on ecosystem maturity, 2/5 on TCO for self-hosted.

Scoring forces honest comparison. It surfaces tradeoffs explicitly rather than letting the loudest voice win.

Step 5: State what you're optimizing for

The decision is only good given your current priorities. Document: "We chose X because we weighted operational simplicity at 5 (team of 4, no dedicated ops) and throughput at 2 (current volume doesn't require it). If volume grows to >50k/msg/s or team grows to include a dedicated platform engineer, re-evaluate."

This is what allows future teams to re-evaluate correctly rather than cargo-culting the original decision.

Step 6: Total cost of ownership

Don't compare licensing costs. Compare: - Migration cost (how much work to move from current to new) - Learning curve (time before team is productive — not just "get it running" but "debug it at 2am") - Ongoing ops (patching, upgrades, capacity planning, incident response) - Tooling (monitoring, alerting, backup, restore) - Debugging when it breaks (how hard is it to find root cause in production)

Common Evaluations Architects Face

Message Queue: RabbitMQ vs Kafka vs Cloud-native (SQS/Pub-Sub)

Criterion	RabbitMQ	Kafka	SQS/Cloud-native
Throughput	Medium (tens of thousands/s)	High (millions/s)	Medium — managed ceiling
Message replay	No (messages consumed = gone)	Yes — consumers can re-read from offset	No (SQS) / Limited (Pub-Sub)
Ops complexity	Low-medium — manageable for small team	High — requires Kafka expertise	Minimal — managed service
Routing flexibility	High — exchanges, bindings, dead-letter	Limited — topics and consumer groups	Simple FIFO / fan-out
Vendor lock-in	None	None	Yes — cloud vendor
Good for	Task queues, RPC, complex routing, <10k msg/s	Event streaming, audit log, analytics, replay	Cloud-native teams, simple queues, don't want ops

Decision driver: Do you need replay? If yes, Kafka (or cloud event streams like AWS Kinesis). Do you have ops capacity for Kafka? If no, Kafka is the wrong answer regardless of throughput. RabbitMQ is underrated — it solves most task queue problems with far less operational overhead.

Database: Postgres vs MySQL vs Managed

Criterion	Postgres (self-hosted)	MySQL (self-hosted)	Managed (RDS/CloudSQL)
JSON support	Excellent (JSONB, indexable)	Adequate	Depends on engine
Feature set	Richer (window functions, CTEs, PostGIS)	Adequate	Depends on engine
Replication	Logical + streaming	Binary log	Handled by provider
Ops overhead	Requires DBA attention for tuning	Similar	Low — provider handles backups, failover, patching
Cost	Hosting only	Hosting only	Higher hosting, lower engineering
Choose when	You have DBA capacity and specific tuning needs	Migrating from MySQL or ecosystem requirement	Team doesn't want to operate databases

Default choice for new systems: managed Postgres. The managed overhead is worth it for most teams. Choose self-hosted only if you have a DBA and specific performance requirements that managed services can't accommodate.

Cache: Redis vs Memcached

Criterion	Redis	Memcached
Data structures	Strings, hashes, lists, sets, sorted sets, streams	Strings only
Pub/sub	Yes	No
Persistence	Optional (RDB, AOF)	No
Clustering	Redis Cluster	Built-in sharding
Ops complexity	Moderate	Lower
Choose when	Almost always — default choice	Pure key-value at extreme scale with nothing else needed

Default choice: Redis. Memcached is marginally faster for pure get/set at very high throughput. In practice, the operational simplicity of having one cache system that also handles sessions, rate limiting, and pub/sub outweighs the throughput difference.

Avoiding Evaluation Theater

The tell: “We evaluated 5 options and chose Kafka” with no documented criteria, no scoring, no tradeoffs stated. This is post-hoc rationalization — the decision was made before the evaluation started.

PoC before decision for high-risk choices. A PoC is not a demo. It tests the riskiest assumption — the thing that, if wrong, invalidates the decision. If you’re evaluating Kafka for replay: your PoC should test whether your consumer can replay 90 days of messages at your projected volume within your SLA window. Not whether Kafka starts and a message flows through it.

Time-box the evaluation: - Low-risk (mature technology, reversible choice): 3–5 days - High-risk (new technology, expensive to reverse): 1–2 weeks - Evaluations that drag on produce analysis paralysis, not better decisions

Anti-Patterns

Choosing based on job posting trends — “Kafka is on all the job listings” tells you the market is using Kafka. It tells you nothing about whether your system needs Kafka. Trend-following is not evaluation.

Choosing based on what the team wants to learn — valid input, not the primary criterion. “We want to learn Kafka” belongs in the learning curve row of the evaluation matrix, not in the decision driver.

Not including “stay with current” as an option — if your current solution is excluded from comparison, the evaluation is biased toward switching. Include it. If it loses, the case for switching is stronger.

PoC that tests only the happy path — testing that Kafka receives and delivers a message proves nothing. The happy path always works. Test: what happens when a consumer crashes mid-batch, when a topic is full, when the consumer group falls behind, when the broker restarts.

TCO calculation that excludes migration — a new technology that’s cheaper to run but costs 6 months of engineering time to migrate to is not cheaper. Include the migration in the cost column.

Evaluating without weights — every criterion having equal weight means operational complexity ties with throughput. That’s almost never the right call. Weight explicitly.

Šta da pitaš AI

- “We need to choose between [A] and [B] for [specific use case]. Our constraints are [list]. Our weighted criteria are [list with weights]. Evaluate each option per criterion — score 1–5 and explain the score. Do not give a recommendation, just compare.”
- “We’re considering adopting [technology]. What are the operational failure modes at [our scale and team size]? What expertise does the team need to operate it safely in production?”
- “We chose [technology] 2 years ago. Our situation has changed: [describe changes]. Re-evaluate that decision against [alternatives] given the new context.”
- “Build a comparison table: [A] vs [B] vs [C] on these criteria: [list]. Don’t recommend — just compare so I can apply our weights.”
- “Design a PoC for evaluating [technology] for [use case]. What is the riskiest assumption we need to test? What does pass look like? What does fail tell us?”

Lab: Technology Evaluation

Exercise 1: Message Queue Evaluation

Context

E-commerce platform. Currently using RabbitMQ for order processing. Volume: 5,000 orders/day (~0.06 msg/s average, peaks at ~20 msg/s during flash sales). Team: 5 engineers, no dedicated ops. Stack: Symfony API + Go worker + Postgres.

New requirements: 1. Real-time analytics dashboard showing order trends (requires streaming order events to an analytics consumer) 2. Compliance audit log with 90-day replay capability (auditors can re-run any 90-day window)

RabbitMQ in its current configuration cannot satisfy requirement 2 (no replay) and satisfies requirement 1 only partially (no durable consumer replay).

Options to evaluate

- **Option A: RabbitMQ (stay)** — extend current setup, potentially with message archiving workaround
- **Option B: Kafka (replace RabbitMQ)** — self-hosted, team operates it
- **Option C: AWS Kinesis (replace RabbitMQ)** — managed cloud stream, vendor-specific

Step 1: Define your constraints

List the constraints that eliminate options or bound the decision. Consider: team size, ops capacity, existing cloud provider (assume AWS), compliance requirements (90-day replay must be durable and auditable), timeline.

Write your constraints here before proceeding.

Step 2: Define criteria with weights

Complete this table. Add or remove rows as needed. Assign weights 1–5 (5 = most important for this decision).

Criterion	Weight (1–5)	Why this weight for this decision
Replay capability		
Throughput		
Operational complexity		
Team expertise required		
Cost (TCO, 1 year)		
Migration effort from RabbitMQ		
Ecosystem / tooling		

Step 3: Score each option

Complete this matrix. Score 1–5 per criterion per option. Add a brief justification for each score.

Criterion	Weight	RabbitMQ (A)	Score	Kafka (B)	Score	Kinesis (C)	Score
Replay capability							
Throughput							
Operational complexity							
Team expertise required							
Cost (TCO)							
Migration effort							
Ecosystem / tooling							
Weighted total							

Calculate: weighted total = sum of (weight × score) for each option.

Step 4: State the decision

Write: “We choose [option] because we are optimizing for [primary criteria]. The key tradeoff we accept is [what we’re sacrificing]. If [condition changes], we should re-evaluate.”

Reference answer (review after completing)

Expand after you've written your own answer

****Constraints:**** - Team of 5, no dedicated ops — eliminates self-operated systems with high operational overhead unless the team explicitly accepts the investment - Already on AWS — AWS-native options carry no additional vendor lock-in beyond existing commitment - 90-day replay must be durable and auditable — eliminates vanilla RabbitMQ (messages are deleted after consumption) - Timeline: assume normal (not emergency) — 2–3 week evaluation window acceptable ****Sample scoring rationale:**** | Criterion | Weight | RabbitMQ | Score | Kafka | Score | Kinesis | Score | |---|---|---|---|---|---|---|---| | Replay capability | 5 | Needs archiving workaround; not native | 2 | Native, configurable retention | 5 | Native, 7–365 day retention | 5 | | Throughput | 2 | Sufficient for current volume | 4 | Massive headroom | 5 | Sufficient, managed ceiling | 4 | | Operational complexity | 5 | Low — team already operates it | 5 | High — Kafka expertise, ZooKeeper/KRaft, JVM tuning | 2 | Minimal — managed service | 5 | | Team expertise required | 4 | Already known | 5 | Significant ramp-up needed | 2 | Moderate — AWS SDK, Kinesis concepts | 4 | | Cost (TCO 1yr) | 3 | Low — existing infra | 5 | Medium — EC2/EKS overhead + eng time for ops | 3 | Medium — per-shard pricing, higher than EC2 at scale | 3 | | Migration effort | 3 | Minimal — extend, not replace | 5 | High — parallel run, consumer migration, data cutover | 2 | Medium — producers/consumers need rewrite | 3 | | Ecosystem / tooling | 2 | Mature, well-known | 4 | Excellent — Kafka Connect, Kafka Streams, Confluent | 5 | Good — AWS ecosystem, Kinesis Data Analytics | 4 | ****Weighted totals (approximate):**** - RabbitMQ: $(2 \times 5) + (4 \times 2) + (5 \times 5) + (5 \times 4) + (5 \times 3) + (5 \times 3) + (4 \times 2) = 10 + 8 + 25 + 20 + 15 + 15 + 8 = 101$ - Kafka: $(5 \times 5) + (5 \times 2) + (2 \times 5) + (2 \times 4) + (3 \times 3) + (2 \times 3) + (5 \times 2) = 25 + 10 + 10 + 8 + 9 + 6 + 10 = 78$ - Kinesis: $(5 \times 5) + (4 \times 2) + (5 \times 5) + (4 \times 4) + (3 \times 3) + (3 \times 3) + (4 \times 2) = 25 + 8 + 25 + 16 + 9 + 9 + 8 = 100$ ****Decision:** Kinesis (narrowly over extended RabbitMQ). ******“We choose Kinesis because we are optimizing for operational simplicity (team of 5, no ops headcount) and replay capability (compliance requirement). The key tradeoffs we accept are AWS vendor lock-in (beyond existing commitment) and higher per-unit cost at scale. We are not choosing Kafka because the operational burden for self-hosting is too high for our team size, and the throughput advantage is irrelevant at our volume. We are not extending RabbitMQ because the replay workaround adds operational complexity without solving the compliance requirement cleanly. If we grow to a team with a dedicated platform engineer, or if volume exceeds 100k msg/s, re-evaluate Kafka.” ****Alternative valid answer:**** RabbitMQ with a message archive pattern (write to S3 on consume, replay from S3 for audit) is legitimate if migration cost is weighted heavily. The scoring exercise should surface this tradeoff explicitly.

Exercise 2: PoC Design

Context

You’ve chosen Kinesis (or Kafka — use whichever you chose in Exercise 1) for the order processing and compliance audit log use case. Before committing to full implementation, you want to run a PoC.

A bad PoC tests “does the technology work” — it always does. A good PoC tests the riskiest assumption about your specific use case.

Your task

Design the PoC by answering these four questions:

1. What specifically are you testing?

Not “does Kinesis receive and deliver messages.” State the specific capability or behavior that, if it fails, would invalidate the decision. What did you assume would be true that you haven’t verified against your actual use case?

2. What is the riskiest assumption?

State the single assumption that is most likely to be wrong and most expensive to discover late. For the audit log use case, think about: replay latency at volume, consumer group behavior during catch-up, integration with your existing Go worker, cost at projected volume.

3. What does a passing PoC look like?

Define the success criterion precisely. “It works” is not a success criterion. State: observable behavior, measurable threshold, and how you verify it.

4. What does a failing PoC tell you?

If the PoC fails, what does that mean for the decision? Does it mean “don’t use Kinesis” or “Kinesis needs to be configured differently” or “our Go worker needs to be redesigned”? A failing PoC should give you information, not just a red light.

PoC Design Template

```
Technology:
Use case being tested:

Riskiest assumption:
    "We assumed that [X]. This could be wrong because [Y]."
```

PoC scope (what we build):

- []
- []
- [] (keep it minimal – only what's needed to test the assumption)

Success criterion:

"[Observable behavior] is [measurable threshold] when [conditions]."

Verified by: [how you measure it]

Failure criterion:

If [observable failure], we conclude [specific conclusion].

This means: [go back to RabbitMQ extended / re-evaluate Kafka / reconfigure Kinesis as follows]

Time box: [X days]

Who builds it: [names/roles]

Reference answer (review after completing)

Expand after you've written your own answer

****For Kinesis:**** ****Riskiest assumption:**** "We assumed a single Go worker can replay 90 days of order events from Kinesis within a 4-hour window (auditor SLA), at our projected data volume (~5k orders/day × 90 days = 450k events), without hitting Kinesis shard throughput limits or incurring prohibitive cost." This is the right assumption to test because: - Kinesis pricing is per-shard-hour + data retrieval. 90-day replay at volume might be expensive. - Kinesis shard read limit is 2MB/s per shard. With 1 shard, 450k events at ~500 bytes/event = 225MB. Fine. But if events are larger or volume grows, sharding strategy matters. - The Go worker doesn't exist yet for Kinesis. Integration complexity is unknown. ****PoC scope (minimal):**** - Produce 450,000 synthetic order events to a Kinesis stream (simulating 90 days of history) - Consume all 450,000 events from the beginning using the Go worker - Measure: total replay time, cost (Kinesis pricing calculator or actual AWS bill for test run), consumer throughput ****Success criterion:**** Go worker completes full 90-day replay of 450,000 events in under 4 hours. AWS cost for one replay operation is under \$X (define your budget threshold). Consumer produces correct audit output (verify record count and spot-check event content). ****Failure criterion:**** If replay takes >4 hours or cost exceeds budget — examine whether adding shards reduces time proportionally (if yes: solvable by provisioning more shards; re-evaluate cost). If cost is prohibitive regardless of shards, this is a structural problem with the Kinesis pricing model for this use case — go back and re-evaluate S3-based archive

replay. **Time box:** 3 days. Day 1: set up Kinesis stream, write producer, load test data. Day 2: Go worker integration, run replay, measure. Day 3: analyze results, write PoC report.

Evaluation Matrix Template

Copy this for any technology evaluation:

```
## Technology Evaluation: [Decision Name]
Date: [date]
Decision owner: [name]

### Constraints (these eliminate options)
- [constraint 1]
- [constraint 2]

### Options considered
- Option A: [name] – [one-line description]
- Option B: [name] – [one-line description]
- Option C: [name] – [one-line description]

### Evaluation criteria

| Criterion | Weight (1-5) | Rationale |
|---|---|---|
| [criterion] | | |

### Scoring matrix

| Criterion | Wt | Option A | Score | Option B | Score | Option C | Score |
|---|---|---|---|---|---|---|---|
| [criterion] | | | | | | | |
| Weighted total | | | | | | | |

### Decision

We choose [option] because [primary criteria we're optimizing for].

Key tradeoffs accepted: [what we're sacrificing].

Re-evaluate if: [conditions that would change the decision].
```

Checklist before moving on

- ☐ You can name the 6 steps of the evaluation framework in order without looking
- ☐ You can explain why “stay with current” must always be an option
- ☐ You know the primary decision driver for RabbitMQ vs Kafka vs cloud-native queues (replay + ops capacity)
- ☐ You can distinguish a PoC from a demo and state what a good PoC tests
- ☐ You can write a falsifiable success criterion for a PoC (measurable threshold, observable behavior)
- ☐ You can calculate a weighted score and use it to make a documented, reversible decision